

Analyse und Optimierung von MULTIFIT Algorithmen

Fridjoff Hempel

23. September 2026

Zusammenfassung

Diese Arbeit untersucht MULTIFIT, ein Approximationsverfahren von Coffman, Garey und Johnson (1978) für die Makespan-Minimierung auf identischen Maschinen. Das Verfahren rät eine Schranke an den Makespan und überlässt dem Bin-Packing-Verfahren First Fit Decreasing (FFD) als Subroutine die Frage, ob die Aufgaben in ebenso viele Behälter dieser Größe passen, wie Maschinen vorhanden sind; eine Binärsuche verengt die Schranke anschließend. Im ersten Teil der Arbeit wird die Performanzanalyse von MULTIFIT geschlossen aufbereitet: Die auf Coffman, Garey und Johnson (1978) zurückgehende Schranke $5/4$ für das Grenzverhältnis, an welchem das im schlechtesten Fall erreichte Verhältnis von erzeugtem zu optimalem Makespan hängt, wird vollständig bewiesen und mit dem Makespan nach k Suchschritten verbunden; die schärfere Schranke $13/11$ von Cao (1995) wird eingeordnet, und an einer Instanz von Friesen (1984) wird gezeigt, dass sie sich für mindestens 13 Maschinen nicht verbessern lässt. Im zweiten Teil der Arbeit wird die Subroutine von MULTIFIT ausgetauscht. Für Modified First Fit Decreasing (MFFD) von Johnson und Garey (1985) als Subroutine wird gezeigt, dass es innerhalb von MULTIFIT dieselbe Zuordnung wie FFD liefert, sobald die mittlere Maschinenlast mindestens doppelt so groß ist wie die größte Aufgabe. Der $\text{OPT} + 1$ -Algorithmus von Jansen und Solis-Oba (2010) für Instanzen mit d verschiedenen Aufgabengrößen wird zu einem ε -dualen Verfahren im Sinne von Hochbaum und Shmoys (1987) umgebaut, welches höchstens die optimale Binzahl verwendet und dafür die Kapazität um den Faktor $1 + \varepsilon$ überschreiten darf; MULTIFIT erreicht damit einen Makespan von höchstens $(1 + \varepsilon)(1 + 2^{-k})$ mal dem Optimum, wobei $\varepsilon = 1/(2d - 1)$ ist. Für das Konfigurations-LP von Gilmore und Gomory (1961) trägt die beim Runden entstehende Schranke dieselbe Schlussweise dagegen nicht. Sämtliche untersuchten Verfahren sind implementiert und praktisch getestet worden. Auf 9092 Benchmark-Instanzen überschreitet kein gemessenes Verhältnis zum optimalen Makespan den Wert $13/11$, und MFFD liefert keinen systematisch kleineren Makespan als FFD. Auf 1080 Instanzen mit wenigen verschiedenen Aufgabengrößen liefert MULTIFIT mit dem $\text{OPT} + 1$ -Algorithmus auf 453 Instanzen einen kleineren, auf 8 jedoch einen größeren Makespan als mit FFD. Eine bessere Bin-Packing-Garantie sichert somit keinen kleineren Makespan; dafür kommt es darauf an, bei welchen der getesteten Schranken die Subroutine mit der Maschinenzahl auskommt und, wo sie die Schranke überschreiten darf, wie weit sie das tut.

Inhaltsverzeichnis

1	Einleitung	5
2	Grundlagen	8
2.1	Scheduling auf identischen Maschinen	8
2.2	Bin Packing und FFD	9
2.3	MULTIFIT	9
3	Performanzanalyse von MULTIFIT	12
3.1	Einordnung und Chronologie	12
3.2	Minimale Gegenbeispiele	12
3.3	Löschlemma	13
3.4	Dominanzlemma	14
3.5	Aufgaben je Bin der q -Packung	15
3.6	Größenschränken	16
3.7	Die Schranke $r_m \leq 5/4$	16
3.8	Von r_m zur Güte von MULTIFIT	17
3.9	Beweisidee der Schranke 13/11	18
3.10	Untere Schranke	19
4	Alternative Subroutinen	20
4.1	MFFD als Subroutine in MULTIFIT	20
4.1.1	Der Algorithmus MFFD	20
4.1.2	Kollaps von MFFD auf FFD	21
4.2	OPT+1-Algorithmus von Jansen und Solis-Oba	23
4.2.1	Laufzeit	23
4.2.2	Implementierung	24
4.2.3	Theoretische Anwendbarkeit als MULTIFIT-Subroutine	26
4.2.4	Modifikation zu einem ε -dualen Approximationsalgorithmus	27
4.3	Das Konfigurations-LP von Gilmore und Gomory als Subroutine	31
4.3.1	Das Konfigurationsmodell	31
4.3.2	Eigenschaften des Konfigurations-LPs	32
4.3.3	Rundung zu einer zulässigen Packung	33
4.3.4	Einsatz in MULTIFIT	34
4.3.5	Implementierung	35
5	Empirische Evaluation	37
5.1	FFD gegen MFFD	37
5.1.1	Benchmark-Datensätze	37

5.1.2	Versuchsaufbau	41
5.1.3	Kollaps von MFFD auf FFD	42
5.1.4	Makespan-Vergleich	43
5.1.5	Güte gegenüber dem Optimum	44
5.1.6	Laufzeit	46
5.1.7	Sensitivität gegenüber der Binärsuchtiefe	47
5.1.8	Fallstudie: die Friesen-Instanz	49
5.1.9	Grenzen der Auswertung	50
5.2	High-Multiplicity-Subroutinen	51
5.2.1	Datensätze	51
5.2.2	Versuchsaufbau	52
5.2.3	Güte gegenüber dem Optimum	53
5.2.4	Die Garantien im Test	56
5.2.5	Laufzeit	58
5.2.6	Skalierung in der Multiplizität	59
5.2.7	Einschränkungen dieses Vergleichs	61
6	Fazit	62

1 Einleitung

Ein Betrieb verteilt eingehende Aufträge auf mehrere gleich schnelle Maschinen, ein Rechenzentrum verteilt Rechenaufträge auf die Knoten eines Clusters. In beiden Fällen ist dieselbe Frage zu beantworten: Welche Aufgabe gehört auf welche Maschine? Entscheidend ist dabei nicht die Summe der Bearbeitungszeiten, sondern der Zeitpunkt, zu dem die zuletzt beschäftigte Maschine ihre Arbeit beendet. Dieser Zeitpunkt heißt *Makespan*, und seine Minimierung ist der Gegenstand dieser Arbeit.

Formal sind n Aufgaben mit bekannten Bearbeitungszeiten und m identische Maschinen gegeben. Jede Aufgabe läuft ohne Unterbrechung auf genau einer Maschine; gesucht ist eine Zuordnung mit kleinstmöglichem Makespan. In der Notation von Graham u. a. [Gra+79] trägt dieses Problem den Namen $P \parallel C_{\max}$. Kapitel 2 führt diese Notation zusammen mit den übrigen Bezeichnungen ein. Das zugehörige Entscheidungsproblem ist NP-vollständig [GJ79]; sofern $P \neq NP$ gilt, berechnet folglich kein Algorithmus mit polynomieller Laufzeit den minimalen Makespan. Gefragt sind deshalb Verfahren, welche schnell eine Zuordnung liefern und deren *Approximationsgüte* sich beweisen lässt. Diese Güte ist der Faktor, um den der erzeugte Makespan den optimalen im schlechtesten Fall höchstens übersteigt.

Bekannte Ansätze. Der erste Algorithmus mit einer solchen Garantie stammt von Graham [Gra66]; mit dem Makespan-Problem nimmt die Analyse beweisbar guter Verfahren ihren Ausgang [HS87, S. 145]. Sein List-Scheduling durchläuft die Aufgaben in einer gegebenen Reihenfolge und legt jede auf die gerade am wenigsten belastete Maschine; der erzeugte Makespan übersteigt das Optimum dabei um höchstens den Faktor $2 - 1/m$. Dieser Faktor ist *scharf*, das heißt, kleiner lässt er sich nicht wählen: Es gibt Instanzen, auf welchen ihm das Verhältnis von erzeugtem zu optimalem Makespan beliebig nahe kommt [Gra66, S. 1571]. Werden die Aufgaben zuvor absteigend sortiert, so sinkt der Faktor auf $4/3 - 1/(3m)$, und auch dieser Wert ist scharf [Gra69, Theorem 2]. Diese Regel ist unter dem Namen *Longest Processing Time* bekannt.

Einen anderen Weg gehen Coffman, Garey und Johnson [CGJ78] mit MULTIFIT. Statt die Aufgaben unmittelbar zu verteilen, rät das Verfahren eine Schranke C an den Makespan und überlässt einem Bin-Packing-Algorithmus die Frage, ob die Aufgaben in m Behälter der Kapazität C passen. Beides ist dieselbe Frage, denn eine solche Packung ist genau eine Zuordnung, bei der keine Maschine länger als C arbeitet. Dieser Algorithmus heißt im Folgenden *Subroutine*; Coffman, Garey und Johnson [CGJ78] setzen dafür First Fit Decreasing (FFD) ein, welches die Aufgaben absteigend sortiert und jede in den ersten Behälter legt, in den sie noch passt. Die Packung erzeugt damit die Subroutine; eine Binärsuche über C , nach k Halbierungsschritten abgebrochen, wählt unter den akzeptierten Schranken die kleinste aus und gibt deren Packung zurück. Die Güte von MULTIFIT

hängt damit an einer rein kombinatorischen Größe, dem *Grenzverhältnis* r_m : Übersteigt eine Kapazität den optimalen Makespan um mehr als den Faktor r_m , so kommt FFD mit m Behältern aus; diese Folgerung aus Definition 2.7 ist Implikation (2.3). Für diesen Wert weisen Coffman, Garey und Johnson [CGJ78] zunächst $r_m \leq 1,22$ nach. Über Friesen [Fri84] und Yue [Yue90] ist die Schranke später auf $13/11$ verschärft worden, und Cao [Cao95] gibt dafür einen vollständigen Beweis; Abschnitt 3.1 stellt diese Entwicklung im Einzelnen dar.

Beliebig nah an das Optimum führen Approximationsschemata: Zu jeder vorgegebenen Genauigkeit $\varepsilon > 0$ liefern sie eine Zuordnung mit einem Makespan von höchstens $(1 + \varepsilon)$ mal dem Optimum. Für eine feste Maschinenzahl gibt Sahni [Sah76] ein solches Schema an, und Hochbaum und Shmoys [HS87] behandeln den Fall beliebiger Maschinenzahl über *duale* Approximation, bei der nicht die Zahl der Behälter, sondern deren Kapazität überschritten werden darf. Beahlt wird die Genauigkeit mit Laufzeit: Das Schema von Hochbaum und Shmoys läuft in $O((n/\varepsilon)^{1/\varepsilon^2})$, was die Autoren selbst als nicht praktikabel bezeichnen [HS87, S. 146]. MULTIFIT liegt zwischen beiden Linien, denn seine Güte ist fest, jeder Suchschritt kostet aber nur einen einzigen Aufruf der Subroutine. Einmaliges Sortieren und $k + 1$ solche Aufrufe ergeben eine Laufzeit in $O(k n \log n)$, bei fester Suchtiefe also $O(n \log n)$ (Abschnitt 2.3).

Verbessert worden ist MULTIFIT auch, ohne die Subroutine anzutasten. Friesen und Langston [FL86] ersetzen den Zweig, welcher eine Kapazität verwirft, durch eine Reparatur: Eine beschränkte Zahl besonders ungünstig belegter Maschinen wird zusammen mit den dabei übrig gebliebenen Aufgaben optimal neu belegt. Das Verhältnis von erzeugtem zu optimalem Makespan sinkt dadurch auf $72/61$ und liegt damit unter $13/11$; asymptotisch ist dieser Wert scharf, und die Laufzeitordnung $O(n \log n)$ bleibt erhalten. Diese Arbeit geht den anderen Weg und lässt die Suche unangetastet.

Welche Subroutine dort steht, ist ein eigener Strang der Literatur. FFD benötigt höchstens $11/9$ mal so viele Behälter wie eine optimale Packung, zuzüglich einer additiven Konstanten; diese Schranke ist scharf [Dós07]. Für Modified First Fit Decreasing (MFFD) ist mit $71/60$ eine kleinere obere Schranke bewiesen [JG85; YZ95]. Da $11/9$ scharf ist, fällt das asymptotische Worst-Case-Verhältnis von MFFD damit echt kleiner aus als jenes von FFD; ob MFFD den Wert $71/60$ überhaupt annimmt, ist dagegen offen. Kommen in einer Instanz nur wenige verschiedene Aufgabengrößen vor, so erreicht der OPT + 1-Algorithmus von Jansen und Solis-Oba [JS10] sogar eine Packung mit höchstens einem Behälter über dem Optimum; sein Name benennt genau diese Garantie. Ob ein solcher Vorteil den Makespan von MULTIFIT verkleinert, ist damit allerdings nicht gesagt: Deswegen Gütegarantie beruht auf einer bestimmten Eigenschaft von FFD, welche eine schärfere Bin-Packing-Schranke nicht ersetzt.

Daneben stehen exakte Verfahren, etwa die Arc-Flow-Modelle von Mrad und Souayah [MS18] oder die Suchverfahren von Akram u. a. [Akr+25]. Sie liefern den minimalen Makespan, besitzen im schlechtesten Fall aber keine polynomielle Laufzeit; in Kapitel 5 dienen ihre publizierten Werte als Vergleichsgrundlage.

Beitrag dieser Arbeit. Die Arbeit verfolgt zwei Ziele. Das erste ist eine geschlossene Aufbereitung der Performanzanalyse von MULTIFIT. Cao [Cao95] dient dabei als primäre Quelle, setzt die Ergebnisse von Coffman, Garey und Johnson [CGJ78] jedoch über weite Strecken voraus und führt einzelne Beweisschritte nur knapp aus. Kapitel 3 führt deshalb die Herleitung der Schranke $r_m \leq 5/4$ vollständig aus und verbindet sie in Theorem 3.12 mit dem Makespan, den MULTIFIT nach k Suchschritten zurückgibt. Die schärfere Schranke $13/11$ wird eingeordnet und ihre Beweisidee dargestellt, nicht aber bewiesen; Caos Fallunterscheidung bleibt außerhalb des Umfangs dieser Arbeit. Die zugehörige untere Schranke wird dagegen geführt: Eine Instanz von Friesen [Fri84] belegt $r_{13} \geq 13/11$, und durch Auffüllen überträgt sich das auf jedes $m \geq 13$.

Das zweite Ziel betrifft die Subroutine, denn innerhalb von MULTIFIT ist sie austauschbar. Kapitel 4 untersucht drei Alternativen theoretisch. Zwei Ergebnisse daraus sind eigenständige Beiträge. Zum einen gibt das Kollaps-Lemma eine Klasse von Instanzen an, auf der MFFD und FFD innerhalb von MULTIFIT stets dieselbe Zuordnung erzeugen. Zum anderen wird der OPT + 1-Algorithmus von Jansen und Solis-Oba [JS10] zu einem ε -dualen Approximationsalgorithmus umgebaut. Zusammen mit dem Ergebnis von Hochbaum und Shmoys [HS87] garantiert dieser Umbau nach k Suchschritten einen Makespan von höchstens $(1 + \varepsilon)(1 + 2^{-k})$ mal dem Optimum. Als dritte Alternative dient das Konfigurations-LP von Gilmore und Gomory [GG61], welches jede mögliche Füllung eines Behälters als eigene Variable führt und die Ganzzahligkeit zunächst fallen lässt. Für dieses Programm wird umgekehrt gezeigt, dass die beim Runden entstehende Schranke die Schlussweise hinter $13/11$ nicht unmittelbar trägt.

Was der Austausch praktisch bewirkt, misst Kapitel 5 in zwei Versuchsreihen. Die erste vergleicht FFD und MFFD auf 9 092 Instanzen aus fünf publizierten Benchmark-Datensätzen, die zweite vier Subroutinen für Instanzen mit wenigen verschiedenen Aufgaben Größen auf 1 080 eigens erzeugten Instanzen. Kein gemessenes Verhältnis überschreitet dabei $13/11$ - für MULTIFIT+FFD liegt dieser Wert knapp unter der bewiesenen Schranke $13/11 + 2^{-k}$, für die übrigen Kombinationen ist er lediglich eine Vergleichslinie. Zugleich liefern beide Reihen Instanzen, auf denen die schärfere Bin-Packing-Garantie den größeren Makespan liefert; sie sichert also keinen kleineren.

Aufbau. Kapitel 2 legt die Grundlagen: Problemstellung und Notation, Bin Packing mit FFD sowie der Algorithmus MULTIFIT samt seinem Startintervall. Kapitel 3 enthält die Performanzanalyse, Kapitel 4 die drei alternativen Subroutinen und Kapitel 5 die empirische Auswertung. Kapitel 6 fasst die Ergebnisse zusammen und benennt die offen gebliebenen Fragen. Sämtliche Implementierungen und Messskripte liegen unter <https://github.com/hempl111/MultiFit-BscThs>; den Versuchsaufbau beschreibt Abschnitt 5.1.2.

2 Grundlagen

2.1 Scheduling auf identischen Maschinen

Definition 2.1 (Instanz und Makespan). Eine *Instanz* Γ besteht aus n Aufgaben mit Größen $p_1, \dots, p_n > 0$. Eine *Zuordnung* von Γ auf m Maschinen ist ein Tupel $P = (P_1, \dots, P_m)$ paarweise disjunkter Mengen mit

$$P_1 \uplus P_2 \uplus \dots \uplus P_m = \{1, \dots, n\}.$$

Die *Last* einer Maschine i ist $l_i = \sum_{j \in P_i} p_j$, der *Makespan* der Zuordnung ist $C_{\max}(P) = \max_{1 \leq i \leq m} l_i$. Der kleinste Makespan einer Zuordnung von Γ auf m Maschinen wird mit $C_{\max}^*(\Gamma, m)$ bezeichnet.

Zu einer Eingabe des Scheduling-Problems gehören damit stets beide Angaben: die Aufgabenliste Γ und die Maschinenzahl m . Sie werden hier getrennt geführt und nicht zu einem Objekt zusammengefasst, weil dieselbe Aufgabenliste in Abschnitt 2.2 noch einmal auftritt - dort als Bin-Packing-Instanz, zu der keine Maschinenzahl gehört. Jede Aussage nennt die Maschinenzahl deshalb ausdrücklich, sobald sie gebraucht wird.

In der Drei-Feld-Notation $\alpha \mid \beta \mid \gamma$ von Graham u. a. [Gra+79] heißt dieses Problem $P \parallel C_{\max}$: Das Feld $\alpha = P$ steht für identische parallele Maschinen, das leere Feld β dafür, dass an die Aufgaben keine weiteren Einschränkungen gestellt werden, und $\gamma = C_{\max}$ für den Makespan als Zielgröße. Das P dieser Notation bezeichnet die Maschinenumgebung und ist von der Zuordnung P aus Definition 2.1 zu unterscheiden. Die Aufgaben werden dabei weder unterbrochen noch auf mehrere Maschinen aufgeteilt.

Das zugehörige Entscheidungsproblem - gibt es zu einer gegebenen Schranke $T > 0$ eine Zuordnung mit Makespan höchstens T ? - ist NP-vollständig [GJ79]. Sofern $P \neq NP$ gilt, berechnet daher kein Algorithmus mit polynomieller Laufzeit den Wert $C_{\max}^*(\Gamma, m)$. Bekannt sind aber die beiden folgenden Schranken, auf die sich die Kapitel 4 und 5 wiederholt stützen. Dabei bezeichnet $p_{\max} := \max_j p_j$ durchgehend die größte Aufgabengröße.

Lemma 2.2 (Untere Schranken). Für jede Instanz Γ und jede Maschinenzahl $m \geq 1$ gilt

$$C_{\max}^*(\Gamma, m) \geq \max\left(\frac{1}{m} \sum_j p_j, p_{\max}\right).$$

Beweis. Jede Zuordnung verteilt die Gesamtlast $\sum_j p_j$ auf m Maschinen, sodass mindestens eine Maschine eine Last von mindestens $\frac{1}{m} \sum_j p_j$ trägt. Ferner liegt die größte Aufgabe in genau einem P_i , dessen Last l_i damit mindestens $p_{\max}$ beträgt. Beides gilt für jede Zuordnung, insbesondere für eine mit minimalem Makespan. $\square$

Definition 2.3 (Approximationsgüte). Sei A ein Algorithmus, der zu einer Instanz Γ und einer Maschinenzahl m eine Zuordnung berechnet, und sei $C_{\max}^A(\Gamma, m)$ deren Makespan. Die Güte von A auf m Maschinen ist

$$R_m(A) = \sup \left\{ C_{\max}^A(\Gamma, m) / C_{\max}^*(\Gamma, m) \mid \Gamma \text{ Instanz} \right\}.$$

Steht die Maschinenzahl im Kontext fest, so wird das zweite Argument weggelassen und kurz $C_{\max}^*(\Gamma)$ beziehungsweise $C_{\max}^A(\Gamma)$ geschrieben.

2.2 Bin Packing und FFD

Definition 2.4 (Bin Packing). Zu einer Instanz Γ und einer Kapazität $C > 0$ fragt das Bin-Packing-Problem nach der kleinsten Anzahl von Bins der Kapazität C , auf die sich die Aufgaben so verteilen lassen, dass kein Bin eine Last über C trägt. Diese Anzahl wird mit $\text{OPT}[\Gamma, C]$ bezeichnet.

Bin Packing kommt ohne Maschinenzahl aus: Gesucht ist die Anzahl der Bins, nicht eine Verteilung auf eine vorgegebene Anzahl von Maschinen. Beide Probleme arbeiten deshalb auf derselben Instanz Γ ; über die Maschinenzahl sind sie zueinander dual.

Lemma 2.5 (Dualität). Sei Γ eine Instanz, $m \geq 1$ eine Maschinenzahl und $C, C' > 0$ Kapazitäten. Dann gilt $C_{\max}^*(\Gamma, m) \leq C$ genau dann, wenn $\text{OPT}[\Gamma, C] \leq m$. Ferner ist $C \mapsto \text{OPT}[\Gamma, C]$ monoton fallend, das heißt aus $C \leq C'$ folgt $\text{OPT}[\Gamma, C'] \leq \text{OPT}[\Gamma, C]$.

Beweis. Eine Zuordnung mit Makespan höchstens C verteilt die Aufgaben auf m Mengen mit Last jeweils höchstens C , ist also eine Packung in höchstens m Bins der Kapazität C ; leere Maschinen entsprechen ungenutzten Bins. Umgekehrt liefert eine Packung in höchstens m solche Bins eine Zuordnung auf m Maschinen mit Makespan höchstens C . Die Monotonie folgt, da jede Packung mit Kapazität C auch für jede Kapazität $C' \geq C$ zulässig ist. $\square$

First Fit legt jede Aufgabe in einer gegebenen Reihenfolge in den ersten Bin, in den sie passt, und eröffnet einen neuen Bin, falls keiner passt. *First Fit Decreasing* (FFD) wendet First Fit auf die absteigend sortierte Aufgabenliste an. Mit $\text{FFD}[\Gamma, C]$ wird die Anzahl der dabei erzeugten Bins bezeichnet. Es gilt

$$\text{FFD}[\Gamma, C] \leq \frac{11}{9} \text{OPT}[\Gamma, C] + \frac{6}{9}, \quad (2.1)$$

und diese Schranke ist scharf [Dós07].

2.3 MULTIFIT

MULTIFIT [CGJ78] löst $P \parallel C_{\max}$ nicht direkt, sondern führt eine Binärsuche über die Kapazität C aus und ruft in jeder Iteration FFD auf. Ausgehend von

$$c_{\text{low}}^{(0)} = \max\left(\frac{1}{m} \sum_j p_j, p_{\max}\right), \quad c_{\text{high}}^{(0)} = \frac{1}{m} \sum_j p_j + p_{\max} \quad (2.2)$$

wird k -mal die Intervallmitte $C = \frac{1}{2}(c_{\text{low}} + c_{\text{high}})$ getestet: Gilt $\text{FFD}[\Gamma, C] \leq m$, so wird $c_{\text{high}} := C$ gesetzt und die erzeugte Packung gespeichert, andernfalls $c_{\text{low}} := C$. Zurückgegeben wird die zuletzt gespeicherte Packung; ihr Makespan wird mit $C_{\text{max}}^{\text{MF}}(\Gamma)$ bezeichnet, die Variante mit k Halbierungsschritten mit $\text{MF}(k)$. Ein Aufruf mit $c_{\text{high}}^{(0)}$ vor der Schleife stellt sicher, dass stets eine gespeicherte Packung vorliegt; er ist der Rückfall für den Fall, dass alle k Tests scheitern, und lässt sich ebenso ans Ende verschieben. Als obere Grenze wählen Coffman, Garey und Johnson [CGJ78] ursprünglich $\max(\frac{2}{m} \sum_j p_j, p_{\text{max}})$ [FL86, S. 37]; diese Arbeit verwendet durchgehend die Grenze aus (2.2).

Die Laufzeit setzt sich aus $k + 1$ Aufrufen der Subroutine zusammen: den k Tests der Schleife und dem Aufruf für den Rückfall. Die Aufgabenliste wird dabei einmal sortiert, und jeder Aufruf kostet mit einer geeigneten Datenstruktur für First Fit $O(n \log n)$. Insgesamt ergibt das $O(k n \log n)$, bei fester Suchtiefe k also $O(n \log n)$ [FL86, S. 35].

Dass beide Grenzen aus (2.2) zulässig gewählt sind, hält das folgende Lemma fest: Die untere Grenze überschreitet den gesuchten Wert nicht, und an der oberen gelingt der Test.

Lemma 2.6 (Startintervall). Für jede Instanz Γ und jede Maschinenzahl $m \geq 1$ gilt

- (a) $c_{\text{low}}^{(0)} \leq C_{\text{max}}^*(\Gamma, m)$,
- (b) $\text{FFD}[\Gamma, C] \leq m$ für jedes $C \geq c_{\text{high}}^{(0)}$ und
- (c) $C_{\text{max}}^*(\Gamma, m) \leq c_{\text{high}}^{(0)}$.

Beweis. Aussage (a) ist Lemma 2.2. Für (b) sei angenommen, FFD eröffne bei Kapazität C einen $(m + 1)$ -ten Bin, und zwar für die Aufgabe p_j . Zu diesem Zeitpunkt existieren genau m Bins mit Lasten $l_1, \dots, l_m$, und p_j passt in keinen davon, also $l_i + p_j > C$ für alle i und damit $\sum_{i=1}^m l_i > m(C - p_j)$. Da p_j noch nicht platziert ist, gilt zugleich $\sum_{i=1}^m l_i \leq \sum_{t \neq j} p_t$. Beides zusammen ergibt

$$C < \frac{1}{m} \sum_t p_t + \frac{m-1}{m} p_j \leq \frac{1}{m} \sum_t p_t + p_{\text{max}} = c_{\text{high}}^{(0)}.$$

Für $C \geq c_{\text{high}}^{(0)}$ ist das ausgeschlossen, es entsteht also kein $(m + 1)$ -ter Bin. Aussage (c) folgt aus (b): Bei Kapazität $c_{\text{high}}^{(0)}$ erzeugt FFD höchstens m Bins, also ist $\text{OPT}[\Gamma, c_{\text{high}}^{(0)}] \leq m$ und damit $C_{\text{max}}^*(\Gamma, m) \leq c_{\text{high}}^{(0)}$ nach Lemma 2.5. $\square$

Die Güte von MULTIFIT wird über den folgenden Wert ausgedrückt, der die Tiefe der Binärsuche ausklammert und allein das Verhältnis von FFD zum Optimum erfasst.

Definition 2.7 (Grenzverhältnis r_m , nach [Cao95, S. 79]). Eine Instanz Γ heißt (p/q) -Gegenbeispiel für m Maschinen, wenn $\text{FFD}[\Gamma, p] > m \geq \text{OPT}[\Gamma, q]$ gilt. Es sei

$$r_m = \sup\{p/q \mid \text{es gibt ein } (p/q)\text{-Gegenbeispiel für } m \text{ Maschinen}\}.$$

Kapitel 3 beweist die Schranke $r_m \leq 5/4$, ordnet die schärfere Schranke $r_m \leq 13/11$ in die Entwicklung der bekannten Ergebnisse ein und erläutert die Beweisidee von Cao [Cao95]; dieser Beweis selbst wird dort nicht in voller Länge geführt. Ferner wird gezeigt,

dass $13/11$ für $m \geq 13$ nicht verbessert werden kann. Ebenfalls dort behandelt wird, wie aus r_m die Güte $R_m(\text{MF}(k)) \leq r_m + 2^{-k}$ folgt; der additive Term geht auf die endliche Tiefe der Binärsuche zurück.

Für die Binärsuche ist die folgende Konsequenz von Definition 2.7 entscheidend:

$$C > r_m \cdot C_{\max}^*(\Gamma, m) \implies \text{FFD}[\Gamma, C] \leq m. \quad (2.3)$$

Zum Beweis sei $q := C_{\max}^*(\Gamma, m)$ und $C > r_m q$. Nach Lemma 2.5 gilt $\text{OPT}[\Gamma, q] \leq m$, denn q ist der optimale Makespan. Wäre nun $\text{FFD}[\Gamma, C] > m$, so erfüllte Γ beide Bedingungen aus Definition 2.7 und wäre ein (C/q) -Gegenbeispiel für m Maschinen. Da r_m das Supremum aller Quotienten solcher Gegenbeispiele ist, folgte $C/q \leq r_m$ - im Widerspruch zu $C > r_m q$. Also gilt $\text{FFD}[\Gamma, C] \leq m$.

MULTIFIT akzeptiert damit jede Kapazität oberhalb von $r_m \cdot C_{\max}^*(\Gamma, m)$ und kann seine obere Intervallgrenze bis dorthin absenken. Das ist die Eigenschaft, auf der die Schranke aus Kapitel 3 beruht - und die eine andere Subroutine an Stelle von FFD ebenfalls erfüllen muss, wenn dieselbe Schlussweise greifen soll.

Bemerkung 2.8 (Nicht-Monotonie von FFD). Die Binärsuche sucht keine Schwelle, denn $C \mapsto \text{FFD}[\Gamma, C]$ fällt nicht monoton. Für $\Gamma = (40, 39, 38, 36, 23, 22, 20, 19, 18, 15, 13, 13, 5)$ und $m = 4$ etwa gilt $\text{FFD}[\Gamma, 76] = 4$, aber $\text{FFD}[\Gamma, 77] = 5$. Bei $C = 76$ ist $39 + 38 = 77 > 76$; die Aufgabe 38 eröffnet daher einen dritten Bin, und es entstehen die Lasten $(76, 75, 75, 75)$. Bei $C = 77$ passen 39 und 38 gemeinsam in den zweiten Bin, der damit voll ist; am Ende bleibt eine 13er-Aufgabe allein übrig, und es entstehen die Lasten $(76, 77, 70, 65, 13)$. Eine größere Kapazität kann also mehr Bins erfordern. Weshalb die Binärsuche die Schranke aus Kapitel 3 dennoch einhält, wird dort ausgearbeitet.

3 Performanzanalyse von MULTIFIT

3.1 Einordnung und Chronologie

Coffman, Garey und Johnson [CGJ78] führen MULTIFIT ein und zeigen zweierlei: dass sich die Güte des Verfahrens auf das Grenzverhältnis zurückführen lässt, $R_m(\text{MF}(k)) \leq r_m + 2^{-k}$, und dass $r_m \leq 1,22$ gilt.

Friesen [Fri84] verschärft die zweite Aussage auf $r_m \leq 6/5$ und gibt zugleich eine Instanz für $m = 13$ an, die das Verhältnis $13/11$ erzwingt. Damit ist $13/11$ als untere Schranke fixiert und der verbleibende Spielraum auf das Intervall zwischen $13/11 \approx 1,182$ und $6/5 = 1,2$ eingengt; die Instanz wird in Abschnitt 3.10 behandelt. Für die daraus folgende Güteschranke $R_m(\text{MF}(k)) \leq 6/5 + 2^{-k}$ geben Yue, Kellerer und Yu [YKY88] einen einfachen Beweis an.

Den Spielraum zu schließen beansprucht zuerst Yue [Yue90] mit dem Nachweis $r_m \leq 13/11$. Nach Darstellung von Cao [Cao95] bleiben in diesem Beweis Fälle unbehandelt; Cao gibt daher einen eigenen, vollständigen Beweis derselben Schranke - die Arbeit, die dieser Bachelorarbeit als primäre Quelle dient.

Alle genannten Werte sind obere Schranken an r_m ; scharf ist von ihnen nur der letzte, und zwar für jede Maschinenzahl $m \geq 13$. Die Instanz von Friesen [Fri84] erreicht ihn für $m = 13$ und lässt sich durch Auffüllen auf größere Maschinenzahlen übertragen - beides in Abschnitt 3.10. Für $m < 13$ ist damit nur die obere Schranke $r_m \leq 13/11$ aus [Cao95] gesichert; über den genauen Wert trifft diese Arbeit dort keine Aussage. Zu beachten ist außerdem, dass r_m nach Definition 2.7 kein Verhältnis von Makespans ist, sondern das Supremum der Quotienten p/q über alle Gegenbeispiele; wie daraus eine Aussage über den von MULTIFIT erzeugten Makespan wird, zeigt Abschnitt 3.8.

3.2 Minimale Gegenbeispiele

Die Herleitung der Schranke $r_m \leq 5/4$ in den Abschnitten 3.2 bis 3.7 geht auf Coffman, Garey und Johnson [CGJ78] zurück. Von dort stammen die Beweistechnik - Annahme eines kleinstmöglichen Gegenbeispiels und Widerspruch aus dessen Struktur - sowie die Struktureigenschaften, welche die folgenden Lemmata festhalten; die Arbeit führt sie zu der schärferen Schranke $r_m \leq 1,22$ fort, deren zusätzliche Argumente hier nicht dargestellt werden. Cao [Cao95] übernimmt diese Grundlage ausdrücklich („We follow the definitions and notations in this paper“, [Cao95, S. 79]) und setzt etwa die Dominanz eines Bins der optimalen Packung durch einen FFD-Bin [Cao95, S. 88] sowie die untere Schranke $p - q$ für die kleinste Aufgabe [Cao95, S. 80] ohne Beweis voraus; weitere Lemmata führt er auf Yue [Yue90] zurück. Formulierung und Ausführung der Beweise sind

eigene Ausarbeitung.

Im Folgenden seien $p > q > 0$ fest. Die Aufgaben einer Instanz Γ seien absteigend nummeriert, also $p_1 \geq \dots \geq p_n$; FFD bearbeitet sie dann in dieser Reihenfolge. Mit $|\Gamma| = n$ wird die Anzahl der Aufgaben bezeichnet und mit Γ_J für $J \subseteq \{1, \dots, n\}$ die Instanz aus den Aufgaben $(p_j)_{j \in J}$.

Definition 3.1 (Minimales Gegenbeispiel, nach [CGJ78]). Ein (p/q) -Gegenbeispiel Γ für m Maschinen im Sinne von Definition 2.7 heißt *minimal*, wenn keine Instanz Γ' mit $|\Gamma'| < |\Gamma|$ ein (p/q) -Gegenbeispiel für irgendeine Maschinenzahl m' ist.

Die Minimalität bezieht sich also auf die Aufgabenzahl und läuft über alle Maschinenzahlen zugleich. Da $|\Gamma|$ eine natürliche Zahl ist, existiert zu jedem (p/q) -Gegenbeispiel auch ein minimales. Cao [Cao95] minimiert dagegen bei fester Maschinenzahl und fordert dafür zusätzlich $r_{m'} \leq p/q$ für alle $m' < m$; die hier gewählte Fassung kommt ohne diese Bedingung aus.

Bemerkung 3.2 (Teilinstanzen). Für jede Instanz Γ , jedes $J \subseteq \{1, \dots, n\}$ und jede Kapazität $C > 0$ gilt $\text{OPT}[\Gamma_J, C] \leq \text{OPT}[\Gamma, C]$. Streicht man aus einer Packung von Γ alle Aufgaben außerhalb von J , so entsteht eine Packung von Γ_J mit höchstens ebenso vielen Bins.

Lemma 3.3 (Gestalt eines minimalen Gegenbeispiels, nach [CGJ78]). Sei Γ ein minimales (p/q) -Gegenbeispiel für m Maschinen. Dann gilt

- (a) $\text{FFD}[\Gamma, p] = m + 1$,
- (b) der $(m + 1)$ -te Bin enthält genau die Aufgabe p_n und
- (c) $l_i + p_n > p$ für die Lasten $l_1, \dots, l_m$ der übrigen Bins.

Beweis. FFD platziert die Aufgaben in der Reihenfolge $p_1, \dots, p_n$, und die Platzierung von p_j hängt nur von $p_1, \dots, p_j$ ab. Für $J = \{1, \dots, j\}$ erzeugt FFD auf Γ_J bei Kapazität p daher dieselben Platzierungen wie auf Γ . Wegen $\text{FFD}[\Gamma, p] > m$ existiert ein Index j , für den FFD einen $(m + 1)$ -ten Bin eröffnet; für $J = \{1, \dots, j\}$ gilt dann $\text{FFD}[\Gamma_J, p] = m + 1 > m$. Nach Bemerkung 3.2 ist zudem $\text{OPT}[\Gamma_J, q] \leq \text{OPT}[\Gamma, q] \leq m$, also ist Γ_J ein (p/q) -Gegenbeispiel für m Maschinen. Die Minimalität von Γ erzwingt $|\Gamma_J| = |\Gamma|$ und damit $j = n$. Da p_n zuletzt platziert wird, folgen (a) und (b). Aussage (c) gilt, weil FFD sonst p_n in einen der ersten m Bins gelegt hätte, statt einen neuen zu eröffnen. $\square$

Bemerkung 3.4 (Maschinenzahl). In einem (p/q) -Gegenbeispiel für m Maschinen gilt $m \geq 2$. Für $m = 1$ folgt nämlich aus $\text{OPT}[\Gamma, q] \leq 1$ zunächst $\sum_j p_j \leq q$ und wegen $q < p$ dann $\text{FFD}[\Gamma, p] = 1$, im Widerspruch zu $\text{FFD}[\Gamma, p] > 1$. Da $p > q$ beliebig war, treten für $m = 1$ nur Gegenbeispiele mit $p \leq q$ auf; es gilt also $r_1 \leq 1$.

3.3 Löschlemma

Zu einer Instanz Γ und einer Kapazität $C > 0$ bezeichne $B_1, \dots, B_N \subseteq \{1, \dots, n\}$ die von FFD erzeugten Bins, nummeriert in der Reihenfolge ihrer Eröffnung. Für $j \in \{1, \dots, n\}$ sei $B_i^{<j} = B_i \cap \{1, \dots, j - 1\}$ der Inhalt des i -ten Bins unmittelbar vor der Platzierung

von p_j und $l_i^{<j} = \sum_{t \in B_i^{<j}} p_t$ dessen Last. Da die Bins in aufsteigender Nummerierung eröffnet werden, sind zu diesem Zeitpunkt genau die Bins $B_1, \dots, B_{t(j)}$ mit $t(j) = \max\{i : B_i^{<j} \neq \emptyset\}$ vorhanden. FFD legt p_j in den Bin mit dem kleinsten Index $i \leq t(j)$, für den $l_i^{<j} + p_j \leq C$ gilt; existiert kein solcher, so wird $B_{t(j)+1}$ eröffnet.

Lemma 3.5 (Löschlemma, nach [CGJ78]). Sei Γ eine Instanz, $C > 0$ eine Kapazität, seien $B_1, \dots, B_N$ die von FFD bei Kapazität C erzeugten Bins und sei $k \in \{1, \dots, N\}$. Für $J = \{1, \dots, n\} \setminus B_k$ erzeugt FFD auf Γ_J bei Kapazität C genau die Bins B_i mit $i \neq k$, eröffnet in aufsteigender Nummerierung. Insbesondere gilt $\text{FFD}[\Gamma_J, C] = \text{FFD}[\Gamma, C] - 1$.

Beweis. Induktion über die Aufgaben aus J in aufsteigender Nummerierung. Behauptet wird: Unmittelbar vor der Platzierung von p_j mit $j \in J$ sind im Lauf auf Γ_J genau die nichtleeren Mengen $B_i^{<j}$ mit $i \neq k$ als Bins vorhanden, und zwar nach aufsteigendem Index i geordnet.

Sei $j_0 = \min J$. Alle Aufgaben mit kleinerem Index liegen in B_k , also ist $B_i^{<j_0} = \emptyset$ für $i \neq k$, und im Lauf auf Γ_J ist noch nichts platziert.

Sei die Behauptung für $j \in J$ erfüllt und i^* der Index des Bins, in den p_j im Lauf auf Γ gelegt wird; wegen $j \notin B_k$ ist $i^* \neq k$. Nach Induktionsvoraussetzung stehen im Lauf auf Γ_J dieselben Bins mit denselben Lasten zur Verfügung, nur ohne B_k . Ist $i^* \leq t(j)$, so gilt $l_{i^*}^{<j} + p_j > C$ für alle $i < i^*$, insbesondere für alle diese i mit $i \neq k$; First Fit wählt also auch dort B_{i^*} . Andernfalls passt p_j in keinen vorhandenen Bin, und beide Läufe eröffnen einen neuen. Die Aufgaben zwischen j und der nächsten Aufgabe aus J liegen sämtlich in B_k und ändern im Lauf auf Γ nur dessen Inhalt; damit gilt die Behauptung auch für die nächste Aufgabe aus J .

Beide Läufe stimmen somit auf allen Aufgaben aus J überein. Jeder eröffnete Bin ist nichtleer, also erzeugt FFD auf Γ_J genau die $N - 1$ Bins B_i mit $i \neq k$. $\square$

3.4 Dominanzlemma

Ist Γ ein (p/q) -Gegenbeispiel für m Maschinen, so gilt $\text{OPT}[\Gamma, q] \leq m$; es gibt also eine Zerlegung $\{1, \dots, n\} = Q_1 \uplus \dots \uplus Q_m$ mit $\sum_{j \in Q_l} p_j \leq q$ für alle l , wobei leere Teile zugelassen sind. Eine solche Zerlegung heißt im Folgenden q -Packung von Γ .

Definition 3.6 (Dominanz, nach [CGJ78]). Seien $A, B \subseteq \{1, \dots, n\}$. Dann wird A von B *dominiert*, wenn es eine injektive Abbildung $\varphi: A \rightarrow B$ mit $p_{\varphi(j)} \geq p_j$ für alle $j \in A$ gibt.

Lemma 3.7 (Dominanzlemma, nach [CGJ78]). Sei Γ ein minimales (p/q) -Gegenbeispiel für m Maschinen, seien $B_1, \dots, B_{m+1}$ die von FFD bei Kapazität p erzeugten Bins und sei $Q_1, \dots, Q_m$ eine q -Packung von Γ . Dann wird kein Q_i von einem B_k dominiert.

Beweis. Angenommen, $\varphi: Q_i \rightarrow B_k$ ist injektiv mit $p_{\varphi(j)} \geq p_j$ für alle $j \in Q_i$.

Zunächst darf $\varphi(x) = x$ für alle $x \in Q_i \cap B_k$ angenommen werden. Ist nämlich $x \in Q_i \cap B_k$ mit $\varphi(x) \neq x$ und liegt x nicht im Bild von φ , so ersetze man $\varphi(x)$ durch x . Andernfalls ist $x = \varphi(y)$ für genau ein $y \in Q_i$ mit $y \neq x$; man ersetze dann $\varphi(y)$

durch $\varphi(x)$ und $\varphi(x)$ durch x . In beiden Fällen bleibt φ injektiv mit Werten in B_k , und wegen $p_{\varphi(x)} \geq p_x = p_{\varphi(y)} \geq p_y$ bleibt die Größenbedingung erfüllt. Da die Anzahl der $x \in Q_i \cap B_k$ mit $\varphi(x) = x$ dabei wächst, ist die Annahme nach endlich vielen Schritten erfüllt.

Damit gilt $\varphi(j) \notin Q_i$ für jedes $j \in Q_i \setminus B_k$: Wäre $\varphi(j) \in Q_i$, so läge $\varphi(j)$ in $Q_i \cap B_k$, also $\varphi(\varphi(j)) = \varphi(j)$, und die Injektivität ergäbe $j = \varphi(j) \in B_k$ im Widerspruch zu $j \notin B_k$.

Sei nun $J = \{1, \dots, n\} \setminus B_k$ und

$$Q'_l = (Q_l \setminus B_k) \cup \{j \in Q_i \setminus B_k : \varphi(j) \in Q_l\} \quad (l \neq i).$$

Diese $m-1$ Mengen sind paarweise disjunkt und überdecken J : Jedes $j \in J$ liegt entweder in einem Q_l mit $l \neq i$ oder in $Q_i \setminus B_k$, und im zweiten Fall ist $\varphi(j)$ nach dem Vorigen in einem Q_l mit $l \neq i$ enthalten. Da $j \mapsto \varphi(j)$ die zweite Menge injektiv nach $Q_l \cap B_k$ abbildet und dabei die Größen nicht verkleinert, gilt

$$\sum_{j \in Q'_l} p_j \leq \sum_{j \in Q_l \setminus B_k} p_j + \sum_{x \in Q_l \cap B_k} p_x = \sum_{j \in Q_l} p_j \leq q,$$

also $\text{OPT}[\Gamma_J, q] \leq m-1$.

Nach dem Löschlemma 3.5 und Lemma 3.3(a) ist zugleich $\text{FFD}[\Gamma_J, p] = \text{FFD}[\Gamma, p] - 1 = m > m-1$, und wegen Bemerkung 3.4 ist $m-1 \geq 1$. Also ist Γ_J ein (p/q) -Gegenbeispiel für $m-1$ Maschinen. Da B_k nicht leer ist, gilt $|\Gamma_J| < |\Gamma|$ - im Widerspruch zur Minimalität von Γ . $\square$

In dieser Form - ein Bin der q -Packing gegen einen FFD-Bin - benutzt Cao [Cao95] die Aussage ebenfalls [Cao95, S. 88].

3.5 Aufgaben je Bin der q -Packing

Lemma 3.8 (Mindestens drei Aufgaben je Bin, nach [CGJ78]). Sei Γ ein minimales (p/q) -Gegenbeispiel für m Maschinen und sei $Q_1, \dots, Q_m$ eine q -Packing von Γ . Dann gilt $|Q_i| \geq 3$ für alle i .

Beweis. Angenommen, es ist $|Q_i| \leq 2$ für ein i . Es wird gezeigt, dass Q_i dann von einem der Bins $B_1, \dots, B_{m+1}$ dominiert wird, im Widerspruch zu Lemma 3.7. Benutzt wird dabei mehrfach, dass wegen der absteigenden Nummerierung $p_t \geq p_j$ für $t < j$ gilt: Jede vor p_j platzierte Aufgabe ist mindestens so groß wie p_j .

Ist $Q_i = \emptyset$, so wird Q_i von B_1 dominiert, denn die leere Abbildung erfüllt Definition 3.6. Ist $Q_i = \{x\}$, so wird Q_i von demjenigen Bin dominiert, der x enthält.

Sei also $Q_i = \{x, y\}$ mit $x < y$, und seien B_a und B_b die Bins mit $x \in B_a$ und $y \in B_b$. Aus $x, y \in Q_i$ folgt $p_x + p_y \leq q < p$.

Im Fall $a = b$ wird Q_i von B_a durch die identische Abbildung dominiert.

Im Fall $b > a$ war B_a bei der Platzierung von y bereits eröffnet, denn er enthält die zuvor platzierte Aufgabe x ; da y nicht dort abgelegt wurde, gilt $l_a^{<y} + p_y > p$. Wäre

$B_a^{<y} = \{x\}$, so folgte $p_x + p_y > p$ im Widerspruch zu $p_x + p_y < p$. Also enthält $B_a^{<y}$ eine Aufgabe $z \neq x$, und wegen $z < y$ ist $p_z \geq p_y$. Die Abbildung $x \mapsto x, y \mapsto z$ zeigt, dass Q_i von B_a dominiert wird.

Im Fall $b < a$ war B_b bei der Platzierung von x bereits eröffnet: Die Bins werden in aufsteigender Nummerierung eröffnet, und B_a existiert zu diesem Zeitpunkt oder wird gerade von x eröffnet. Also ist $B_b^{<x} \neq \emptyset$; für jede Aufgabe $z \in B_b^{<x}$ gilt $z < x$ und damit $p_z \geq p_x$, und wegen $y > x$ ist $z \neq y$. Die Abbildung $x \mapsto z, y \mapsto y$ zeigt, dass Q_i von B_b dominiert wird. $\square$

3.6 Größenschränken

Lemma 3.9 (Größenschränken, nach [CGJ78]). Sei Γ ein minimales (p/q) -Gegenbeispiel für m Maschinen und $s = p_n$ die kleinste Aufgabe. Dann gilt

- (a) $s > \frac{m}{m-1}(p - q)$ und
- (b) $p_j \leq q - 2s$ für alle $j \in \{1, \dots, n\}$.

Beweis. Zu (a): Nach Lemma 3.3 erzeugt FFD bei Kapazität p genau $m + 1$ Bins, der letzte enthält nur $p_n = s$, und für die Lasten der übrigen gilt $l_i + s > p$. Summation über $i = 1, \dots, m$ ergibt

$$\sum_j p_j - s = \sum_{i=1}^m l_i > m(p - s).$$

Andererseits ist $\sum_j p_j \leq mq$, denn eine q -Packung verteilt alle Aufgaben auf m Bins der Kapazität q . Zusammen folgt $mq - s > mp - ms$, also $(m - 1)s > m(p - q)$; wegen $m \geq 2$ (Bemerkung 3.4) darf durch $m - 1$ geteilt werden.

Zu (b): Sei Q_i der Bin einer q -Packung mit $j \in Q_i$. Nach Lemma 3.8 enthält Q_i neben j mindestens zwei weitere Aufgaben, deren Größen jeweils mindestens s betragen. Aus $\sum_{t \in Q_i} p_t \leq q$ folgt daher $p_j \leq q - 2s$. $\square$

3.7 Die Schranke $r_m \leq 5/4$

Theorem 3.10 (nach [CGJ78]). Für jede Maschinenzahl $m \geq 1$ gilt $r_m \leq 5/4$.

Beweis. Es wird gezeigt, dass es zu $p > \frac{5}{4}q$ kein (p/q) -Gegenbeispiel gibt; nach Definition 2.7 folgt daraus $r_m \leq 5/4$ für jedes m .

Angenommen, es gäbe eines. Dann gibt es auch ein minimales; sei Γ ein minimales (p/q) -Gegenbeispiel für eine Maschinenzahl m , sei $s = p_n$ und sei $Q_1, \dots, Q_m$ eine q -Packung von Γ . Aus Lemma 3.9(a) und $\frac{m}{m-1} > 1$ folgt

$$s > p - q > \frac{1}{4}q,$$

denn $p > \frac{5}{4}q$ ist gleichbedeutend mit $p - q > \frac{1}{4}q$. Vier Aufgaben hätten damit eine Gesamtgröße von mehr als q , sodass jedes Q_i höchstens drei Aufgaben enthält und nach Lemma 3.8 genau drei. Insbesondere ist $n = 3m$, und aus $\sum_{t \in Q_i} p_t \leq q$ folgt $q \geq 3s$.

Sei nun $i \in \{1, \dots, m\}$ und angenommen, B_i enthielte höchstens zwei Aufgaben. Nach Lemma 3.9(b) wäre dann $l_i \leq 2(q - 2s)$; wegen $q \geq 3s$ ist dieser Wert positiv. Aus $s > p - q$ und $5q < 4p$ ergibt sich

$$2q - 3s < 2q - 3(p - q) = 5q - 3p < 4p - 3p = p$$

und damit $l_i \leq 2q - 4s < p - s$ - im Widerspruch zu Lemma 3.3(c), wonach $l_i > p - s$ gilt.

Jeder der Bins $B_1, \dots, B_m$ enthält also mindestens drei Aufgaben. Zusammen mit der Aufgabe p_n im $(m + 1)$ -ten Bin (Lemma 3.3(b)) folgt $n \geq 3m + 1$ im Widerspruch zu $n = 3m$. $\square$

Die Schranke $5/4 = 1,25$ ist eine obere Schranke an r_m und nicht scharf: Coffman, Garey und Johnson [CGJ78] zeigen mit weiteren Argumenten $r_m \leq 1,22$, und nach Cao [Cao95] gilt $r_m \leq 13/11$. Sie ist auch kein Worst-Case-Verhältnis von Makespans - welche Aussage über MULTIFIT aus einer Schranke an r_m folgt, klärt der folgende Abschnitt. Der Weg von $5/4$ zu $13/11$ führt bei Cao [Cao95] über Gewichtsfunktionen und eine Fallunterscheidung über die Größenverhältnisse; beides beschreibt Abschnitt 3.9.

3.8 Von r_m zur Güte von MULTIFIT

Bemerkung 3.11 (Untere Schranke an r_m). Für jedes $m \geq 1$ gilt $r_m \geq 1$. Sei dazu Γ die Instanz aus $m + 1$ Aufgaben der Größe 1 und $p \in [1, 2)$. Bei Kapazität p passen keine zwei Aufgaben in denselben Bin, also ist $\text{FFD}[\Gamma, p] = m + 1 > m$, während $\text{OPT}[\Gamma, 2] = \lceil (m + 1)/2 \rceil \leq m$ gilt. Damit ist Γ ein $(p/2)$ -Gegenbeispiel für m Maschinen, und das Supremum über $p < 2$ liefert $r_m \geq 1$.

Auch die folgende Schranke stammt aus [CGJ78]; der Beweis ist eine eigene Ausarbeitung.

Theorem 3.12 (nach [CGJ78]). Für jede Maschinenzahl $m \geq 1$ und jedes $k \geq 0$ gilt $R_m(\text{MF}(k)) \leq r_m + 2^{-k}$.

Beweis. Sei Γ eine Instanz. Im Verlauf der Binärsuche aus Abschnitt 2.3 gelten durchgehend zwei Invarianten.

Erstens ist $\text{FFD}[\Gamma, c_{\text{high}}] \leq m$, und die gespeicherte Packung ist die von FFD bei Kapazität c_{high} erzeugte. Für den Startwert ist das Lemma 2.6(b); c_{high} wird nur dann auf eine Kapazität C gesetzt, wenn $\text{FFD}[\Gamma, C] \leq m$ gilt, und in diesem Fall wird zugleich die dort erzeugte Packung gespeichert.

Zweitens ist $c_{\text{low}} \leq r_m \cdot C_{\text{max}}^*(\Gamma)$. Für den Startwert folgt das aus Lemma 2.6(a) zusammen mit Bemerkung 3.11. Gesetzt wird $c_{\text{low}} := C$ nur, wenn $\text{FFD}[\Gamma, C] > m$ gilt; nach (2.3) ist dann $C \leq r_m \cdot C_{\text{max}}^*(\Gamma)$.

Jeder Suchschritt ersetzt eine der beiden Grenzen durch die Intervallmitte, halbiert die Intervallbreite also genau. Nach k Schritten gilt daher mit (2.2) und Lemma 2.2

$$c_{\text{high}} - c_{\text{low}} = 2^{-k} (c_{\text{high}}^{(0)} - c_{\text{low}}^{(0)}) = 2^{-k} \min\left(\frac{1}{m} \sum_j p_j, p_{\text{max}}\right) \leq 2^{-k} C_{\text{max}}^*(\Gamma).$$

Jede getestete Kapazität ist mindestens $c_{\text{low}}^{(0)} \geq p_{\text{max}}$; jede Aufgabe passt also in einen leeren Bin, und die Lasten der erzeugten Packung überschreiten die Kapazität nicht. Die zurückgegebene Packung gehört nach der ersten Invariante zu c_{high} , sodass $C_{\text{max}}^{\text{MF}}(\Gamma) \leq c_{\text{high}}$ ist. Zusammen ergibt sich

$$C_{\text{max}}^{\text{MF}}(\Gamma) \leq c_{\text{low}} + (c_{\text{high}} - c_{\text{low}}) \leq (r_m + 2^{-k})C_{\text{max}}^*(\Gamma).$$

Division durch $C_{\text{max}}^*(\Gamma)$ und Übergang zum Supremum über alle Instanzen liefert die Behauptung. $\square$

Mit Theorem 3.10 folgt daraus $R_m(\text{MF}(k)) \leq 5/4 + 2^{-k}$ und mit der Schranke von Cao [Cao95] entsprechend $R_m(\text{MF}(k)) \leq 13/11 + 2^{-k}$.

Der Beweis benutzt die Monotonie von FFD an keiner Stelle, sondern allein die einseitige Implikation (2.3). Damit ist das in Bemerkung 2.8 offen gebliebene Beispiel aufgelöst. Für die dortige Instanz Γ mit $m = 4$ ist $\sum_j p_j = 301$ und $p_{\text{max}} = 40$, die Suche startet also bei $c_{\text{low}}^{(0)} = 75,25$ und $c_{\text{high}}^{(0)} = 115,25$. Die ersten drei Schritte akzeptieren 95,25, 85,25 und 80,25; der vierte verwirft $C = 77,75$ wegen $\text{FFD}[\Gamma, 77,75] = 5$ und schließt damit alle kleineren Kapazitäten aus - auch $C = 76$, wo FFD vier Bins mit den Lasten (76, 75, 75, 75) erzeugt. Da die Aufgabengrößen ganzzahlig sind und $\lceil 301/4 \rceil = 76$ ist, gilt $C_{\text{max}}^*(\Gamma) = 76$. Zurückgegeben wird für jedes $k \geq 6$ eine Packung mit Makespan 78. Ein Widerspruch zu Theorem 3.12 entsteht daraus nicht: Verworfen werden kann eine Kapazität nach (2.3) nur unterhalb von $r_4 \cdot C_{\text{max}}^*(\Gamma)$, und dieser Wert beträgt nach Cao [Cao95] höchstens $\frac{13}{11} \cdot 76 = 89,82$. Verloren geht nicht die Schranke, sondern das Optimum: Nach dem vierten Schritt kann die Suche nicht mehr unter 77,75 gehen. Davon zu unterscheiden ist die Nicht-Monotonie des Makespans in k , die in Abschnitt 5.1.7 behandelt wird.

3.9 Beweisidee der Schranke 13/11

Der Beweis von Cao [Cao95] folgt demselben Schema: Angenommen wird ein minimales (p/q) -Gegenbeispiel, und der Widerspruch entsteht aus dessen Struktur. Zusätzlich gebraucht werden eine feinere Einteilung der FFD-Bins und Gewichtsfunktionen.

Definition 3.13 (k -Bin). Sei $i < N$ und $j = \min B_{i+1}$. Dann heißt B_i ein k -Bin mit $k = |B_i^{<j}|$; er heißt *regulär*, falls $|B_i| = k$ gilt, und andernfalls *Fallback- k -Bin* [Cao95, S. 80].

Ein k -Bin nimmt also k Aufgaben auf, bevor der nächste Bin eröffnet wird; ein Fallback- k -Bin erhält später weitere.

Eine Gewichtsfunktion ordnet jeder Aufgabe ein Gewicht zu, das von ihrer Größe und vom Typ ihres FFD-Bins abhängt. Sie wird so gewählt, dass jeder FFD-Bin ein Gesamtgewicht von mindestens x trägt und jeder Bin der q -Packung höchstens x ; von der zweiten Eigenschaft sind einzelne Bins der q -Packung ausgenommen, die erste gilt ohne Ausnahme. Ist w das Gesamtgewicht aller Aufgaben und E das Mehrgewicht der

Ausnahmebins, so folgt $w \geq (m+1)x$ aus der ersten und $w \leq mx + E$ aus der zweiten Eigenschaft; ein Widerspruch entsteht, sobald $E < x$ nachgewiesen ist.

Cao [Cao95] setzt $p/q = (120-3\delta)/100$ mit $\delta < 20/33$ an, schreibt die kleinste Aufgabe als $p_n = 20 + \Delta - 3\delta$ mit $\Delta > 0$ und unterscheidet die vier Fälle $\Delta > 5\delta$, $\frac{15}{4}\delta \leq \Delta \leq 5\delta$, $2,5\delta \leq \Delta < \frac{15}{4}\delta$ und $\Delta < 2,5\delta$. Die Gewichte hängen dabei von Aufgabentypen X_2 – X_6 , Y_1 – Y_3 und Z ab, die über Größenbereiche relativ zu Δ und δ definiert sind. Die Fallunterscheidung füllt die Seiten 81–96 von [Cao95] und stützt sich zusätzlich auf zwei Lemmata aus [Yue90]; sie wird hier nicht nachvollzogen.

3.10 Untere Schranke

Lemma 3.14 (Untere Schranke). Für jedes $m \geq 13$ gilt $r_m \geq 13/11$.

Beweis. Sei Γ die Instanz aus $m - 13$ Aufgaben der Größe 66 und den 39 Aufgaben

$$40 \text{ (8-mal)}, \quad 25 \text{ (8-mal)}, \quad 24 \text{ (2-mal)}, \quad 17 \text{ (2-mal)}, \quad 16 \text{ (3-mal)}, \quad 13 \text{ (16-mal)}$$

von Friesen [Fri84]; dieselben 39 Aufgaben verwenden auch Friesen und Langston [FL86, Beispiel 1, S. 38]. Die Gesamtgröße beträgt $66m$.

Die Packung aus $(40, 13, 13)$ achtmal, $(25, 25, 16)$ dreimal, $(25, 24, 17)$ zweimal und je einem Bin für die Aufgaben der Größe 66 belegt m Bins mit Last genau 66; also ist $\text{OPT}[\Gamma, 66] \leq m$.

Sei $66 \leq C < 78$. Die Aufgaben der Größe 66 werden zuerst platziert und belegen wegen $2 \cdot 66 > C$ je einen eigenen Bin; wegen $C - 66 < 12 < 13$ nehmen diese Bins keine weitere Aufgabe auf. Auf den übrigen Aufgaben erzeugt FFD die Bins $(40, 25)$ achtmal, $(24, 24, 17)$, $(17, 16, 16, 16)$ und $(13, 13, 13, 13, 13)$ dreimal - jeder mit Last 65 - sowie einen Bin für die verbleibende Aufgabe der Größe 13, die wegen $65 + 13 = 78 > C$ in keinen der übrigen passt. Also ist $\text{FFD}[\Gamma, C] = m + 1 > m$.

Damit ist Γ für jedes solche C ein $(C/66)$ -Gegenbeispiel für m Maschinen, und das Supremum über $C < 78$ liefert $r_m \geq 78/66 = 13/11$. $\square$

Mit $r_m \leq 13/11$ nach Cao [Cao95] folgt aus Lemma 3.14 $r_m = 13/11$ für alle $m \geq 13$; für $2 \leq m < 13$ trifft diese Arbeit über den genauen Wert keine Aussage, während Bemerkung 3.4 zusammen mit Bemerkung 3.11 den Wert $r_1 = 1$ festlegt. Die Instanz für $m = 13$ wird in Abschnitt 5.1.8 empirisch untersucht. Auf der Schranke $13/11$ beruhen über (2.3) die Abschätzungen zu MULTIFIT+FFD in den Kapiteln 4 und 5; für die übrigen Subroutinen dient sie dort nur als Vergleichslinie.

4 Alternative Subroutinen

Die Quellen dieses Kapitels benutzen jeweils eigene Bezeichnungen: Jansen und Solis-Oba [JS10] schreiben β für die Kapazität und n_i für die Vielfachheiten, Gilmore und Gomory [GG61] nennen die Kapazität eine *Lagerlänge*. Auch für den zentralen Begriff dieses Kapitels - den Inhalt eines einzelnen Bins, hier *Konfiguration* genannt und in Definition 4.3 festgelegt - führt jede Quelle ein eigenes Wort. Bei Jansen und Solis-Oba steht $\mathcal{C}$ für die Menge dieser Objekte, bei Gilmore und Gomory heißen sie *Schnittmuster*. Hier wird stattdessen so weit wie möglich die in Kapitel 2 festgelegte Notation verwendet: Instanz Γ , Kapazität C , Binzahlen $\text{FFD}[\Gamma, C]$ und $\text{OPT}[\Gamma, C]$. Das macht die drei Subroutinen ohne Übersetzungsschritt vergleichbar und erspart es, denselben Begriff mehrfach zu definieren. Zwei Zeichen werden in diesem Kapitel gleichwohl neu belegt: $p_1 > \dots > p_d$ bezeichnet ab Abschnitt 4.2 die *verschiedenen* Aufgabengrößen statt der einzelnen Aufgaben, und q steht in Definition 4.3 für die Anzahl der Konfigurationen, nicht für die Kapazität einer q -Packung aus Kapitel 3.

4.1 MFFD als Subroutine in MULTIFIT

4.1.1 Der Algorithmus MFFD

Modified First Fit Decreasing (MFFD) geht auf Johnson und Garey [JG85] zurück. Der Algorithmus löst dasselbe Problem wie FFD - Bin Packing bei fester Kapazität nach Definition 2.4 - und erhält dieselbe Eingabe: die Aufgaben einer Instanz Γ und eine Kapazität C . Er klassifiziert die Aufgaben relativ zu C in vier Klassen: *large* ($p_j > C/2$), *medium* ($C/3 < p_j \leq C/2$), *small* ($C/6 < p_j \leq C/3$) und *tiny* ($p_j \leq C/6$). Anschließend durchläuft er fünf Phasen [JG85].

- Phase 1.* Für jede *large*-Aufgabe wird ein eigener Bin eröffnet, in absteigender Reihenfolge der Größen. Aufgaben der übrigen Klassen eröffnen hier keinen Bin.
- Phase 2.* Vorwärtsschritt über diese Bins. In jeden Bin wird die größte noch unplatzierte *medium*-Aufgabe gelegt, die dort hineinpasst; passt keine, bleibt der Bin unverändert. Jeder Bin erhält so höchstens eine *medium*-Aufgabe.
- Phase 3.* Rückwärtsschritt über diejenigen Bins, die in Phase 2 keine *medium*-Aufgabe erhalten haben. Passen die beiden kleinsten noch unplatzierten *small*-Aufgaben nicht gemeinsam in den Bin, so bleibt er unverändert; andernfalls werden die kleinste noch unplatzierte *small*-Aufgabe und danach die größte noch passende *small*-Aufgabe eingefügt.

Phase 4. Erneuter Vorwärtsschritt über alle Bins. In jeden Bin wird wiederholt die größte noch unplatzierte Aufgabe beliebiger Klasse gelegt, die noch hineinpasst, bis keine mehr passt.

Phase 5. Die danach noch unplatzierten Aufgaben werden mit FFD in neue Bins gepackt.

Mit $\text{MFFD}[\Gamma, C]$ wird die Anzahl der insgesamt erzeugten Bins bezeichnet. Johnson und Garey [JG85] zeigen

$$\text{MFFD}[\Gamma, C] \leq \frac{71}{60} \text{OPT}[\Gamma, C] + \frac{31}{6}; \quad (4.1)$$

Yue und Zhang [YZ95] verschärfen die additive Konstante auf 1.

Die beiden Faktoren $71/60$ und $11/9$ sind dabei nicht von derselben Art. Für FFD ist die Schranke (2.1) scharf [Dós07]; das asymptotische Worst-Case-Verhältnis von FFD beträgt also $11/9 \approx 1,2222$. Für MFFD ist $71/60 \approx 1,1833$ dagegen nur als obere Schranke bewiesen - ob es angenommen wird, ist hier nicht belegt. Gesichert ist damit die Aussage: Das asymptotische Worst-Case-Verhältnis von MFFD ist *höchstens* $71/60$ und damit echt kleiner als das von FFD.

Die Laufzeit bleibt dabei in derselben Größenordnung wie die von FFD. Sortieren und Klassifizieren kosten $O(n \log n)$. In den Phasen 2 bis 4 wird jeder der höchstens n Bins konstant oft betrachtet, und insgesamt werden höchstens n Aufgaben platziert; hält man die Klassenlisten in sortierten Suchstrukturen, so kostet jede Suche nach der größten noch passenden Aufgabe $O(\log n)$. Phase 5 ist ein FFD-Aufruf. Insgesamt ergibt das $O(n \log n)$.

Beide Schranken beziehen sich auf das Bin-Packing-Problem bei fester Kapazität und sind asymptotischer Natur: Die additive Konstante fällt erst für große $\text{OPT}[\Gamma, C]$ nicht mehr ins Gewicht. Ob sich der kleinere Faktor überträgt, wenn MFFD als Subroutine in MULTIFITs Binärsuche eintritt, ist damit nicht beantwortet. Der folgende Abschnitt gibt eine Bedingung an, unter der er vollständig entfällt.

4.1.2 Kollaps von MFFD auf FFD

Der Vorteil von MFFD kann innerhalb von MULTIFIT vollständig entfallen, und zwar aus einem strukturellen Grund, der sich exakt fassen lässt. Phase 1 eröffnet für jede large-Aufgabe einen eigenen Bin; die Phasen 2 bis 4 durchlaufen ausschließlich die dabei entstandene Binliste und füllen sie auf. Phase 5 übergibt alle danach noch unplatzierten Aufgaben an FFD. Gibt es keine large-Aufgabe, so ist die Binliste nach Phase 1 leer, die Phasen 2 bis 4 sind wirkungslos, und Phase 5 wendet FFD auf die vollständige Aufgabenliste an. Beide Subroutinen erzeugen dann dieselbe Packung, insbesondere dieselbe Binzahl:

$$p_{\max} \leq C/2 \implies \text{MFFD}[\Gamma, C] = \text{FFD}[\Gamma, C]. \quad (4.2)$$

Entscheidend ist, dass MULTIFIT nicht beliebige Kapazitäten testet, sondern nur solche oberhalb der unteren Grenze $c_{\text{low}}^{(0)}$ des Startintervalls (2.2). Daraus folgt ein Kriterium, das allein von der Instanz abhängt.

Lemma 4.1 (Kollaps von MFFD auf FFD). Sei Γ eine Instanz mit n Aufgaben der Größen $p_1, \dots, p_n$, sei $m \geq 1$ eine Maschinenzahl und

$$\rho(\Gamma) := \frac{\sum_{j=1}^n p_j}{m \cdot p_{\max}}.$$

Betrachtet wird MULTIFIT mit dem Startintervall aus (2.2), dessen untere Grenze $c_{\text{low}}^{(0)}$ ist. Ist $\rho(\Gamma) \geq 2$, so liefern MULTIFIT+FFD und MULTIFIT+MFFD für jede Iterationszahl k dieselbe Partition; insbesondere stimmen die Makespans überein.

Beweis. Zunächst gilt $c_{\text{low}} \leq c_{\text{high}}$ als Invariante über alle Iterationen: Anfangs ist sie vorausgesetzt, und jeder Schritt ersetzt entweder c_{high} oder c_{low} durch das arithmetische Mittel $\frac{1}{2}(c_{\text{low}} + c_{\text{high}})$, das zwischen beiden Grenzen liegt. Damit ist jede getestete Kapazität C mindestens so groß wie die zu diesem Zeitpunkt aktuelle untere Grenze, und da c_{low} im Verlauf der Suche nur wächst, gilt $C \geq c_{\text{low}}^{(0)}$ für jede getestete Kapazität.

Aus $\rho(\Gamma) \geq 2$ folgt $2p_{\max} \leq \sum_j p_j/m \leq c_{\text{low}}^{(0)} \leq C$, also $p_{\max} \leq C/2$ für jede getestete Kapazität C . Nach dem oben beschriebenen Phasenablauf erzeugen beide Subroutinen bei jedem einzelnen Test dieselbe Packung, nach (4.2) insbesondere dieselbe Binzahl. Die Binärsuche trifft damit in jeder Iteration dieselbe Verzweigungsentscheidung; per Induktion über die k Iterationen stimmen die Intervallgrenzen und die jeweils gespeicherte beste Partition zu jedem Zeitpunkt überein. $\square$

Bemerkung 4.2. Der Beweis nutzt von der oberen Intervallgrenze nichts und gilt daher für jede zulässige Wahl, etwa die hier verwendete Grenze $c_{\text{high}}^{(0)} = \sum_j p_j/m + p_{\max}$ oder die ursprüngliche Grenze $\max(2 \sum_j p_j/m, p_{\max})$ von Coffman, Garey und Johnson [CGJ78]. Er überträgt sich außerdem auf die Variante der Binärsuche, welche nur ganzzahlige Kapazitäten testet und in Abschnitt 5.2.2 beschrieben wird: Sind alle p_j ganzzahlig, so folgt aus $\rho(\Gamma) \geq 2$ auch $\lfloor \sum_j p_j/m \rfloor \geq \lfloor 2p_{\max} \rfloor = 2p_{\max}$, und alle getesteten Kapazitäten bleiben $\geq 2p_{\max}$.

Für gleichverteilte Aufgabengrößen lässt sich das Kriterium in das Verhältnis n/m übersetzen. Sind die p_j unabhängig gleichverteilt auf $[0, P]$, so ist $\mathbb{E}[\sum_j p_j] = nP/2$ und $p_{\max} \leq P$, also

$$\mathbb{E}[\rho(\Gamma)] \geq \frac{nP/2}{mP} = \frac{n}{2m}.$$

Ab $n/m \geq 4$ erreicht der Erwartungswert von $\rho(\Gamma)$ damit den Wert 2. Daraus folgt allerdings weder, dass $\rho(\Gamma) \geq 2$ für eine einzelne Instanz gilt, noch dass es wahrscheinlich ist: Bei $n/m = 4$ ist die Schranke gerade 2, und Instanzen mit wenigen sehr großen und vielen sehr kleinen Aufgaben verletzen $\rho(\Gamma) \geq 2$ auch bei großem n/m . Lemma 4.1 ist von der Größenverteilung dagegen unabhängig.

Der Austausch von FFD gegen MFFD kann sich also nur auf Instanzen mit $\rho(\Gamma) < 2$ auswirken; für die Subroutinen der folgenden Abschnitte gilt das nicht, da Lemma 4.1 allein die Phasen von MFFD benutzt.

Damit ist zugleich gesagt, was Lemma 4.1 *nicht* leistet. Die Bedingung $\rho(\Gamma) \geq 2$ beschreibt eine Klasse von Instanzen; auf ihr liefern beide Subroutinen dieselbe Ausgabe,

über das Verhalten auf allen übrigen Instanzen folgt daraus nichts. Eine schärfere obere Schranke für MULTIFIT+MFFD folgt daraus nicht: Auf der Klasse stimmen beide Zuordnungen überein, dort gilt für MULTIFIT+MFFD also dieselbe Schranke wie für MULTIFIT+FFD, und außerhalb sagt das Lemma nichts. Wo die Instanz von Friesen [Fri84] liegt, ist dagegen für die *untere* Schranke von Belang: Sie hat $\rho = 858/(13 \cdot 40) = 1,65 < 2$ und liegt damit außerhalb der Klasse (Abschnitt 5.1.8). Läge sie innerhalb, so erzwänge sie das Verhältnis 13/11 auch für MULTIFIT+MFFD; so aber bleibt offen, ob MULTIFIT+MFFD eine kleinere Schranke erfüllt. Das Lemma grenzt also ein, wo ein Unterschied zwischen den beiden Subroutinen überhaupt möglich ist. Wie groß der Anteil der Instanzen mit $\rho(\Gamma) < 2$ auf den Benchmark-Datensätzen ist und wie sich MFFD dort verhält, wird in Abschnitt 5.1.3 untersucht.

4.2 OPT+1-Algorithmus von Jansen und Solis-Oba

Jansen und Solis-Oba [JS10] geben einen Algorithmus für das Cutting-Stock-Problem an, der bei d verschiedenen Objektlängen - in der Notation dieser Arbeit: d verschiedenen Aufgabengrößen - eine Packung in höchstens $\text{OPT} + 1$ Bins berechnet. Cutting Stock ist Bin Packing in der *High-Multiplicity*-Darstellung: Statt n einzelner Aufgaben werden d verschiedene Größen $p_1 > p_2 > \dots > p_d > 0$ mit Vielfachheiten $u_1, \dots, u_d$ angegeben, wobei $\sum_{i=1}^d u_i = n$. Diese Darstellung wird in diesem Kapitel durchgehend verwendet. Die Implementierung und die Messungen in Abschnitt 5.2 arbeiten mit ganzzahligen Kapazitäten; die Aussagen dieses Abschnitts verlangen das nur dort, wo es ausdrücklich dabeisteht - Lemma 4.8 etwa kommt mit $C \geq p_{\max}$ aus.

Definition 4.3 (Konfiguration). Eine *Konfiguration* zur Kapazität C ist ein Vektor $a = (a_1, \dots, a_d) \in \mathbb{Z}_{\geq 0}^d$ mit

$$\sum_{i=1}^d a_i p_i \leq C \quad \text{und} \quad a_i \leq u_i \quad \text{für alle } i.$$

Sie beschreibt den Inhalt *eines* zulässigen Bins. Die Menge aller Konfigurationen wird mit $\mathcal{K}(\Gamma, C)$ bezeichnet, ihre Anzahl mit $q = |\mathcal{K}(\Gamma, C)|$.

Eine Packung ist damit eine Auswahl von Konfigurationen mit Wiederholung, die jede Größe oft genug abdeckt. Mit $J[\Gamma, C]$ wird im Folgenden die Anzahl der Bins bezeichnet, die der Algorithmus von Jansen und Solis-Oba bei Kapazität C auf Γ erzeugt.

4.2.1 Laufzeit

Die Laufzeit beträgt [JS10, Theorem 1.1]

$$d^{O(d^2)} \times 2^{O(8^d)} \times O((\log^2 n + \log C)^3 \log^5 n).$$

Dabei erscheint $\log C$, weil die Kapazität Teil der Eingabegröße ist. Die Laufzeit ist polynomiell in $\log n$, aber doppelt exponentiell in d .

4.2.2 Implementierung

Die Implementierung folgt den drei Schritten des Algorithmus von Jansen und Solis-Oba: Partitionierung der Aufgaben in große und kleine, Lösung eines gemischt-ganzzahligen linearen Programms (MILP) über alle großen Konfigurationen und anschließende Rundung der fraktionalen Lösung. Als Eingabe dienen drei Parameter: die Liste der d verschiedenen Größen $p = (p_1, \dots, p_d)$, die zugehörigen Vielfachheiten $u = (u_1, \dots, u_d)$ sowie die ganzzahlige Kapazität C . Anstelle von Lenstras Algorithmus [Len83] wird Gurobi eingesetzt: Gurobi bietet keine polynomielle Worst-Case-Garantie, ist für die hier auftretenden MILP-Größen aber praktikabel; die gemessenen Laufzeiten stehen in Abschnitt 5.2.5.

Im Originalalgorithmus wird über alle Teilmengen der Größe 2^d der weiter unten in Schritt 1 eingeführten Menge $\mathcal{K}^B$ aller großen Konfigurationen iteriert, da Lenstras Algorithmus eine feste obere Schranke an die Anzahl ganzzahliger Variablen voraussetzt [JS10]. Dass solche Teilmengen genügen, folgt aus einem Ergebnis von Eisenbrand und Shmonin [ES06] (dort zitiert in [JS10]): Das ganzzahlige Programm über alle Konfigurationen besitzt eine optimale Lösung, in der höchstens 2^d Variablen positiv sind. Mit Gurobi entfällt die Einschränkung: Es genügt, ein einziges MILP mit allen großen Konfigurationen als ganzzahlige Variablen aufzustellen. Die Subset-Enumeration wäre in der Praxis auch gar nicht durchführbar: Mit der Schranke $|\mathcal{K}^B| \leq \binom{3d-1}{d}$, welche Schritt 1 begründet, sind es für $d = 3$ bis zu $\binom{56}{8} \approx 1,4 \cdot 10^9$ Teilmengen, für $d = 5$ bis zu etwa 10^{70} - verglichen mit einem einzigen MILP, dessen Zahl ganzzahliger Variablen für festes d konstant beschränkt ist.

Schritt 1: Partitionierung in große und kleine Aufgaben

Zunächst wird der Schwellenwert

$$\varepsilon = \frac{1}{2d-1}$$

berechnet. Aufgaben mit Größe $p_i \geq \varepsilon C$ heißen *groß*, alle übrigen *klein*; α bezeichnet die Anzahl der großen Größen, sodass $p_1, \dots, p_\alpha$ groß sind. Für eine Konfiguration $a \in \mathcal{K}(\Gamma, C)$ heißt $a^B = (a_1, \dots, a_\alpha)$ ihr *großer Anteil*; die Menge aller großen Anteile wird mit $\mathcal{K}^B$ bezeichnet. Die Wahl von ε ist nicht willkürlich: Da jede große Aufgabe mindestens $\varepsilon C = C/(2d-1)$ groß ist, passen höchstens $1/\varepsilon = 2d-1$ große Aufgaben in einen Bin. Ein großer Anteil ist damit ein Vektor $a^B \in \mathbb{N}_0^\alpha$ mit $\sum_i a_i^B \leq 2d-1$, wovon es höchstens $\binom{3d-1}{d}$ viele gibt. Das begrenzt $|\mathcal{K}^B|$ auf eine Konstante für festes d .

Als triviale obere Schranke für die Binärsuche wird $m^* = n = \sum_{i=1}^d u_i$ gesetzt, da jede Aufgabe in einem eigenen Bin untergebracht werden kann.

Schritt 2: Binärsuche nach dem kleinsten zulässigen m

Für jeden Kandidaten m wird geprüft, ob die folgende MILP-Instanz zulässig ist [JS10]:

$$\begin{aligned}
& \sum_{\substack{a \in \mathcal{K}(\Gamma, C) \\ a^B = b}} x_a \leq y_b, \quad \forall b \in \mathcal{K}^B \\
& \sum_{b \in \mathcal{K}^B} y_b = m \\
& \sum_{b \in \mathcal{K}^B} b_i y_b \geq u_i, \quad \forall i = 1, \dots, \alpha \\
& \sum_{a \in \mathcal{K}(\Gamma, C)} a_i x_a \geq u_i, \quad \forall i = \alpha + 1, \dots, d \\
& x_a \geq 0, \quad y_b \in \mathbb{Z}_{\geq 0}
\end{aligned}$$

Die Variablen und Nebenbedingungen bedeuten:

- $y_b \in \mathbb{Z}_{\geq 0}$ ist die Anzahl der Bins, deren großer Anteil die Konfiguration $b \in \mathcal{K}^B$ ist.
- $x_a \geq 0$ ist die fraktionale Anzahl der Bins mit vollständiger Konfiguration a . x_a darf gebrochen sein; die Ganzzahligkeit wird erst in Schritt 3 durch Runden hergestellt.
- a_i bzw. b_i gibt an, wie viele Aufgaben der Größe p_i in der Konfiguration a bzw. b enthalten sind.

Eine Binärsuche über $\{1, \dots, m^*\}$ findet das kleinste zulässige m in $O(\log n)$ Gurobi-Aufrufen und speichert es zusammen mit der Lösung (x^+, y^+) als neues m^* .

Schritt 3: Rundung und Konstruktion der Packung

Die y^+ -Werte sind ganzzahlig; für jedes $b \in \mathcal{K}^B$ werden y_b^+ Bins mit den entsprechenden großen Aufgaben belegt. Da das MILP $\geq$ -Schranken verwendet, kann y^+ einzelne Größen öfter als u_i -mal nutzen; diese Aufgaben werden im Nachhinein vom Algorithmus entfernt. Die fraktionalen x^+ -Werte werden abgerundet: $\tilde{x}_a = \lfloor x_a^+ \rfloor$. Für jede Konfiguration a mit $\tilde{x}_a \geq 1$ werden $\tilde{x}_a$ Bins mit passendem großem Anteil a^B um die kleinen Aufgaben von a ergänzt. Da $\tilde{x}_a = \lfloor x_a^+ \rfloor \leq x_a^+$, folgt aus Nebenbedingung 1 des MILP, dass für jedes b genügend Bins bereitstehen: $\sum_{a^B=b} \tilde{x}_a \leq y_b^+$. Kleine Aufgaben, die nach dem Abrunden nicht abgedeckt sind, verteilen Jansen und Solis-Oba [JS10] zunächst mit Next Fit in nicht absteigender Größe auf die vorhandenen m^* Bins. Nach ihrem Rundungsargument bleiben dabei höchstens $2d - 1$ Aufgaben übrig, jede kleiner als εC , und diese passen gemeinsam in einen einzigen zusätzlichen Bin; die Packung verwendet damit insgesamt höchstens $m^* + 1$ Bins [JS10, Lemma 3.1]. Die Implementierung verteilt die Reste an dieser Stelle mit First Fit in absteigender Größe und legt nur das, was danach noch übrig ist, in den zusätzlichen Bin. Das Rundungsargument von Jansen und Solis-Oba [JS10] ist für Next Fit geführt; mit $J[\Gamma, C]$ ist im Folgenden deshalb stets die publizierte Fassung

gemeint, für die die Schranke $\text{OPT}[\Gamma, C] + 1$ bewiesen ist. Ob sie auch für die abweichende Verteilung gilt, ist hier nicht gezeigt; Abschnitt 5.2.4 prüft sie für die gemessene Implementierung empirisch nach.

4.2.3 Theoretische Anwendbarkeit als MULTIFIT-Subroutine

Die äußere Binärsuche in MULTIFIT akzeptiert C genau dann, wenn $J[\Gamma, C] \leq m$. Wie schon $\text{FFD}[\Gamma, \cdot]$ (Bemerkung 2.8) kann auch $J[\Gamma, \cdot]$ nicht monoton fallen; ein Zeuge dafür wird an der Implementierung in Abschnitt 5.2.4 gemessen. Anders als bei FFD lässt sich die Abweichung hier jedoch beziffern.

Lemma 4.4 (Begrenzte Nicht-Monotonie). Für alle $C \in \mathbb{N}$, $\delta \in \mathbb{N}_{\geq 1}$ und alle Instanzen Γ gilt

$$J[\Gamma, C + \delta] \leq J[\Gamma, C] + 1.$$

Beweis. Aus der Minimalität von $\text{OPT}[\Gamma, C]$ folgt $J[\Gamma, C] \geq \text{OPT}[\Gamma, C]$. Da des Weiteren jede Packung bei Kapazität C auch bei Kapazität $C + \delta$ gültig ist, gilt $\text{OPT}[\Gamma, C + \delta] \leq \text{OPT}[\Gamma, C]$. Damit folgt

$$J[\Gamma, C + \delta] \leq \text{OPT}[\Gamma, C + \delta] + 1 \leq \text{OPT}[\Gamma, C] + 1 \leq J[\Gamma, C] + 1. \quad \square$$

Korollar 4.5. Gilt $J[\Gamma, C] \geq m + 2$, so existiert keine gültige Packung von Γ in m Bins der Kapazität C .

Beweis. Aus der $\text{OPT}+1$ -Garantie von Jansen und Solis-Oba [JS10] folgt $J[\Gamma, C] \leq \text{OPT}[\Gamma, C] + 1$, also $\text{OPT}[\Gamma, C] \geq J[\Gamma, C] - 1 \geq m + 1$. $\square$

Lemma 4.4 schließt einen Anstieg nicht aus, begrenzt ihn aber auf 1: Beim Übergang $C \rightarrow C + \delta$ kann höchstens der Fall $J[\Gamma, C] < J[\Gamma, C + \delta]$ mit Differenz 1 eintreten. Der Grund liegt in Schritt 3: Die abgerundeten Werte $\lfloor x_a^+ \rfloor$ hinterlassen kleine Aufgaben, die keiner Konfiguration zugeordnet wurden. Passen diese Restobjekte in die verbleibenden Bins, so wird kein Extrabin benötigt und $J[\Gamma, C] = \text{OPT}[\Gamma, C]$. Passen sie nicht, entsteht ein zusätzlicher Bin und $J[\Gamma, C] = m^* + 1$, wobei $m^* \leq \text{OPT}[\Gamma, C]$ ist. Nicht-Monotonie tritt auf, wenn dieser Übergang bei C und $C + \delta$ unterschiedlich ausfällt, obwohl $\text{OPT}[\Gamma, C] = \text{OPT}[\Gamma, C + \delta]$: Werden die Restobjekte bei C verteilt, bei $C + \delta$ jedoch nicht, so gilt $J[\Gamma, C + \delta] = J[\Gamma, C] + 1 > J[\Gamma, C]$.

Da $J[\Gamma, C] = m + 1$ zweideutig ist - entweder $\text{OPT}[\Gamma, C] = m + 1$ (echte Kapazitätsknappheit) oder $\text{OPT}[\Gamma, C] = m$ (Rundungsartefakt von Schritt 3) - behandelt die Binärsuche diesen Fall konservativ als Ablehnung. Bleibt $J[\Gamma, C] = m + 1$ für jede getestete Kapazität, so liefert die Suche keine Packung in m Bins; auf den Instanzen aus Abschnitt 5.2.3 tritt dieser Fall nicht auf. Wird eine Kapazität akzeptiert, so konvergiert die Suche gleichwohl möglicherweise nicht gegen die kleinste zulässige.

Güte und Einschränkungen. Da $J[\Gamma, C] \leq m$ stets $\text{OPT}[\Gamma, C] \leq m$ impliziert - nach Definition des Optimums gilt $\text{OPT}[\Gamma, C] \leq J[\Gamma, C]$ -, akzeptiert die Suche nur Kapazitäten, bei denen tatsächlich eine Packung in m Bins existiert. Weil sie C mit $J[\Gamma, C] = m+1$ (und potenziell $\text{OPT}[\Gamma, C] = m$) verwirft, konvergiert sie im schlechtesten Fall gegen den optimalen Makespan auf $m-1$ Maschinen. Mit der Kurzschreibweise $C_{\max}^*(\Gamma) = C_{\max}^*(\Gamma, m)$ aus Definition 2.1 trägt folgende Kette: Für $C \geq C_{\max}^*(\Gamma, m-1)$ ist $\text{OPT}[\Gamma, C] \leq m-1$ nach Lemma 2.5, also $J[\Gamma, C] \leq m$ nach der $\text{OPT} + 1$ -Garantie - solche Kapazitäten akzeptiert die Suche stets. Liegt $C_{\max}^*(\Gamma, m-1)$ im Startintervall, so ist der im Grenzwert $k \rightarrow \infty$ zurückgegebene Makespan damit durch $C_{\max}^*(\Gamma, m-1)$ beschränkt, das Verhältnis zum Optimum also durch $C_{\max}^*(\Gamma, m-1)/C_{\max}^*(\Gamma, m)$; bei endlichem k kommt der Term 2^{-k} hinzu. Das ist eine Schranke je Instanz; als Güte im Sinn von Definition 2.3 gelesen, also als Supremum über alle Instanzen, liefert sie nur den Wert 2 - siehe das Beispiel am Ende dieses Abschnitts - und damit eine weit schwächere Aussage als 13/11. Bestimmt in beiden Fällen die Durchschnittslast das Optimum, gilt also $C_{\max}^*(\Gamma, m) \approx \sum_j p_j/m$ und $C_{\max}^*(\Gamma, m-1) \approx \sum_j p_j/(m-1)$, so ist

$$\frac{C_{\max}^*(\Gamma, m-1)}{C_{\max}^*(\Gamma, m)} \approx \frac{\sum_j p_j/(m-1)}{\sum_j p_j/m} = \frac{m}{m-1},$$

und $m/(m-1) < 13/11$ gilt für alle $m \geq 7$. Das ist eine Abschätzung unter dieser Annahme und keine Schranke: Sie setzt voraus, dass beide Optima von der Durchschnittslast bestimmt werden, und trägt nicht, sobald einzelne große Aufgaben sie nach oben drücken. Im Allgemeinen kann $C_{\max}^*(\Gamma, m-1)/C_{\max}^*(\Gamma, m)$ bis zu 2 betragen: m Aufgaben der Größe 1 auf m Maschinen liefern $C_{\max}^*(\Gamma, m) = 1$ und $C_{\max}^*(\Gamma, m-1) = 2$.

Der Algorithmus setzt schließlich zwingend voraus, dass d konstant ist. Nur dann ist die Laufzeit polynomiell in $\log n$; für allgemeines Scheduling mit $d = n$ macht die doppelt exponentielle Abhängigkeit den Algorithmus praktisch unbrauchbar. Anwendbar bleibt das Verfahren daher nur für High-Multiplicity-Instanzen mit kleinem d ; Abschnitt 5.2.5 misst den Bereich $d \leq 6$.

4.2.4 Modifikation zu einem ε -dualen Approximationsalgorithmus

Primale und duale Approximation

Approximationsalgorithmen für Bin Packing können die Gütegarantie auf zwei Weisen erfüllen. Der *primale* Ansatz hält die Kapazität C ein und verwendet dafür bis zu $(1 + \varepsilon) \cdot \text{OPT}[\Gamma, C]$ Bins. Der *duale* Ansatz kehrt diese Priorität um: Er verwendet höchstens $\text{OPT}[\Gamma, C]$ Bins und lässt dafür eine Kapazitätsüberschreitung um den Faktor $(1 + \varepsilon)$ zu.

Definition 4.6 (ε -dualer Approximationsalgorithmus [HS87]). Ein Polynomialzeitalgorithmus $\mathcal{A}$ heißt *ε -dualer Approximationsalgorithmus* für Bin Packing, wenn er für jede Instanz Γ und jede Kapazität $C \geq p_{\max}$ eine Packung berechnet, die

- (i) höchstens $\text{OPT}[\Gamma, C]$ Bins verwendet, und
- (ii) jeden Bin mit Gesamtlast $\leq (1 + \varepsilon) C$ befüllt.

Der duale Ansatz ist das entscheidende Werkzeug für das Makespan-Problem. Der Zusammenhang beider Optima ist genau die Dualität aus Lemma 2.5, $\text{OPT}[\Gamma, C] \leq m \iff C_{\max}^*(\Gamma) \leq C$; Hochbaum und Shmoys [HS87] formulieren sie über skalierte Aufgaben-Größen. Ein ε -dualer Bin-Packing-Algorithmus lässt sich daher direkt in eine Binärsuche über die Kapazität einbetten: Sie halbiert das Kapazitätsintervall und übernimmt jede Kapazität, bei der er höchstens m Bins ausgibt. Diese Prozedur ist strukturell identisch mit MULTIFITs Binärsuche.

Theorem 4.7 (Hochbaum und Shmoys [HS87], Theorem 1). Die Binärsuche über die Kapazität benutze einen ε -dualen Bin-Packing-Algorithmus als Subroutine; ihr Startintervall enthalte $C_{\max}^*(\Gamma)$ und seine Breite übersteige $C_{\max}^*(\Gamma)$ nicht. Wird sie nach k Iterationen abgebrochen, so besitzt der erzeugte Schedule einen Makespan von höchstens

$$(1 + \varepsilon)(1 + 2^{-k}) \cdot C_{\max}^*(\Gamma).$$

Für $k \rightarrow \infty$ konvergiert die Schranke gegen $(1 + \varepsilon) \cdot C_{\max}^*(\Gamma)$, die Güte also gegen $1 + \varepsilon$. Die Konstruktion eines ε -dualen Bin-Packing-Algorithmus ist damit der entscheidende Baustein.

Jansen-Dual: Modifikation des Rundungsschritts

Der Algorithmus aus Abschnitt 4.2.2 gibt höchstens $\text{OPT}[\Gamma, C] + 1$ Bins aus; der Extrabin entsteht in Schritt 3, weil die kleinen Aufgaben, die nach dem Verteilen auf die vorhandenen Bins übrig bleiben, dort in einen zusätzlichen Bin der Kapazität C gepackt werden [JS10]. Die folgende Modifikation verteilt diese Aufgaben stattdessen auf bestehende Bins und eliminiert den Extrabin - auf Kosten einer geringen Kapazitätsüberschreitung.

Jansen-Dual (modifizierter Schritt 3). Schritte 1 und 2 sind identisch zum ursprünglichen Algorithmus. Schritt 3 ändert sich wie folgt:

1. Runde ab: $\bar{x}_a = \lfloor x_a^+ \rfloor$ für alle Konfigurationen a .
2. Weise jedem der m^* Bins seine große Konfiguration und die abgerundeten kleinen Aufgaben zu. Wie im Original werden Aufgaben, die dabei öfter als u_i -mal abgedeckt sind, im Nachhinein entfernt; jede Aufgabe liegt danach in höchstens einem Bin.
3. Sammle die verbliebenen kleinen Aufgaben L (nicht abgedeckt durch die gerundeten $\bar{x}$ -Werte). Anders als im Original werden sie nicht vorab mit Next Fit verteilt; der Beweis von Lemma 4.8 braucht keine Schranke an $|L|$.
4. Platziere jede Aufgabe aus L in einem Bin mit aktueller Last $< C$.
5. Gib die m^* Bins aus - kein zusätzlicher Bin wird geöffnet.

Lemma 4.8 (Jansen-Dual ist ε -dual für $\varepsilon = 1/(2d - 1)$). Sei Γ eine Instanz mit n Aufgaben in d verschiedenen Größen, $C \geq p_{\max}$ eine Kapazität und $\varepsilon = 1/(2d - 1)$; d sei konstant. Jansen-Dual berechnet eine Packung aller Aufgaben von Γ , für die gilt:

- (i) die Anzahl der verwendeten Bins beträgt höchstens $\text{OPT}[\Gamma, C]$;
- (ii) jeder Bin trägt eine Gesamtlast von höchstens $(1 + \varepsilon)C$.

Beweis. (i) $\text{Binzahl} \leq \text{OPT}[\Gamma, C]$. m^* ist das kleinste m , für das das MILP in Schritt 2 zulässig ist. Da die optimale Packung in $\text{OPT}[\Gamma, C]$ Bins eine zulässige MILP-Lösung liefert, ist das MILP für $m = \text{OPT}[\Gamma, C]$ zulässig; daher gilt $m^* \leq \text{OPT}[\Gamma, C]$ per Definition. Jansen-Dual öffnet keinen zusätzlichen Bin und verwendet daher genau $m^* \leq \text{OPT}[\Gamma, C]$ Bins.

(ii) *Gesamtlast jedes Bins* $\leq (1 + \varepsilon)C$. Jede Konfiguration a erfüllt $\sum_i p_i a_i \leq C$, weshalb jeder Bin nach dem Abrunden in Schritt 3.1 eine Last $\leq C$ trägt. Die verbleibenden Aufgaben aus L sind ausschließlich kleine Aufgaben mit Größe $< \varepsilon C$.

Aus der Zulässigkeit des MILPs folgt zunächst, dass die Gesamtgröße aller Aufgaben höchstens m^*C beträgt. Für $b \in \mathcal{K}^B$ sei dazu $g_b = \sum_{i \leq \alpha} p_i b_i$ die Größe der großen Aufgaben eines Bins mit großem Anteil b ; wegen $b \in \mathcal{K}^B$ ist $g_b \leq C$. Die Bedarfszeilen der großen Größen liefern

$$\sum_{i \leq \alpha} p_i u_i \leq \sum_{i \leq \alpha} p_i \sum_b b_i y_b = \sum_b g_b y_b.$$

Jede Konfiguration a hält die Kapazität ein, weshalb ihr kleiner Anteil $\sum_{i > \alpha} p_i a_i \leq C - g_{a^B}$ erfüllt. Mit den Bedarfszeilen der kleinen Größen und der ersten Nebenbedingung folgt

$$\sum_{i > \alpha} p_i u_i \leq \sum_a \left(\sum_{i > \alpha} p_i a_i \right) x_a \leq \sum_b (C - g_b) \sum_{a^B=b} x_a \leq \sum_b (C - g_b) y_b.$$

Addiert man beide Ungleichungen, so heben sich die g_b auf und es bleibt

$$\sum_{i=1}^d p_i u_i \leq C \sum_b y_b = m^* C.$$

Entscheidend ist dabei, dass große und kleine Aufgaben desselben Bins sich *eine* Kapazität teilen; schätzt man die beiden Bedarfszeilen unabhängig voneinander ab, so liefern sie nur $2m^*C$ und tragen die folgende Schlussweise nicht.

Angenommen nun, beim Platzieren einer Aufgabe $s \in L$ in Schritt 3.4 wäre kein Bin mit aktueller Last $< C$ mehr vorhanden, d.h. alle m^* Bins trügen eine Last von mindestens C . Dann betrüge die Gesamtlast der Bins mindestens m^*C . Da s noch nicht platziert ist, ist sie zugleich strikt kleiner als die Gesamtgröße aller Aufgaben, sodass

$$m^*C < \sum_{i=1}^d p_i u_i$$

folgte - im Widerspruch zur soeben gezeigten Ungleichung.

Also findet Schritt 3.4 stets einen Bin mit aktueller Last $< C$; nach dem Einfügen von s ist die Last dieses Bins kleiner als $C + \varepsilon C = (1 + \varepsilon)C$.

Laufzeit. Die Größe des MILP aus Schritt 2 hängt für festes d nach Abschnitt 4.2.1 allein von d ab; Schritt 3.4 platziert dagegen jede Aufgabe aus L einzeln und kostet mit einem Zeiger über die Bins $O(|L| + m^*)$ Schritte mit $|L| \leq n$, sodass die Laufzeit polynomiell in n ist. Bei Jansen und Solis-Oba [JS10] erlaubt die Schranke $|L| \leq 2d - 1$ eine von n unabhängige Restbehandlung; für Korollar 4.9 ist der Unterschied ohne Belang.

Alle Aufgaben von Γ werden gepackt: große und abgerundete kleine Aufgaben durch Schritt 3.2, die verbliebenen kleinen Aufgaben aus L durch Schritt 3.4. $\square$

Korollar 4.9 (MULTIFIT mit Jansen-Dual). MULTIFIT mit Jansen-Dual als Subroutine liefert nach k Iterationen der Binärsuche einen Schedule mit Makespan von höchstens

$$(1 + \varepsilon)(1 + 2^{-k}) \cdot C_{\max}^*(\Gamma), \quad \varepsilon = \frac{1}{2d - 1}.$$

Beweis. Nach Lemma 4.8 ist Jansen-Dual ein ε -dualer Approximationsalgorithmus für Bin Packing. MULTIFITs Startintervall (2.2) erfüllt die Voraussetzung von Theorem 4.7: Es enthält $C_{\max}^*(\Gamma)$ nach Lemma 2.6(a) und (c), und für seine Breite gilt

$$c_{\text{high}}^{(0)} - c_{\text{low}}^{(0)} = \min\left(\frac{1}{m} \sum_j p_j, p_{\max}\right) \leq C_{\max}^*(\Gamma)$$

nach Lemma 2.2. Die Gütegarantie folgt damit aus Theorem 4.7. $\square$

Implementierungsoptimierung: Wegfall der inneren Binärsuche. Jansens Schritt 2 sucht das minimale m^* per Binärsuche und braucht dafür $O(\log n)$ MILP-Aufrufe. Im MULTIFIT-Kontext ist die Maschinenzahl m jedoch von außen bekannt und fest. Eine Subroutine wie FFD berechnet (approximativ) eine Packung mit möglichst wenigen Bins, und MULTIFIT vergleicht deren Binzahl mit m . Für diesen Vergleich genügt aber schon die Antwort auf die Frage, ob eine Packung in höchstens m Bins der Kapazität C gefunden wird. Diese Frage beantwortet ein *einzig*er MILP-Aufruf mit der Nebenbedingung $\sum_b y_b \leq m$:

- **Lösbar:** Die Packung wird wie in Schritt 3 konstruiert und zurückgegeben.
- **Unlösbar:** C ist zu klein; MULTIFIT erhöht die Kapazität.

Diese Variante minimiert die Binzahl nicht mehr, erfüllt also Eigenschaft (i) von Definition 4.6 nicht: Sie gibt bis zu m Bins aus, auch wenn $\text{OPT}[\Gamma, C] < m$ ist. Erhalten bleibt genau das, was Theorem 4.7 für die Binärsuche braucht: Der Aufruf ist für jedes $C \geq C_{\max}^*(\Gamma)$ lösbar, gibt höchstens m Bins aus und hält die Last $\leq (1 + \varepsilon)C$. Für die Lastschranke trägt das Widerspruchsargument aus dem Beweis von Lemma 4.8 mit m an Stelle von m^* , sofern die Lösung alle m Bins belegt; belegt sie weniger, ist die Last des am wenigsten gefüllten Bins erst recht $< C$. Die Anzahl der MILP-Aufrufe reduziert sich dadurch von $O(k \log n)$ auf $O(k)$, wobei k die Anzahl der MULTIFIT-Iterationen ist.

Diese Verkürzung ist an die Modifikation gebunden und lässt sich nicht auf den unveränderten $\text{OPT} + 1$ -Algorithmus übertragen. Dort entscheidet über die Annahme einer Kapazität der Wert $J[\Gamma, C]$, und der ist m^* oder $m^* + 1$: Der zusätzliche Bin aus Schritt 3 wird geöffnet, wenn nach dem Abrunden der x^+ -Werte kleine Aufgaben übrig bleiben, die sich nicht mehr auf die vorhandenen Bins verteilen lassen. Ein einzelner Aufruf mit

dem festen m an Stelle von m^* liefert in diesem Fall $m + 1$ Bins und verwirft die Kapazität, obwohl das MILP für m Bins lösbar war. Die Entscheidung hinge dann nicht mehr an der Lösbarkeit, sondern am Rundungsrest. Erst die ε -duale Fassung, die diese Reste nach Lemma 4.8 auf die bereits geöffneten Bins verteilt, macht den zusätzlichen Bin - und mit ihm die Suche nach m^* - entbehrlich. Der Vergleich in Abschnitt 5.2 misst beide Fassungen in genau dieser Form: den $\text{OPT} + 1$ -Algorithmus mit innerer Binärsuche, seine ε -duale Variante ohne.

4.3 Das Konfigurations-LP von Gilmore und Gomory als Subroutine

FFD und MFFD sind kombinatorische Verfahren, welche die Aufgaben nach festen Regeln nacheinander platzieren; der Algorithmus von Jansen und Solis-Oba löst dagegen ein ganzzahliges Programm über Konfigurationen und rundet dessen Lösung. Dieselbe Modellierung liegt schon einem klassischen Zugang zum Bin Packing zugrunde, dem linearen Programm über Konfigurationen - kurz *Konfigurations-LP* - von Gilmore und Gomory [GG61; GG63]. Dieser Abschnitt beschreibt das Modell, seine Rundung zu einer zulässigen Packung und den Einbau des Ergebnisses als Subroutine in MULTIFITs Binärsuche.

4.3.1 Das Konfigurationsmodell

Grundlage ist dieselbe High-Multiplicity-Darstellung wie in Abschnitt 4.2: eine Instanz Γ mit d verschiedenen Größen $p_1 > \dots > p_d > 0$ und Vielfachheiten $u_1, \dots, u_d$, eine feste Kapazität C und die Konfigurationsmenge $\mathcal{K}(\Gamma, C)$ aus Definition 4.3. Gilmore und Gomory [GG61] nennen eine Konfiguration ein *Schnittmuster*. Mit einer Variablen x_a je Konfiguration - der Anzahl der nach Muster a gepackten Bins - lautet das ganzzahlige Programm (IP)

$$\min \sum_{a \in \mathcal{K}(\Gamma, C)} x_a \quad \text{u.d.N.} \quad \sum_{a \in \mathcal{K}(\Gamma, C)} a_i x_a \geq u_i \quad (i = 1, \dots, d), \quad x_a \in \mathbb{Z}_{\geq 0}. \quad (\text{IP})$$

Die $\geq$ -Form ist keine Abschwächung: Deckt eine Lösung eine Größe öfter ab als nötig, so entfernt man die überzähligen Aufgaben aus den betroffenen Bins; das Ergebnis ist wieder eine Auswahl von Konfigurationen und braucht nicht mehr Bins als zuvor. Umgekehrt liefert jede Packung in k Bins eine zulässige Lösung von (IP) mit Zielfunktionswert k . Der Optimalwert von (IP) ist daher genau $\text{OPT}[\Gamma, C]$ aus Definition 2.4.

Definition 4.10 (Konfigurations-LP). Ersetzt man in (IP) die Bedingung $x_a \in \mathbb{Z}_{\geq 0}$ durch $x_a \geq 0$, so entsteht das *Konfigurations-LP*

$$\min \sum_{a \in \mathcal{K}(\Gamma, C)} x_a \quad \text{u.d.N.} \quad \sum_{a \in \mathcal{K}(\Gamma, C)} a_i x_a \geq u_i \quad (i = 1, \dots, d), \quad x_a \geq 0; \quad (\text{LP})$$

sein Optimalwert wird mit $\text{LP}[\Gamma, C]$ bezeichnet.

Das zugehörige duale Programm (D) lautet

$$\max \sum_{i=1}^d u_i y_i \quad \text{u.d.N.} \quad \sum_{i=1}^d a_i y_i \leq 1 \quad \text{für alle } a \in \mathcal{K}(\Gamma, C), \quad y \geq 0. \quad (\text{D})$$

Es hat eine Nebenbedingung je Konfiguration, also q Stück. Genau hier setzt der Beitrag von Gilmore und Gomory [GG61; GG63] an: Statt das LP mit allen q Spalten aufzustellen, löst man es über einer kleinen Spaltenteilmenge, liest die Duallösung y^* ab und sucht eine verletzte Nebenbedingung von (D) durch

$$\max \left\{ \sum_{i=1}^d y_i^* a_i \mid \sum_{i=1}^d p_i a_i \leq C, \quad 0 \leq a_i \leq u_i, \quad a \in \mathbb{Z}^d \right\},$$

also durch ein beschränktes Rucksackproblem. Ist der Optimalwert größer als 1, so liefert das zugehörige a eine neue Spalte und das eingeschränkte LP wird erneut gelöst; ist er höchstens 1, so ist y^* für (D) zulässig und das eingeschränkte LP bereits optimal. Dieses Verfahren heißt *Spaltengenerierung*.

Für die vorliegende Arbeit wird es nicht benötigt. Ist d konstant, so lässt sich $\mathcal{K}(\Gamma, C)$ vollständig aufzählen: Wegen $a_i \leq \min(\lfloor C/p_i \rfloor, u_i)$ gilt $q \leq \prod_{i=1}^d (\min(\lfloor C/p_i \rfloor, u_i) + 1)$, für festes d also ein in C polynomieller Wert. Die Spaltengenerierung ist genau das Werkzeug, mit dem das Modell auch dann handhabbar bleibt, wenn d mit n wächst und die Aufzählung ausscheidet.

4.3.2 Eigenschaften des Konfigurations-LPs

Lemma 4.11 (Schranken und Monotonie). Für jede Instanz Γ und jede Kapazität $C \geq p_{\max}$ gilt

- (a) $\lceil \text{LP}[\Gamma, C] \rceil \leq \text{OPT}[\Gamma, C]$,
- (b) $\text{LP}[\Gamma, C] \geq (\sum_{j=1}^n p_j)/C$,
- (c) $C \mapsto \text{LP}[\Gamma, C]$ ist monoton fallend.

Beweis. (a) (LP) entsteht aus (IP) durch Weglassen der Ganzzahligkeitsbedingung, also ist $\text{LP}[\Gamma, C] \leq \text{OPT}[\Gamma, C]$; da $\text{OPT}[\Gamma, C]$ ganzzahlig ist, folgt die Aussage.

(b) Jede Konfiguration a erfüllt $\sum_i a_i p_i \leq C$. Für eine zulässige Lösung x folgt $\sum_j p_j = \sum_i u_i p_i \leq \sum_i p_i \sum_a a_i x_a = \sum_a x_a \sum_i a_i p_i \leq C \sum_a x_a$.

(c) Für $C \leq C'$ ist jede Konfiguration zur Kapazität C auch eine zur Kapazität C' ; die Vielfachheitsschranken $a_i \leq u_i$ hängen nicht von der Kapazität ab. Der zulässige Bereich des LPs zu C' enthält damit denjenigen zu C (aufgefüllt mit Nullen), und das Minimum kann nicht größer werden. $\square$

Teil (b) besagt, dass das LP mindestens so scharf ist wie die Volumenschranke, aus der auch die untere Intervallgrenze $c_{\text{low}}^{(0)}$ in (2.2) stammt. Teil (c) ist bemerkenswert im Licht von Bemerkung 2.8: Die *Schranke* $\text{LP}[\Gamma, \cdot]$ fällt monoton, während $\text{FFD}[\Gamma, \cdot]$ das nicht tut. Für die gerundete Binzahl aus Abschnitt 4.3.3 wird hier keine Monotonie behauptet; sie hängt von der jeweils gewählten Optimallösung ab.

Der eigentliche Hebel für die Rundung ist die geringe Zeilenzahl des LPs.

Lemma 4.12 (Träger einer Optimalecke). Das Konfigurations-LP besitzt eine optimale Ecke, und in jeder Ecke x des zulässigen Bereichs gilt $|\{a : x_a > 0\}| \leq d$.

Beweis. Der zulässige Bereich ist durch d Bedarfszeilen und q Vorzeichenbedingungen $x_a \geq 0$ beschrieben. Eine *Ecke* ist ein zulässiger Punkt, in dem q linear unabhängige dieser Ungleichungen mit Gleichheit erfüllt sind - gleichwertig: ein zulässiger Punkt, der sich nicht als echte Konvexkombination zweier anderer zulässiger Punkte schreiben lässt. Dass ein lösbares lineares Programm mit beschränkter Zielfunktion stets eine Optimallösung in einer Ecke besitzt, ist Standard [WS11, Kap. 11]. Höchstens d der q straffen Ungleichungen können Bedarfszeilen sein; also sind mindestens $q - d$ Vorzeichenbedingungen straff und ebenso viele Variablen gleich null. $\square$

Für das Konfigurations-LP des Bin Packing ist das die übliche Begründung des Rundungsverlusts; Williamson und Shmoys [WS11, Abschnitt 4.6] verwenden dasselbe Argument.

4.3.3 Rundung zu einer zulässigen Packung

Aus einer optimalen Ecke x^* wird in drei Schritten eine Packung gebaut. Sei $F = \{a : x_a^* \notin \mathbb{Z}\}$ die Menge der fraktionalen Einträge; nach Lemma 4.12 ist $|F| \leq d$.

Schritt 1. Abrunden. Für jede Konfiguration a werden $\lfloor x_a^* \rfloor$ Bins nach Muster a gepackt. Weil (IP) in $\geq$ -Form vorliegt, kann eine Größe dabei überdeckt werden; sind von Größe p_i keine Aufgaben mehr übrig, so bleiben die entsprechenden Plätze des Musters unbesetzt.

Schritt 2. Restbedarf bestimmen. Es sei $r_i = u_i - \sum_a \lfloor x_a^* \rfloor a_i$ für $r_i > 0$ und $r_i = 0$ sonst; das sind die nach Schritt 1 noch unplatzierten Aufgaben.

Schritt 3. Restbedarf packen. Der Restbedarf wird auf zwei Arten gepackt - einmal mit einem Bin je fraktionaler Konfiguration $a \in F$, gefüllt nach deren Muster, und einmal mit FFD - und die kleinere der beiden Packungen wird verwendet. Bleibt ein Musterbin dabei leer, weil für seine Größen kein Restbedarf mehr besteht, so entfällt er; die Musterpackung braucht deshalb höchstens $|F|$ Bins, oft weniger.

Die Anzahl der insgesamt erzeugten Bins wird mit $\text{GG}[\Gamma, C]$ bezeichnet.

Lemma 4.13 (Rundungsschranke). Schritt 3 packt den Restbedarf vollständig, und es gilt

$$\text{OPT}[\Gamma, C] \leq \text{GG}[\Gamma, C] \leq \lceil \text{LP}[\Gamma, C] \rceil + |F| - 1 \leq \text{OPT}[\Gamma, C] + d - 1,$$

sofern $F \neq \emptyset$; andernfalls ist $\text{GG}[\Gamma, C] = \text{LP}[\Gamma, C] = \text{OPT}[\Gamma, C]$.

Beweis. Die untere Schranke ist klar, da das Verfahren eine zulässige Packung liefert.

Der Restbedarf passt in $|F|$ Bins. Aus der Zulässigkeit von x^* folgt für jedes i

$$r_i \leq u_i - \sum_a \lfloor x_a^* \rfloor a_i \leq \sum_a (x_a^* - \lfloor x_a^* \rfloor) a_i = \sum_{a \in F} (x_a^* - \lfloor x_a^* \rfloor) a_i < \sum_{a \in F} a_i,$$

wobei die letzte Ungleichung strikt ist, falls die rechte Summe positiv ist. Da beide Seiten ganzzahlig sind, gilt $r_i \leq \sum_{a \in F} a_i$ für alle i . Verteilt man den Restbedarf auf $|F|$ Bins, von denen der zu $a \in F$ gehörige höchstens a_i Aufgaben der Größe p_i aufnimmt, so wird jede verbliebene Aufgabe platziert und keine Kapazität überschritten. Die in Schritt 3 gewählte Packung ist höchstens so groß, also $\text{GG}[\Gamma, C] \leq \sum_a \lfloor x_a^* \rfloor + |F|$.

Die obere Schranke. Für $a \notin F$ ist $\lceil x_a^* \rceil = \lfloor x_a^* \rfloor = x_a^*$, für $a \in F$ ist $\lceil x_a^* \rceil = \lfloor x_a^* \rfloor + 1 < x_a^* + 1$. Summieren liefert

$$\sum_a \lceil x_a^* \rceil + |F| = \sum_a \lceil x_a^* \rceil < \text{LP}[\Gamma, C] + |F| \leq \lceil \text{LP}[\Gamma, C] \rceil + |F|.$$

Die linke Seite ist ganzzahlig und die erste Ungleichung strikt, also ist sie höchstens $\lceil \text{LP}[\Gamma, C] \rceil + |F| - 1$. Mit Lemma 4.11(a) und $|F| \leq d$ folgt die letzte Abschätzung. Ist $F = \emptyset$, so ist x^* ganzzahlig, das Verfahren packt genau $\text{LP}[\Gamma, C]$ Bins, und mit Lemma 4.11(a) ist dieser Wert optimal. $\square$

Bemerkung 4.14 (Warum nicht einfach aufrunden). Die naheliegende Variante, jeden Eintrag aufzurunden und $\sum_a \lceil x_a^* \rceil$ Bins zu packen, erfüllt dieselbe Schranke - die Rechnung im Beweis ist genau diese Summe. Sie ist aber nie besser: Das Abrunden erzeugt $\sum_a \lfloor x_a^* \rfloor$ Bins, und der Restbedarf braucht höchstens $|F|$ weitere, während das Aufrunden diese $|F|$ Bins immer belegt. Deshalb wird hier abgerundet.

4.3.4 Einsatz in MULTIFIT

Als Subroutine erhält das Verfahren dieselbe Schnittstelle wie FFD und MFFD: Eingabe sind die Aufgaben von Γ und eine Kapazität C , Ausgabe ist eine Packung. MULTIFIT akzeptiert C genau dann, wenn $\text{GG}[\Gamma, C] \leq m$ gilt. Sind die Aufgabengrößen ganzzahlig - wie auf den Instanzen aus Abschnitt 5.2.1 -, so lässt eine Kapazität C dieselben Packungen zu wie $\lfloor C \rfloor$; verwendet wird dort deshalb die Variante der Binärsuche mit ganzzahligen Kapazitäten (Abschnitt 5.2.2).

Gegenüber FFD ändert sich die Bedeutung einer Ablehnung. Wegen $\lceil \text{LP}[\Gamma, C] \rceil \leq \text{OPT}[\Gamma, C] \leq \text{GG}[\Gamma, C]$ zerfällt der Fall $\text{GG}[\Gamma, C] > m$ in zwei Teile:

- Ist bereits $\lceil \text{LP}[\Gamma, C] \rceil > m$, so ist $\text{OPT}[\Gamma, C] > m$, also nach Lemma 2.5 $C < C_{\max}^*(\Gamma)$. Die Ablehnung ist dann *zertifiziert*: Die untere Intervallgrenze wird auf einen Wert gesetzt, der nachweislich unterhalb des optimalen Makespans liegt. FFD zertifiziert eine Ablehnung erst ab $\text{FFD}[\Gamma, C] > \frac{11}{9}m + \frac{6}{9}$: Nach (2.1) folgt daraus $\text{OPT}[\Gamma, C] > m$. Im hier entscheidenden Fall $\text{FFD}[\Gamma, C] = m + 1$ folgt dagegen nichts über $\text{OPT}[\Gamma, C]$.
- Ist $\lceil \text{LP}[\Gamma, C] \rceil \leq m < \text{GG}[\Gamma, C]$, so bleibt offen, ob C zulässig ist; die Rundung hat lediglich keine Packung mit $\leq m$ Bins geliefert. Dieser Fall wird wie eine Ablehnung behandelt, damit die von MULTIFIT zurückgegebene Partition stets zur aktuellen oberen Intervallgrenze gehört.

Die Differenz $\text{GG}[\Gamma, C] - \lfloor \text{LP}[\Gamma, C] \rfloor$ misst damit genau den Verlust, den die Rundung gegenüber der LP-Schranke verursacht.

Anders als bei FFD ist damit nicht gesichert, dass überhaupt eine Packung gespeichert wird: Lemma 2.6(b) gilt für FFD, und Lemma 4.13 lässt auch bei $c_{\text{high}}^{(0)}$ bis zu $m + d - 1$ Bins zu. Wird jede getestete Kapazität abgelehnt, so gibt MULTIFIT keine Packung zurück; auf den Instanzen aus Abschnitt 5.2.3 tritt dieser Fall nicht auf.

Eine Gütegarantie für den Makespan folgt daraus nicht unmittelbar. Die Makespan-Schranke $13/11 + 2^{-k}$ für MULTIFIT+FFD, die Theorem 3.12 mit $r_m \leq 13/11$ nach Cao [Cao95] liefert, beruht auf Implikation (2.3): Oberhalb von $r_m \cdot C_{\text{max}}^*(\Gamma)$ kommt FFD stets mit m Bins aus, die Binärsuche akzeptiert dort also jede Kapazität und kann ihre obere Intervallgrenze bis auf $r_m \cdot C_{\text{max}}^*(\Gamma)$ absenken. Damit sich diese Schlussweise übertragen lässt, müsste eine Konstante r existieren, sodass $C \geq r \cdot C_{\text{max}}^*(\Gamma)$ stets $\text{GG}[\Gamma, C] \leq m$ nach sich zieht. Für $r = 1$ liefert Lemma 4.13 nur $\text{GG}[\Gamma, C] \leq m + d - 1$ und lässt damit gerade die $d - 1$ Bins zu, die das Akzeptanzkriterium nicht mehr zulässt. Größere Werte von r sind damit jedoch nicht ausgeschlossen. Sind $l_1, \dots, l_m$ die Lasten einer optimalen Zuordnung, so trägt je ein Paar von ihnen zusammen höchstens $2 C_{\text{max}}^*(\Gamma)$; es gilt also $\text{OPT}[\Gamma, C] \leq \lceil m/2 \rceil$ für jedes $C \geq 2 C_{\text{max}}^*(\Gamma)$. Mit Lemma 4.13 folgt $\text{GG}[\Gamma, C] \leq \lceil m/2 \rceil + d - 1$ und damit $\text{GG}[\Gamma, C] \leq m$, sobald $m \geq 2(d - 1)$ gilt. Für Instanzen mit wenigen verschiedenen Größen und hinreichend vielen Maschinen trägt die Implikation also bereits mit $r = 2$. Ein Gewinn ist das allerdings nicht: Nach Lemma 2.2 gilt $c_{\text{high}}^{(0)} = \sum_j p_j/m + p_{\text{max}} \leq 2 C_{\text{max}}^*(\Gamma)$, sodass schon das Startintervall den Faktor 2 einhält. Der Fall $r = 2$ zeigt daher nur, dass Lemma 4.13 die Schlussweise nicht widerlegt; eine Schranke unterhalb von 2 liefert er nicht. Ob eine von d unabhängige Konstante $r < 2$ existiert, bleibt offen. Der additive Term hängt von der Anzahl verschiedener Aufgabengrößen ab und ist nur für kleines d klein - im Vergleich zur $\text{OPT} + 1$ -Garantie von Jansen und Solis-Oba [JS10] ist das die schwächere Aussage, erkaufte mit einem erheblich einfacheren Verfahren. Wie oft die Schranke tatsächlich erreicht wird und ob der Rundungsverlust in der Praxis überhaupt auftritt, ist eine empirische Frage.

4.3.5 Implementierung

Die Implementierung liegt in `binpacking/gilmore_gomory_lp.py`. Die Aufgabenliste wird zu (p, u) aggregiert, $\mathcal{K}(\Gamma, C)$ per Tiefensuche mit Kapazitätsschnitt aufgezählt - es werden also nur zulässige Vektoren erzeugt, statt die volle Box aufzuspannen und nachträglich zu filtern - und das LP mit Gurobi gelöst.

Zwei Details sind für die Korrektheit wesentlich. Erstens muss der Löser eine Ecke liefern. Die Schranke $|F| \leq d$ aus Lemma 4.12 gilt nur dort: Sind x und x' zwei verschiedene optimale Ecken, so ist $\frac{1}{2}(x + x')$ ebenfalls optimal, aber keine Ecke und kann bis zu $2d$ positive Einträge haben. Simplexverfahren enden konstruktionsbedingt in einer Ecke [WS11, Anh. A], ein Innere-Punkte-Verfahren ohne anschließenden Crossover dagegen möglicherweise im Inneren einer optimalen Seitenfläche; Gurobi wird deshalb auf die duale Simplex-Methode festgelegt. Zweitens werden die x_a^* mit einer Toleranz auf- und abgerundet, weil der Löser Werte wie 2,9999999996 statt 3 zurückgibt.

Zusätzlich lässt sich mit denselben Spalten statt (LP) direkt (IP) lösen. Das liefert

$\text{OPT}[\Gamma, C]$ exakt und damit eine Referenzlinie, gegen die sich sowohl der Rundungsverlust des LPs als auch der Abstand von MULTIFIT zum bestmöglichen Ergebnis bei jeder einzelnen getesteten Kapazität messen lässt. Auf Instanzen mit kleinem d ist das praktikabel.

Abschnitt 5.2 misst beides: den Rundungsverlust $\text{GG}[\Gamma, C] - \lceil \text{LP}[\Gamma, C] \rceil$ und die Häufigkeit des unentschiedenen Falls, dazu den Makespan gegen MULTIFIT+FFD und gegen die exakte Referenzlinie.

5 Empirische Evaluation

Die Binärsuche von MULTIFIT ruft für jede getestete Kapazität C ein Bin-Packing-Verfahren auf, welches die Aufgaben in möglichst wenige Bins der Kapazität C packt - in der Regel approximativ -, und vergleicht die erreichte Binzahl mit m . Die Suche selbst lässt sich deshalb mit jedem solchen Verfahren durchführen. Die Gütegarantie aus Kapitel 3 überträgt sich dabei jedoch nicht ohne Weiteres: Sie beruht auf Implikation (2.3), welche für FFD formuliert und bewiesen ist. Dieses Kapitel misst in zwei getrennten Versuchsreihen, was der Austausch der Subroutine bewirkt.

Abschnitt 5.1 vergleicht die beiden Subroutinen FFD und MFFD auf 9092 Instanzen aus fünf publizierten Benchmark-Datensätzen. Abschnitt 5.2 betrachtet Instanzen, in denen nur wenige verschiedene Aufgabengrößen vorkommen; ihre Anzahl wird mit d bezeichnet. Auf 1080 eigens erzeugten Instanzen dieser Art werden vier Subroutinen verglichen, welche die kleine Zahl verschiedener Größen ausnutzen: der OPT+1-Algorithmus von Jansen und Solis-Oba [JS10], seine ε -duale Modifikation, das Konfigurations-LP von Gilmore und Gomory [GG61] und dasselbe Programm in ganzzahliger Fassung als exakte Referenz.

Die Trennung ist nicht bloß redaktionell. Die vier High-Multiplicity-Subroutinen setzen ein kleines, konstantes d voraus - beim OPT+1-Algorithmus und seiner ε -dualen Variante wegen der Laufzeit (Abschnitt 4.2.3), beim Konfigurations-LP, weil die Spaltenaufzählung mit d wächst und der additive Term aus Lemma 4.13 gerade $d - 1$ beträgt. Die fünf Benchmark-Datensätze bieten das nicht: Dort liegt d im Median zwischen 20 und 81 und erreicht im Maximum 2363. Beide Reihen benutzen deshalb verschiedene Datensätze. Verschieden ist außerdem die Suche: Abschnitt 5.2.2 testet nur ganzzahlige Kapazitäten, Abschnitt 5.1 dagegen reellwertige, sodass auch die Laufzeiten nicht unmittelbar vergleichbar sind. Jede Reihe beginnt mit den Datensätzen und dem Versuchsaufbau und endet mit den Grenzen der jeweiligen Aussage.

5.1 FFD gegen MFFD

Untersucht werden vier Fragen. Die Abschnitte 5.1.3 und 5.1.4 messen, wie oft sich FFD und MFFD überhaupt unterscheiden, Abschnitt 5.1.5, wie weit die erzeugten Makespans vom Optimum entfernt sind. Abschnitt 5.1.6 beziffert, was der Austausch der Subroutine an Laufzeit kostet, und Abschnitt 5.1.7, wie sich die Tiefe der Binärsuche auswirkt.

5.1.1 Benchmark-Datensätze

Tabelle 5.1 fasst die fünf Datensätze zusammen; die folgenden Abschnitte beschreiben jeden einzeln. In allen fünf sind die Aufgabengrößen ganzzahlig. Für ganze Zahlen $a \leq$

b bezeichnet $\mathcal{U}\{a, \dots, b\}$ die diskrete Gleichverteilung auf $\{a, \dots, b\}$ und $\mathcal{N}(\mu, \sigma^2)$ die Normalverteilung mit Erwartungswert μ und Varianz σ^2 . Soweit nicht anders angegeben, wird jede Aufgabengröße einer Instanz unabhängig aus derselben Verteilung gezogen.

Datensatz	Instanzen	n	m	n/m	ρ	$C_{\max}^*$ bekannt
França	150	10–1 000	5–25	0,4–200,0	0,13–102,71	3
Frangioni	780	10–1 000	5–25	2,0–200,0	0,80–188,24	747
Lawrinenko	3 499	20–220	8–100	2,0–3,0	0,70–2,41	3 488
Berndt	3 059	6–400	2–100	2,0–50,0	0,80–31,24	3 056
Planted	1 604	8–20 000	3–2 000	2,0–10,0	1,00–4,11	1 218
Gesamt	9 092	6–20 000	2–2 000	0,4–200,0	0,13–188,24	8 512

Tabelle 5.1: Übersicht der Benchmark-Datensätze. Angegeben sind jeweils die auftretenden Wertebereiche. $\rho = \sum_j p_j / (m \cdot p_{\max})$ ist die Größe aus Lemma 4.1. OPT-Werte für Frangioni, Lawrinenko, Berndt und einen Teil der Planted-Instanzen stammen aus Akram u. a. [Akr+25]; die drei França-Werte wurden für diese Arbeit exakt berechnet (Abschnitt 5.1.2).

França-Instanzen

França, Gendreau, Laporte und Müller führten diesen Datensatz zur Evaluation einer Composite-Heuristik für $P \parallel C_{\max}$ ein [Fra+94].

Struktur. Jede Aufgabengröße ist $\mathcal{U}\{1, \dots, 100\}$ -verteilt. Für jede Kombination aus $n \in \{10, 50, 100, 500, 1000\}$ und $m \in \{5, 10, 25\}$ gibt es 10 Instanzen - $5 \times 3 \times 10 = 150$ Instanzen mit $n/m \in [0,4; 200]$. Der Datensatz enthält als einziger Instanzen mit $n < m$.

Einordnung. Mit $\rho \in [0,13; 102,71]$ deckt der Datensatz beide Seiten der Schwelle aus Lemma 4.1 ab: 108 der 150 Instanzen erfüllen $\rho \geq 2$, 42 nicht. OPT-Werte liegen im Original nicht bei.

Frangioni-Instanzen

Frangioni, Necciari und Scutellà entwickelten diesen Datensatz für die empirische Analyse von Scheduling-Heuristiken auf $P \parallel C_{\max}$ [FNS04].

Struktur. Für jede obere Größenschranke $R \in \{100, 1000, 10000\}$ werden zwei Verteilungen der Aufgabengrößen unterschieden:

- **U (gleichverteilt):** Jede Größe ist $\mathcal{U}\{1, \dots, R\}$ -verteilt.
- **NU (bimodal):** Jede Größe wird mit Wahrscheinlichkeit $\frac{1}{2}$ aus $\mathcal{U}\{1, \dots, \lfloor R/3 \rfloor\}$ und mit Wahrscheinlichkeit $\frac{1}{2}$ aus $\mathcal{U}\{\lfloor 2R/3 \rfloor, \dots, R\}$ gezogen.

Jede der sechs Kombinationen aus R und Verteilung wird mit denselben Parametern wie der França-Datensatz erzeugt, also mit $n \in \{10, 50, 100, 500, 1000\}$ und $m \in \{5, 10, 25\}$

und je 10 Instanzen. Das ergibt $6 \times 15 \times 10 = 900$ Instanzen; das Original enthält davon die 780 mit $n > m$, es fehlen also die Paare $(n, m) = (10, 10)$ und $(10, 25)$. Für 747 der 780 Instanzen sind OPT-Werte aus Akram u. a. [Akr+25] verfügbar.

Einordnung. Die drei Wertebereiche R unterscheiden sich um je eine Größenordnung bei sonst gleicher Parametrisierung. Die beiden Verteilungen unterscheiden sich auch in ρ : Der Median liegt bei den bimodalen Instanzen bei 18,7, bei den gleichverteilten bei 9,7; der Anteil der Instanzen mit $\rho \geq 2$ beträgt 84,6 % beziehungsweise 81,3 %. Mit 82,9 % ist er über den gesamten Datensatz der höchste aller fünf Datensätze.

Lawrinenko-Instanzen

Dieser Datensatz geht auf unveröffentlichte Arbeiten von Lawrinenko zurück [Akr+25] und wurde von Mrad und Souayah für einen Arc-Flow-Ansatz auf $P \parallel C_{\max}$ eingesetzt [MS18].

Struktur. Die Aufgabengrößen folgen je nach Klasse einer von sieben Verteilungen:

- Klassen 1–3: $\mathcal{U}\{1, \dots, 100\}$, $\mathcal{U}\{20, \dots, 100\}$, $\mathcal{U}\{50, \dots, 100\}$
- Klassen 4–5: $\mathcal{N}(100, 20^2)$, $\mathcal{N}(100, 50^2)$
- Klassen 6–7: $\mathcal{U}\{n, \dots, 4n\}$, $\mathcal{N}(4n, n^2)$

Normalverteilt gezogene Werte werden auf ganze Zahlen gerundet, nicht positive Werte neu gezogen. In den Klassen 6 und 7 hängen die Verteilungsparameter von der Aufgabenzahl n ab. Kombiniert werden die Klassen mit 50 Paaren (n, m) aus drei Gruppen: $n \in \{20, 40, \dots, 200\}$ mit $m = n/2$ und $m = \lfloor 2n/5 \rfloor$, $n \in \{36, 54, \dots, 198\}$ mit $m = n/3$ und $m = \lfloor 4n/9 \rfloor$ sowie $n \in \{22, 44, \dots, 220\}$ mit $m = \lfloor 4n/11 \rfloor$. Mit 10 Instanzen je Klasse und Paar ergeben sich $50 \times 7 \times 10 = 3500$ Instanzen, davon 3 499 originale; für 3 488 ist OPT bekannt.

Einordnung. Die Klassen 1–3 unterscheiden sich ausschließlich in der Untergrenze der Aufgabengrößen, die Klassen 4 und 5 ausschließlich in der Streuung. Mit 3 499 Instanzen ist dies der größte Datensatz; sein n/m liegt durchgehend zwischen 2,0 und 3,0 und sein ρ zwischen 0,70 und 2,41.

Berndt-Instanzen

Berndt, Deppert, Jansen und Rohwedder verwenden diesen Datensatz, um ein Approximationsschema für $P \parallel C_{\max}$ mit klassischen Heuristiken zu vergleichen [Ber+22]. Vier der fünf unten beschriebenen Kategorien (E1 bis E4) gehen nach ihrer Darstellung auf Kedia (1971) zurück und sind seither mehrfach zur Bewertung von Scheduling-Verfahren benutzt worden; die fünfte, BIG, ergänzen sie selbst. Die hier verwendeten Instanzen haben Akram u. a. [Akr+25] nach dieser Beschreibung erzeugt. Für 3 056 der 3 059 originalen Instanzen sind OPT-Werte aus Akram u. a. [Akr+25] verfügbar.

Struktur. Fünf Kategorien; jede Aufgabengröße ist auf dem angegebenen Bereich gleichverteilt, und bei mehreren Bereichen gibt es jede Kombination für jeden davon:

- **E1:** $m \in \{3, 4, 5\}$, $n \in \{2m, 3m, 5m\}$; $\mathcal{U}\{1, \dots, 20\}$ und $\mathcal{U}\{20, \dots, 50\}$.
- **E2:** $m \in \{2, 3\}$ mit $n \in \{10, 30, 50, 100\}$; $m \in \{4, 6, 8, 10\}$ mit $n \in \{30, 50, 100\}$; $\mathcal{U}\{100, \dots, 800\}$.
- **E3:** $m \in \{3, 5, 8, 10\}$, $n = \mu m + \delta$ ($\mu \in \{3, 4, 5\}$, $\delta \in \{1, 2\}$); $\mathcal{U}\{1, \dots, 100\}$ und $\mathcal{U}\{100, \dots, 200\}$.
- **E4:** $(m, n) \in \{(2, 10), (3, 9)\}$; sechs Bereiche, nämlich $\{1, \dots, 20\}$, $\{20, \dots, 50\}$, $\{1, \dots, 100\}$, $\{50, \dots, 100\}$, $\{100, \dots, 200\}$ und $\{100, \dots, 800\}$.
- **BIG:** $m \in \{25, 50, 75, 100\}$, $n = 4m$; $\mathcal{U}\{1, \dots, 1000\}$.

Jede der 102 Kombinationen wird 30-mal erzeugt: $30 \times 102 = 3060$ Instanzen, davon 3 059 originale.

Einordnung. E4 verwendet als einzige Kategorie dieselben (m, n) -Konfigurationen unter sechs verschiedenen Größenbereichen und variiert damit ausschließlich die Größenverteilung. BIG enthält die größten Maschinenzahlen des Datensatzes (m bis 100). Mit 2 365 von 3 059 Instanzen mit $\rho \geq 2$ (77,3%) stellt Berndt die meisten Instanzen, auf denen die Subroutinen nach Lemma 4.1 übereinstimmen müssen.

Planted-Instanzen

Akram, Maas, Sanders und Schreiber führten diesen Datensatz zur Evaluation exakter Löser für $P \parallel C_{\max}$ ein [Akr+25]; bei einem Teil der Instanzen ist der optimale Makespan konstruktionsbedingt bekannt.

Konstruktion. Eine *Planted*-Instanz mit Parametern (n, m, U) wird wie folgt erzeugt: Zunächst erhält jede der m Maschinen genau U Zeitschritte, die alle einer einzigen Aufgabe zugeordnet sind - der Makespan ist exakt U , jede Maschine voll ausgelastet. Iterativ wird ein zusammenhängender Block einer bestehenden Aufgabe zufällig gespalten und als neue Aufgabe eingetragen, bis n Aufgaben vorliegen. Da kein Zeitschritt wegfällt, bleibt der optimale Makespan $C_{\max}^* = U$.

Bei Perturbationsrate $r > 0$ wird ein Anteil r der Aufgaben um 1 vergrößert. Dadurch steigt die Gesamtlast, und $C_{\max}^*$ ist mindestens $U + 1$.

Struktur. Der Datensatz enthält zwei Teile:

- **Originale** (1 064 Instanzen): konvertierte Dateien aus dem Repository von Akram u. a. [Akr+25]; für 1 038 davon ist OPT bekannt.
- **Generierte** (540 Instanzen): mit festem Zufallsstartwert je Instanz reproduzierbar erzeugt, mit

$$(n, m) \in \{(10, 4), (20, 5), (50, 10), (100, 20), (200, 50), (500, 100)\},$$

$$U \in \{100, 300, 1000\} \text{ und } r \in \{0; 0,05; 0,1\}, \text{ je 10 Instanzen.}$$

Für alle 461 Instanzen mit $r = 0$ gilt $C_{\max}^* = U$ per Konstruktion; für weitere 757 Instanzen mit $r > 0$ liefert der exakte Löser des Repositories bekannte OPT-Werte [Akr+25]. Insgesamt sind für 1 218 der 1 604 Instanzen OPT-Werte bekannt.

Einordnung. Die Aufgabengrößen entstehen durch iteratives Aufspalten und sind daher keiner der in den übrigen Datensätzen verwendeten Verteilungen entnommen. Der Datensatz enthält mit n bis 20 000 und m bis 2 000 die größten Instanzen dieser Auswertung.

5.1.2 Versuchsaufbau

Implementierung und Parameter. Gemessen wird MULTIFIT mit den Subroutinen FFD und MFFD, beide in einer $O(n \log n)$ -Implementierung (Segmentbaum für First Fit, `SortedList` für die Phasen 2–4 von MFFD). Die Binärsuche startet mit dem Intervall $[\max(\sum_j p_j/m, p_{\max}), \sum_j p_j/m + p_{\max}]$ und führt $k = 10$ Halbierungsschritte mit reellwertigen Kapazitäten aus; $k = 10$ entspricht nach Theorem 3.12 einem additiven Term von $2^{-10} < 0,001$ in der Gütegarantie von MULTIFIT+FFD; für MULTIFIT+MFFD ist keine Gütegarantie bewiesen. Jede Instanz wird von beiden Subroutinen im selben Prozess und in derselben Reihenfolge bearbeitet, sodass die Laufzeiten paarweise vergleichbar sind.

Messumgebung. Python 3.11.2 auf Linux 6.1 (Intel Core i7-12700H). Die Laufzeit umfasst den vollständigen MULTIFIT-Lauf einschließlich aller $k + 1$ Subroutinenaufrufe, nicht einzelne Packvorgänge.

Referenzwerte. Von den 8 512 Instanzen mit bekanntem Optimum stammen alle bis auf drei aus der Literatur beziehungsweise aus der Konstruktion (Tabelle 5.1). Die drei übrigen sind França-Instanzen und wurden für diese Arbeit exakt gelöst: ganzzahliges Programm mit Zuordnungsvariablen, Gurobi, Optimalität nachgewiesen. Für die verbleibenden 580 Instanzen wird die untere Schranke

$$\text{LB}(\Gamma) = \max\left(\left\lceil \frac{1}{m} \sum_j p_j \right\rceil, p_{\max}\right) \leq C_{\max}^*(\Gamma) \quad (5.1)$$

verwendet; die Aufrundung ist zulässig, weil bei ganzzahligen Aufgabengrößen auch $C_{\max}^*$ ganzzahlig ist.

Reproduktion. Alle Zahlen und Abbildungen dieses Kapitels werden aus den Rohmessungen erzeugt. Die Messung übernimmt `run_benchmark.py` und legt je Datensatz eine `benchmark.csv` ab; daraus erzeugen `summary_tables.py` die Tabellen und Kennzahlen sowie `thesis_figures.py` die Abbildungen. Diese Skripte liegen zusammen mit den Implementierungen der Subroutinen, den Rohmessungen und den selbst erzeugten Instanzen samt Generatoren unter <https://github.com/hempl1111/MultiFit-BscThs>. Die Originalinstanzen - Frangioni, Lawrinenko, Berndt und der entsprechende Teil des Planted-Datensatzes - liegen dort nicht bei; ein Skript lädt sie aus einem festen Stand des Repositories von Akram u. a. [Akr+25] und prüft jede Datei gegen eine Prüfsumme.

5.1.3 Kollaps von MFFD auf FFD

Lemma 4.1 besagt: Aus $\rho(\Gamma) \geq 2$ folgt, dass MULTIFIT+FFD und MULTIFIT+MFFD dieselbe Partition erzeugen. Auf den Benchmark-Datensätzen erfüllen 3489 der 9092 Instanzen (38,4%) diese Bedingung. In keiner davon unterscheiden sich die gemessenen Makespans, wie es Lemma 4.1 verlangt.

Abbildung 5.1 stellt das Kriterium dem Verhältnis n/m gegenüber, auf das sich der Erwartungswert von $\rho(\Gamma)$ in Abschnitt 4.1.2 zurückführen lässt. Auch oberhalb von $n/m = 12$ tritt keine Abweichung auf (760 Instanzen); dieser Wert ist jedoch an den Daten abgelesen und nicht hergeleitet. Das n/m -Kriterium trennt zudem weniger scharf: Die Abweichung mit dem größten n/m liegt bei $n/m = 10,0$ auf und damit deutlich oberhalb von $n/m = 4$, ab dem der Erwartungswert von $\rho(\Gamma)$ nach Abschnitt 4.1.2 den Wert 2 erreicht - woraus für die einzelne Instanz nichts folgt. Bei ρ liegt die größte beobachtete Abweichung dagegen bei $\rho = 1,9475$, unmittelbar unterhalb der bewiesenen Schwelle.

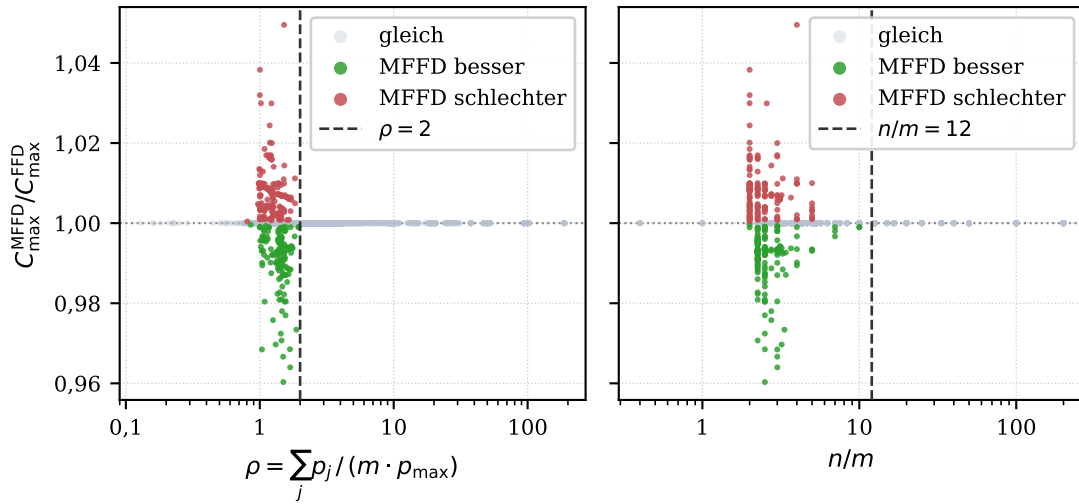


Abbildung 5.1: Makespan von MULTIFIT+MFFD im Verhältnis zu dem von MULTIFIT+FFD, über alle 9092 Instanzen des Benchmarks und aufgetragen gegen $\rho = \sum_j p_j / (m \cdot p_{\max})$ (links) und gegen n/m (rechts). Rechts der Linie $\rho = 2$ liegt keine Abweichung.

Tabelle 5.2 zeigt den Übergang. Der Anteil abweichender Instanzen erreicht sein Maximum mit 11,0% im Bereich $1,25 \leq \rho < 1,5$ und fällt unmittelbar unterhalb der Schwelle auf 0,9%, oberhalb von $\rho = 2$ auf null. Die Schwelle aus Lemma 4.1 ist auf diesen Datensätzen also nicht nur hinreichend, sondern wird auch fast erreicht. Auf den 5603 Instanzen mit $\rho < 2$ ist ein Unterschied möglich; wie er ausfällt, behandelt der nächste Abschnitt.

Bereich	Instanzen	Abweichungen	Anteil
$\rho < 0,5$	11	0	0,0 %
$0,5 \leq \rho < 1,0$	242	11	4,5 %
$1,0 \leq \rho < 1,25$	1 790	140	7,8 %
$1,25 \leq \rho < 1,5$	1 394	153	11,0 %
$1,5 \leq \rho < 1,75$	1 253	68	5,4 %
$1,75 \leq \rho < 2,0$	913	8	0,9 %
$\rho \geq 2,0$	3 489	0	0,0 %

Tabelle 5.2: Anteil der Instanzen, auf denen MULTIFIT+MFFD und MULTIFIT+FFD verschiedene Makespans liefern, gestaffelt nach ρ . Oberhalb von $\rho = 2$ tritt keine Abweichung auf (Lemma 4.1).

5.1.4 Makespan-Vergleich

Über alle 9 092 Instanzen liefert MULTIFIT+MFFD in 200 Fällen (2,2 %) einen kleineren, in 180 Fällen (2,0 %) einen größeren und in 8 712 Fällen (95,8 %) denselben Makespan wie MULTIFIT+FFD (Tabelle 5.3). Sämtliche 380 Abweichungen liegen bei $\rho < 2$, wie es Lemma 4.1 verlangt.

Datensatz	N	MFFD besser	gleich	MFFD schlechter	$C_{\max}^{\text{MFFD}}/C_{\max}^{\text{FFD}}$		
					Min.	Median	Max.
França	150	0	150	0	1,0000	1,0000	1,0000
Frangioni	780	1	775	4	0,9996	1,0000	1,0135
Lawrinenko	3 499	146	3 245	108	0,9603	1,0000	1,0244
Berndt	3 059	17	3 030	12	0,9667	1,0000	1,0112
Planted	1 604	36	1 512	56	0,9640	1,0000	1,0495
Alle	9 092	200	8 712	180	0,9603	1,0000	1,0495

Tabelle 5.3: Vergleich der Makespans von MULTIFIT+MFFD und MULTIFIT+FFD je Datensatz. Die Zeile *Alle* berechnet Anzahlen und Kennzahlen über alle Instanzen gemeinsam.

Die Abweichungen fallen in beide Richtungen aus und bleiben klein: Das Verhältnis $C_{\max}^{\text{MFFD}}/C_{\max}^{\text{FFD}}$ liegt zwischen 0,9603 und 1,0495 (Abbildung 5.2). Auf welcher Seite sie überwiegen, hängt vom Datensatz ab: Auf Lawrinenko und Berndt liefert MFFD häufiger den kleineren, auf Frangioni und Planted häufiger den größeren Makespan (Tabelle 5.3). Einen systematisch kleineren Makespan liefert MULTIFIT+MFFD auf diesen Instanzen also nicht. Aus den Schranken (2.1) und (4.1) ließ sich das auch nicht erwarten: Beide beschränken nur die Binzahl nach oben und sagen nichts über den Makespan von MULTIFIT, und für MULTIFIT+MFFD ist keine Makespan-Schranke bewiesen.

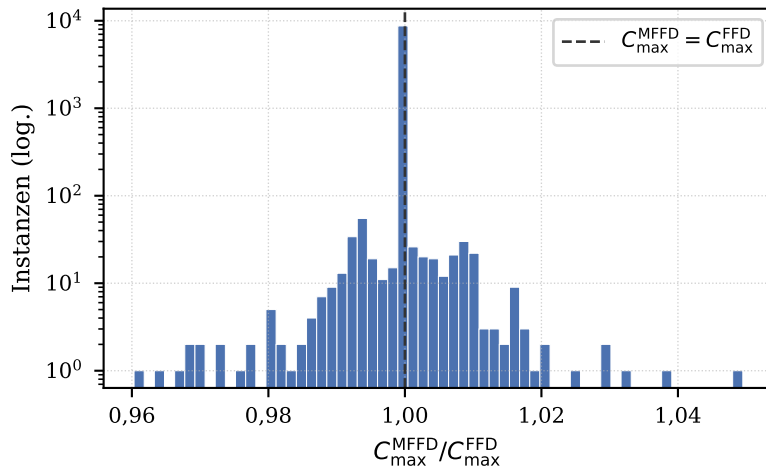


Abbildung 5.2: Verteilung von $C_{\max}^{\text{MFFD}}/C_{\max}^{\text{FFD}}$ über alle 9092 Instanzen (logarithmische y -Achse). Der Balken bei 1 enthält die 8712 Instanzen mit identischem Makespan.

5.1.5 Güte gegenüber dem Optimum

Auf den 8512 Instanzen mit bekanntem Optimum erreicht MULTIFIT+FFD in 26,6 % der Fälle exakt den optimalen Makespan. Das größte beobachtete Verhältnis beträgt für beide Varianten 1,1221 - für MULTIFIT+FFD wie für MULTIFIT+MFFD -, jeweils auf einer Berndt-Instanz (Tabelle 5.4, Abbildung 5.3). Beide Werte liegen unterhalb von $13/11 + 2^{-k} \approx 1,1828$ für $k = 10$. Bewiesen ist diese Schranke nur für MULTIFIT+FFD, nämlich durch Theorem 3.12 zusammen mit $r_m \leq 13/11$ nach Cao [Cao95]; für MULTIFIT+MFFD dient sie hier lediglich als Vergleichslinie.

Datensatz	$C_{\max}^*$ bekannt	MULTIFIT+FFD			MULTIFIT+MFFD		
		Mittel	Max.	$= C_{\max}^*$	Mittel	Max.	$= C_{\max}^*$
França	3	1,0000	1,0000	100,0 %	1,0000	1,0000	100,0 %
Frangioni	747	1,0066	1,0383	29,9 %	1,0067	1,0383	29,5 %
Lawrinenko	3 488	1,0192	1,1080	27,5 %	1,0190	1,1080	26,0 %
Berndt	3 056	1,0182	1,1221	19,1 %	1,0181	1,1221	19,1 %
Planted	1 218	1,0034	1,1000	40,9 %	1,0036	1,1000	40,1 %
Alle	8 512	1,0154	1,1221	26,6 %	1,0154	1,1221	25,9 %

Tabelle 5.4: Verhältnis $C_{\max}^{\text{MF}}/C_{\max}^*$ über die 8512 Instanzen mit bekanntem Optimum. *Mittel* ist das arithmetische Mittel über die Instanzen, die Spalte $= C_{\max}^*$ gibt den Anteil der Instanzen an, auf denen MULTIFIT den optimalen Makespan erreicht. Die Zeile *Alle* berechnet dieselben Kennzahlen über alle Instanzen gemeinsam.

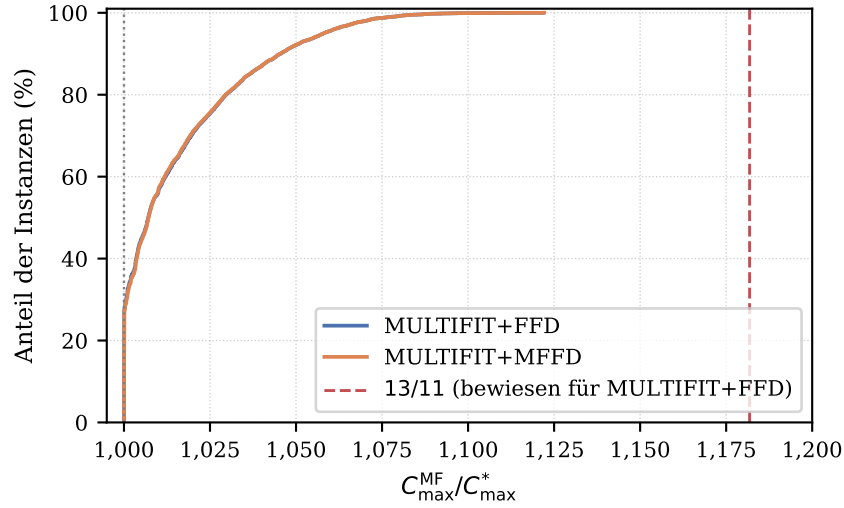


Abbildung 5.3: Empirische Verteilungsfunktion von $C_{\max}^{\text{MF}}/C_{\max}^*$ über die 8512 Instanzen mit bekanntem Optimum. Die gestrichelte Linie markiert 13/11; als Schranke bewiesen ist dieser Wert nur für MULTIFIT+FFD.

Für die 580 Instanzen ohne bekanntes Optimum lässt sich dieselbe Aussage über die untere Schranke aus (5.1) führen: Aus $\text{LB} \leq C_{\max}^*$ folgt $C_{\max}^{\text{MF}}/C_{\max}^* \leq C_{\max}^{\text{MF}}/\text{LB}$. Auf allen 580 dieser Instanzen gilt $C_{\max}^{\text{MF}}/\text{LB} \leq 1,1368$ und damit erst recht $C_{\max}^{\text{MF}}/C_{\max}^* \leq 13/11$ (Tabelle 5.5).

Datensatz	N	$C_{\max}^*$ bekannt	über LB belegt	unbestimmt
França	150	3	147	0
Frangioni	780	747	33	0
Lawrinenko	3 499	3 488	11	0
Berndt	3 059	3 056	3	0
Planted	1 604	1 218	386	0
Gesamt	9 092	8 512	580	0

Tabelle 5.5: Abdeckung der gemessenen Aussage $C_{\max}^{\text{MF}}/C_{\max}^* \leq 13/11$ für MULTIFIT+FFD und MULTIFIT+MFFD. Für Instanzen ohne bekanntes Optimum wird sie über die untere Schranke (5.1) belegt.

Damit gilt für jede der 9092 Instanzen, dass weder MULTIFIT+FFD noch MULTIFIT+MFFD das Verhältnis 13/11 überschreitet; der größte über alle Instanzen mit bekanntem Optimum gemessene Wert ist 1,1221. Das ist eine Aussage über diese Instanzmenge: Für MULTIFIT+FFD ist sie mit der bewiesenen Schranke verträglich, für MULTIFIT+MFFD begründet sie keine. Die Worst-Case-Instanz von Friesen [Fri84], auf der MULTIFIT+FFD das Verhältnis 13/11 exakt annimmt, ist in keinem der fünf Da-

tensätze enthalten; Abschnitt 5.1.8 wertet sie deshalb gesondert für beide Subroutinen aus.

5.1.6 Laufzeit

MFFDs zusätzliche Klassifikation kostet Laufzeit. Der Median des instanzweisen Verhältnisses $t_{\text{MFFD}}/t_{\text{FFD}}$ liegt je nach Datensatz zwischen 1,037 und 1,522 (Tabelle 5.6, Abbildung 5.4 links). Der Mehraufwand ist damit messbar, bleibt auf diesen Datensätzen aber in derselben Größenordnung.

Datensatz	Median FFD (ms)	Median MFFD (ms)	$t_{\text{MFFD}}/t_{\text{FFD}}$	
			Median	95. Perzentil
França	1,600	1,982	1,201	1,837
Frangioni	1,630	2,000	1,173	1,646
Lawrinenko	2,010	1,952	1,037	1,374
Berndt	0,339	0,498	1,522	2,078
Planted	1,743	2,027	1,154	1,855
Alle	1,051	1,095	1,217	1,965

Tabelle 5.6: Laufzeit eines vollständigen MULTIFIT-Laufs ($k = 10$) und paarweises Verhältnis beider Subroutinen je Instanz. Die Zeile *Alle* gibt Median und Perzentil über alle Instanzen gemeinsam an, nicht einen Mittelwert der Datensatzzeilen.

Das Wachstum in n unterscheidet sich zwischen beiden Subroutinen nicht wesentlich: Auf dem Planted-Datensatz, dem einzigen mit einem n -Bereich über drei Größenordnungen, beträgt die Steigung der Ausgleichsgeraden im doppelt-logarithmischen Maßstab 1,14 für FFD und 1,04 für MFFD (Abbildung 5.4 rechts). Eine Steigung nahe 1 ist mit der Laufzeit $O(n \log n)$ beider Implementierungen verträglich. Zur Einordnung der absoluten Werte siehe Abschnitt 5.1.9.

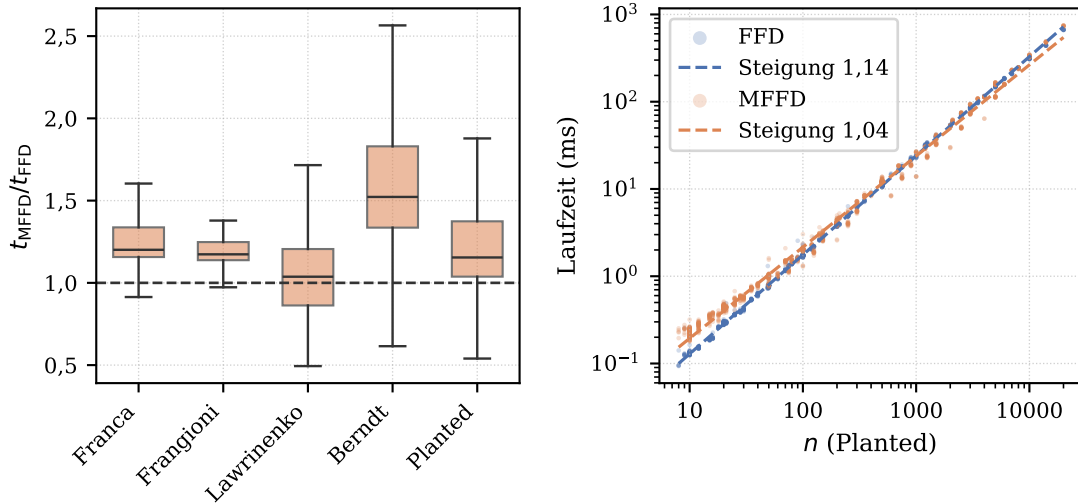


Abbildung 5.4: Links: paarweises Laufzeitverhältnis je Datensatz (ohne Ausreißer). Rechts: Laufzeit gegen n auf dem Planted-Datensatz, dessen n sich über drei Größenordnungen erstreckt, mit Ausgleichsgeraden im doppelt-logarithmischen Maßstab.

5.1.7 Sensitivität gegenüber der Binärsuchtiefe

Nach Theorem 3.12 gilt für MULTIFIT+FFD $R_m(\text{MF}(k)) \leq r_m + 2^{-k}$: Der additive Term, welcher auf die Binärsuche entfällt, halbiert sich mit jedem Halbierungsschritt. Um zu messen, ab welchem k sich das Ergebnis auf den Benchmark-Instanzen nicht mehr ändert, wurde MULTIFIT für $k = 1, \dots, 10$ über alle 9 092 Instanzen ausgeführt.

Das gemessene Maximum liegt für beide Subroutinen bei jeder Tiefe unterhalb von $13/11 + 2^{-k}$, der für MULTIFIT+FFD bewiesenen Schranke (Tabelle 5.7, Abbildung 5.5 rechts): bei $k = 1$ sind es 1,4997 gegenüber 1,6818, bei $k = 5$ dann 1,1221 gegenüber 1,2131 und bei $k = 10$ 1,1221 gegenüber 1,1828.

Ab $k = 5$ ändert sich das Maximum nicht mehr; der Mittelwert sinkt dagegen bis $k = 10$ weiter, von 1,2584 bei $k = 1$ auf 1,0154.

Auch bei $k = 10$ hat sich der Makespan nicht auf allen Instanzen stabilisiert: Bei MULTIFIT+FFD stimmt er auf 91,5 % der Instanzen bei $k = 9$ mit dem bei $k = 10$ überein, auf den übrigen ändert er sich im letzten Halbierungsschritt noch. Da die Suche reellwertige Kapazitäten testet, schrumpft das Suchintervall mit jedem Schritt, wird aber nie leer; ein größeres k kann folglich eine noch kleinere akzeptierte Kapazität und damit einen kleineren Makespan liefern. Die ganzzahlige Suche aus Abschnitt 5.2 endet dagegen, sobald sich untere und obere Grenze des Intervalls treffen.

Der Makespan fällt allerdings nicht durchgängig monoton in k . MULTIFIT verkleinert die Kapazitätsschranke C , nicht den tatsächlich erreichten Makespan: Passt eine Packung mit kleinerem C noch in m Bins, wird sie übernommen, auch wenn ihre

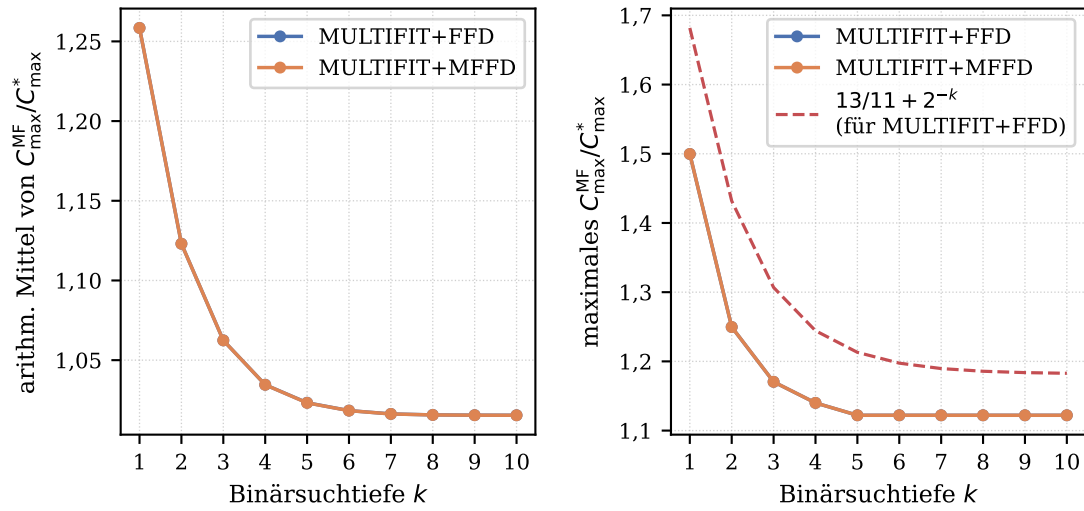


Abbildung 5.5: Arithmetisches Mittel (links) und Maximum (rechts) des Verhältnisses $C_{\max}^{\text{MF}}/C_{\max}^*$ in Abhängigkeit der Binärsuchtiefe k über die 8 512 Instanzen mit bekanntem Optimum. Rechts zusätzlich die für MULTIFIT+FFD bewiesene Schranke $13/11 + 2^{-k}$ nach Cao [Cao95].

k	MULTIFIT+FFD		MULTIFIT+MFFD		$13/11 + 2^{-k}$	stabil
	Mittel	Max.	Mittel	Max.		
1	1,2584	1,4997	1,2584	1,4997	1,6818	0,2 %
2	1,1230	1,2497	1,1230	1,2497	1,4318	1,9 %
3	1,0624	1,1705	1,0624	1,1705	1,3068	6,3 %
4	1,0346	1,1400	1,0345	1,1400	1,2443	14,6 %
5	1,0233	1,1221	1,0232	1,1221	1,2131	27,7 %
6	1,0184	1,1221	1,0183	1,1221	1,1974	48,0 %
7	1,0163	1,1221	1,0162	1,1221	1,1896	69,6 %
8	1,0157	1,1221	1,0156	1,1221	1,1857	83,1 %
9	1,0155	1,1221	1,0155	1,1221	1,1838	91,5 %
10	1,0154	1,1221	1,0154	1,1221	1,1828	100,0 %

Tabelle 5.7: Verhältnis $C_{\max}^{\text{MF}}/C_{\max}^*$ in Abhängigkeit der Binärsuchtiefe k über die 8 512 Instanzen mit bekanntem Optimum. *Mittel* ist das arithmetische Mittel, $13/11 + 2^{-k}$ die für MULTIFIT+FFD bewiesene Schranke. Die letzte Spalte gibt den Anteil der Instanzen an, auf denen MULTIFIT+FFD bei k bereits denselben Makespan liefert wie bei $k = 10$ (die letzte Zeile daher definitionsgemäß 100 %).

größte Binlast über der der bisher gespeicherten Packung liegt. Bei FFD ist das ausgeschlossen: Hat die von FFD bei Kapazität C erzeugte Packung die größte Binlast L , so trifft FFD für jede Kapazität zwischen L und C dieselben Platzierungsentscheidungen und liefert dieselbe Packung; die gespeicherte Binlast kann mit wachsendem k also nicht steigen. MFFD teilt die Aufgaben dagegen relativ zu C in Klassen ein und kann bei kleinerem C schlechter packen. Über alle 9092 Instanzen und alle Übergänge $k \rightarrow k+1$ zählt die Messung entsprechend 0 solche Fälle bei MULTIFIT+FFD und 3 bei MULTIFIT+MFFD; die größte dabei gemessene Verschlechterung beträgt 4,9 % (Instanz `p_cmax-n10-m4-planted-U100-perturb0.05-seed1.txt`: Makespan 102 bei $k = 5$, 107 bei $k = 6$, gemessen mit MULTIFIT+MFFD).

5.1.8 Fallstudie: die Friesen-Instanz

Keine Instanz der fünf Datensätze erreicht das Verhältnis 13/11. Die von Friesen [Fri84] angegebene Instanz, auf der MULTIFIT+FFD diesen Wert annimmt, erlaubt es, das Verhalten beider Subroutinen an einem Extremfall nachzuvollziehen.

Die Instanz besteht aus $n = 39$ Aufgaben auf $m = 13$ Maschinen: achtmal $(40, 13, 13)$, dreimal $(25, 25, 16)$ und zweimal $(25, 24, 17)$. Die Gesamtlast ist $858 = 13 \cdot 66$, und es existiert eine Zuordnung, bei der jede Maschine exakt 66 Lasteinheiten erhält; also $C_{\max}^* = 66$. Wegen $\rho = 858/(13 \cdot 40) = 1,65 < 2$ dürfen sich die Subroutinen nach Lemma 4.1 unterscheiden - und sie tun es:

	MULTIFIT+FFD	MULTIFIT+MFFD
Makespan ($k = 10$)	78	75
Verhältnis zu $C_{\max}^*$	$78/66 = 13/11$	$75/66 \approx 1,1364$

Der Unterschied entsteht bei der Kapazität $C = 75$. Dort ist $C/2 = 37,5$, sodass die acht 40er-Aufgaben *large* sind, und $C/6 = 12,5$, sodass auch die 13er-Aufgaben noch *small* sind. Phase 3 von MFFD kombiniert daher jeden 40er-Bin mit einer 13er-Aufgabe und einer weiteren passenden Aufgabe; MFFD packt die Instanz in 13 Bins, FFD benötigt 14. Bei $C = 78$ dagegen ist $C/6 = 13$ und die 13er-Aufgaben sind *tiny*; Phase 3 bleibt für sie wirkungslos.

Abbildung 5.6 zeigt die Binzahlen beider Subroutinen für jede ganzzahlige Kapazität von $C_{\max}^* = 66$ bis 90. FFD benötigt für jede Kapazität von 66 bis 77 vierzehn Bins und fällt bei $C = 78$ auf elf; die Binärsuche akzeptiert daher erst $C = 78$. MFFD kommt bereits ab $C = 75$ mit 13 Bins aus, weshalb MULTIFIT+MFFD gegen 75 konvergiert. Erreicht wird dieser Wert ab $k \geq 6$; bei kleinerem k liegt das Suchintervall noch zu grob (bei $k = 5$ ergibt sich 76).

Der Übergang von $C = 79$ auf $C = 80$ belegt zugleich, dass die Binzahl von FFD nicht monoton in C fällt. Bei $C = 79$ kombiniert First Fit jede 40er-Aufgabe mit einer 25er- und einer 13er-Aufgabe zur Last 78 und füllt so elf Bins. Bei $C = 80$ passen zwei 40er-Aufgaben gemeinsam in einen Bin; dadurch entfällt diese Kombination, die 25er-Aufgaben belegen eigene Bins, und am Ende bleibt eine 13er-Aufgabe allein in einem zwölften Bin. Erst ab $C = 84$ kommt FFD wieder mit elf Bins aus. Auf dieser Instanz

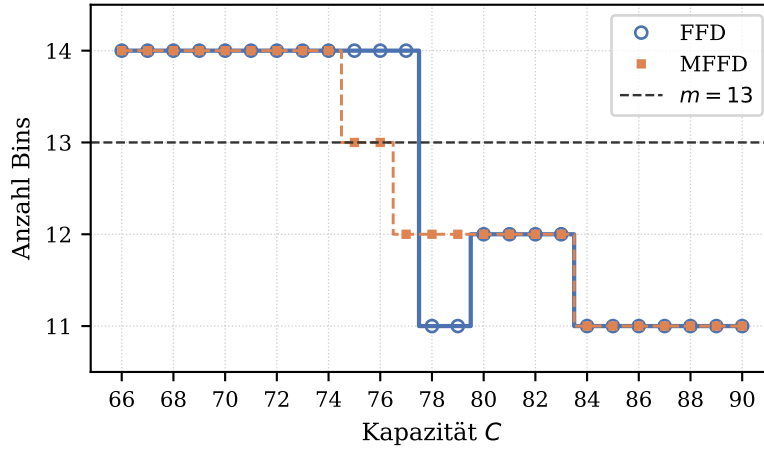


Abbildung 5.6: Anzahl der Bins von FFD und MFFD auf der Friesen-Instanz für jede ganzzahlige Kapazität $C = 66, \dots, 90$. MULTIFIT akzeptiert eine Kapazität, wenn die Binzahl auf oder unter der gestrichelten Linie $m = 13$ liegt. Beim Übergang von $C = 79$ auf $C = 80$ steigt die Binzahl von FFD.

bleibt das ohne Folgen für MULTIFIT, da beide Werte unter $m = 13$ liegen und das Akzeptanzkriterium $\text{FFD}[\Gamma, C] \leq m$ damit in beiden Fällen erfüllt ist.

Dass MULTIFIT+MFFD auf dieser Instanz besser abschneidet, sagt nichts über sein Worst-Case-Verhältnis aus. Ob MULTIFIT+MFFD eine kleinere Schranke als MULTIFIT+FFD erfüllt, ist offen; r_m ist nach Definition 2.7 allein über FFD festgelegt.

5.1.9 Grenzen der Auswertung

- **Herkunft der Optima.** Die OPT-Werte für Frangioni, Lawrinenko, Berndt und einen Teil der Planted-Instanzen stammen aus Akram u. a. [Akr+25] und wurden nicht selbst nachgerechnet. Sie gehen in Abschnitt 5.1.5 unmittelbar in das Verhältnis $C_{\max}^{\text{MF}}/C_{\max}^*$ ein. Die über (5.1) geführte Argumentation ist davon unabhängig.
- **Laufzeitmessung.** Jede Instanz wurde einmal gemessen. Die Werte in Tabelle 5.6 sind Mediane über viele Instanzen, aber keine Mittel über wiederholte Läufe derselben Instanz; einzelne Werte enthalten Messrauschen. Die absoluten Zeiten gelten für die beschriebene Python-Implementierung und Umgebung.
- **Abdeckung des Parameterraums.** 38,4% der Instanzen erfüllen $\rho \geq 2$ und sind nach Lemma 4.1 für den Subroutinenvergleich ohne Aussagekraft. Die verbleibenden 5603 Instanzen konzentrieren sich auf $\rho \in [1; 2]$; sehr kleine Werte $\rho < 0,5$ kommen nur elfmal vor.
- **Reichweite der Ergebnisse.** Alle Angaben beziehen sich auf $k = 10$ und auf diese fünf Datensätze. Aus dem Ausbleiben eines Verhältnisses über 13/11 folgt keine

schärfere Schranke, und aus den beobachteten Abweichungen folgt keine Aussage über das Worst-Case-Verhalten von MULTIFIT+MFFD.

5.2 High-Multiplicity-Subroutinen

Kommen in einer Instanz nur d verschiedene Aufgabengrößen vor, lässt sie sich kompakter kodieren: Statt der n einzelnen Größen genügen die verschiedenen Größen $p_1, \dots, p_d$ und ihre Vielfachheiten $u_1, \dots, u_d$ (High-Multiplicity-Darstellung, Abschnitt 4.2). Die Kodierungslänge liegt dann in $O(d \cdot (\log p_{\max} + \log n))$, während die Liste der Aufgabengrößen n Zahlen umfasst. Erst in dieser Kodierung ist eine Laufzeit polynomiell in $\log n$, wie sie Abschnitt 4.2.1 angibt, überhaupt möglich, denn schon das Einlesen von n einzelnen Größen kostet $\Omega(n)$. Kapitel 4 stellt neben MFFD drei Subroutinen vor, welche diese Darstellung ausnutzen: den OPT+1-Algorithmus von Jansen und Solis-Oba [JS10] (Abschnitt 4.2), seine ε -duale Modifikation (Abschnitt 4.2.4) und das Konfigurations-LP von Gilmore und Gomory (Abschnitt 4.3). Dieser Abschnitt misst, was sie als Subroutine in MULTIFIT leisten.

Gegenüber Abschnitt 5.1 kommt dabei ein Hilfsmittel hinzu: Dasselbe Konfigurationsprogramm liefert in ganzzahliger Fassung $\text{OPT}[\Gamma, C]$ exakt. Bei jeder getesteten Kapazität ist damit bekannt, wie viele Bins mindestens nötig sind - die Schranken aus Kapitel 4 lassen sich nachrechnen, statt sie nur zu zitieren.

5.2.1 Datensätze

Die fünf Datensätze aus Abschnitt 5.1.1 sind für diesen Vergleich ungeeignet: Ihre Instanzen enthalten im Median zwischen 20 und 81 verschiedene Aufgabengrößen, im Maximum 2363. Nach Abschnitt 4.2.3 setzen die Verfahren ein konstantes und sehr kleines d voraus. Für den Vergleich wurden deshalb zwei Datensätze erzeugt.

HM-Gitter. Der erste Datensatz - HM steht für High Multiplicity - umfasst 1080 Instanzen aus einem im Entwurf vollständig balancierten Gitter: $d \in \{3, \dots, 6\}$ verschiedene Aufgabengrößen, drei Szenarien für deren Verteilung, drei Maschinenzahlen zwischen 5 und 20 und drei Verhältnisse n/m mit je zehn Instanzen (Tabelle 5.8). Die Größenbereiche sind aus $\varepsilon = 1/(2d - 1)$ abgeleitet, also aus der Groß-Klein-Schwelle des OPT+1-Algorithmus selbst. Die Instanzen sind klein (n zwischen 15 und 160); variiert werden d , m und n/m , nicht die Vielfachheiten.

HM-Skalierung. 120 Instanzen, die die fehlende Achse ergänzen: $d \in \{3, 4\}$, dieselben drei Szenarien, n von 40 bis 40000 bei festem $n/m = 8$ (also m bis 5000), je fünf Instanzen. Das feste Verhältnis ist konstruktionsbedingt nötig: Bei festem m wüchse mit n auch das Startintervall der Binärsuche (2.2) und mit ihm die Zahl der Konfigurationen; bei festem n/m bleibt die getestete Kapazität in derselben Größenordnung, und es wachsen allein die Vielfachheiten. Damit ein Konfigurations-LP je Kapazität lösbar bleibt, verwirft der Generator jede Größenkombination, bei welcher die obere Schranke $\prod_{i=1}^d (\lfloor C/p_i \rfloor + 1)$

Szenario	Instanzen	n	m	d	ρ
Dense	360	15–160	5–20	3–6	1,34–7,74
Mixed	360	15–160	5–20	3–6	0,52–7,74
Asymmetric	360	15–160	5–20	3–6	0,67–7,00
Gesamt	1 080	15–160	5–20	3–6	0,52–7,74

Tabelle 5.8: Aufbau des HM-Gitters. Je Szenario 360 Instanzen aus $d \in \{3, \dots, 6\}$, drei Maschinenzahlen, drei Verhältnissen n/m und zehn Instanzen je Kombination.

für die Zahl der Konfigurationen über 200 000 liegt. Dabei ist $C = 8\bar{p} + p_{\max}$ mit dem Mittelwert $\bar{p}$ der d Größen, eine von n unabhängige Näherung der oberen Grenze des Startintervalls. Diese Grenze beschränkt d : Schon für $d = 5$ hält im Szenario Asymmetric keine von 2 000 zufällig gezogenen Größenkombinationen sie ein, während für $d \leq 4$ in allen drei Szenarien zulässige Kombinationen existieren. Dieser Datensatz wird nur in Abschnitt 5.2.6 ausgewertet.

Referenzwerte. Für alle 1 080 Instanzen des HM-Gitters wurde $C_{\max}^*(\Gamma)$ für diese Arbeit exakt bestimmt. Nach Lemma 2.5 ist $C_{\max}^*(\Gamma) = \min\{C : \text{OPT}[\Gamma, C] \leq m\}$, und da $\text{OPT}[\Gamma, \cdot]$ monoton fällt, findet eine Binärsuche über C diesen Wert, sobald sich $\text{OPT}[\Gamma, C]$ berechnen lässt. Das leistet das ganzzahlige Konfigurationsprogramm; seine Größe hängt nur von d und C ab, nicht von n oder m . Alle 1 080 Werte sind auf diesem Weg als optimal nachgewiesen.

Degenerierte Instanzen. 24 Instanzen des HM-Gitters erfüllen $\rho(\Gamma) < 1$, bei ihnen übersteigt die größte Aufgabe also die mittlere Maschinenlast; bei 23 von ihnen ist $C_{\max}^*(\Gamma) = p_{\max}$, die untere Schranke aus Lemma 2.2 wird dort angenommen. Auf allen 24 erreichen alle fünf Subroutinen den optimalen Makespan; sie trennen die Verfahren nicht und bleiben in den Gütevergleichen unberücksichtigt. Tabelle 5.9 beruht deshalb auf den übrigen Instanzen, während der Makespan-Vergleich in Tabelle 5.10 alle 1 080 umfasst.

5.2.2 Versuchsaufbau

Subroutinen. Gemessen werden fünf Subroutinen: FFD als Grundlinie, der $\text{OPT} + 1$ -Algorithmus (J), seine ε -duale Variante (J_ε), das Konfigurations-LP mit Rundung (GG) und dasselbe Programm in ganzzahliger Fassung (GG-IP). Letzteres löst $\text{OPT}[\Gamma, C]$ exakt und dient als Referenzlinie.

Implementierungsvarianten der beiden Jansen-Algorithmen. J und J_ε unterscheiden sich nicht nur im Rundungsschritt, sondern auch in der Anzahl der Aufrufe des Löser. Für die Laufzeitmessung in Abschnitt 5.2.5 ist das der entscheidende Unterschied. J wird

mit der inneren Binärsuche gemessen, also in der Suchstruktur der publizierten Fassung; im Verteilen der Reste weicht die Implementierung nach Abschnitt 4.2.2 von ihr ab. Schritt 2 bestimmt dort das kleinste m^* , für welches das MILP zulässig ist, per Binärsuche über $\{1, \dots, n\}$ (Abschnitt 4.2.2); jeder Suchschritt löst ein MILP und halbiert das Intervall, sodass je Kapazität $O(\log n)$ MILP-Aufrufe anfallen. Nötig ist diese Suche, obwohl MULTIFIT die Maschinenzahl m vorgibt: Ein einzelner Aufruf mit m an Stelle von m^* kann wegen des zusätzlichen Bins aus Schritt 3 insgesamt $m + 1$ Bins ausgeben und die Kapazität verwerfen, obwohl die publizierte Form mit $m^* < m$ höchstens m Bins ausgibt und sie annimmt (Abschnitt 4.2.4). J_ε verteilt die Reste dagegen auf die bereits geöffneten Bins und wird deshalb in der Fassung aus Abschnitt 4.2.4 ohne innere Binärsuche gemessen: Je Kapazität wird das MILP einmal mit der Nebenbedingung $\sum_b y_b \leq m$ gelöst.

Parameter. Alle fünf Subroutinen laufen mit $k = 10$ wie in Abschnitt 5.1.2, aber mit *ganzzahligen* Kapazitäten. Da alle Aufgabengrößen ganzzahlig sind, ist jede Binlast ganzzahlig; eine Kapazität C lässt daher dieselben Packungen zu wie $\lfloor C \rfloor$. Die Binärsuche testet deshalb nur ganze Zahlen, setzt nach einer Ablehnung von C die untere Grenze auf $C + 1$ und endet, sobald sich untere und obere Grenze treffen. So prüft jeder Aufruf der Subroutine eine andere Kapazität. Theorem 3.12 überträgt sich auf diese Variante: Bei ganzzahligen Größen ist jede Binlast ganzzahlig, also trifft FFD bei jedem $C' \in [C, C + 1)$ dieselben Entscheidungen wie bei C . Wird C verworfen, so gilt nach (2.3) $C' \leq r_m \cdot C_{\max}^*(\Gamma)$ für alle diese C' , im Supremum also $C + 1 \leq r_m \cdot C_{\max}^*(\Gamma)$ - die Invariante überlebt das Setzen von $c_{\text{low}} := C + 1$. Da sich die Intervallbreite in beiden Zweigen mindestens halbiert, trägt auch der Term 2^{-k} . Für Korollar 4.9 ist das nicht geführt. FFD wird deshalb in dieser Variante neu gemessen und nicht aus Abschnitt 5.1.6 übernommen. Jede Instanz wird von allen fünf Subroutinen im selben Prozess bearbeitet.

Messumgebung. Umgebung wie in Abschnitt 5.1.2; die vier übrigen Subroutinen lösen ihre Programme mit Gurobi 13.0.2. Die Laufzeit umfasst den vollständigen Lauf von MULTIFIT einschließlich aller Aufrufe des Löser, nicht einzelne Packvorgänge.

Reproduktion. Die Messung übernimmt `run_hm_benchmark.py` und legt zwei Dateien ab: eine Zeile je Instanz mit Makespan und Laufzeit jeder Subroutine sowie eine Zeile je Tripel aus Instanz, getesteter Kapazität und Subroutine mit den Kennzahlen des jeweiligen Aufrufs. Daraus erzeugen `summary_tables_hm.py` die Tabellen und Kennzahlen sowie `thesis_figures_hm.py` die Abbildungen. Diese Kette ist von der aus Abschnitt 5.1.2 getrennt.

5.2.3 Güte gegenüber dem Optimum

Tabelle 5.9 und Abbildung 5.7 zeigen den Abstand zum Optimum. MULTIFIT+FFD erreicht auf 56,3% der Instanzen den optimalen Makespan, im schlechtesten Fall das Verhältnis 1,1429. Die drei High-Multiplicity-Subroutinen liegen darüber: 97,3 % für J,

98,6 % für J_ε und 92,4 % für GG; ihre größten Verhältnisse liegen zwischen 1,0303 und 1,0317.

d	Instanzen	Maximum von $C_{\max}^{\text{MF}}/C_{\max}^*$				
		FFD	Jansen OPT+1	Jansen ε -dual	GG-LP	GG-IP (exakt)
3	270	1,1429	1,0317	1,0303	1,0303	1,0000
4	271	1,1327	1,0292	1,0216	1,0254	1,0000
5	259	1,0920	1,0247	1,0232	1,0296	1,0000
6	256	1,1154	1,0116	1,0217	1,0317	1,0000
alle d	1056	1,1429	1,0317	1,0303	1,0317	1,0000

Tabelle 5.9: Größtes Verhältnis $C_{\max}^{\text{MF}}/C_{\max}^*$ je Subroutine, gestaffelt nach der Anzahl d verschiedener Aufgabengrößen. Die 24 Instanzen mit $\rho < 1$ sind ausgenommen.

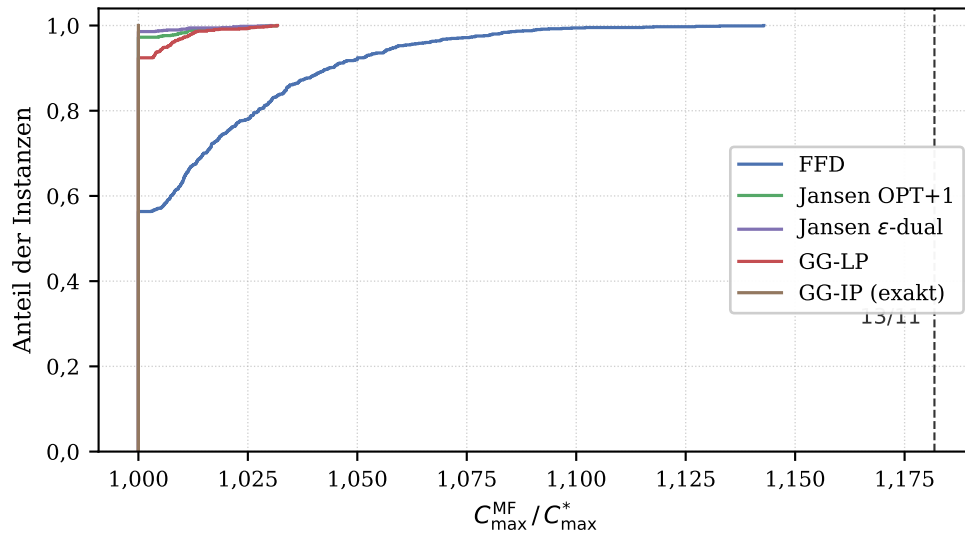


Abbildung 5.7: Empirische Verteilungsfunktion von $C_{\max}^{\text{MF}}/C_{\max}^*$ je Subroutine. Die Verteilung von GG-IP liegt konstruktionsbedingt vollständig bei 1.

Die Spalten der vier nicht exakten Subroutinen sind dabei unterschiedlich zu lesen. Die Spalte zu MULTIFIT+FFD steht der bewiesenen Schranke $13/11 + 2^{-k} \approx 1,1828$ nach Cao [Cao95] gegenüber; sie gilt nach Abschnitt 5.2.2 auch für die hier gemessene ganzzahlige Suche. Die Spalte zu MULTIFIT+ J_ε steht dem Wert $(1 + \varepsilon)(1 + 2^{-k})$ aus Korollar 4.9 gegenüber, wobei $\varepsilon = 1/(2d-1)$ hier zwischen 0,0909 und 0,2000 liegt. Dieser Wert ist nur eine Vergleichslinie: Gemessen wird die Fassung ohne innere Binärsuche, für die das Argument am Ende von Abschnitt 4.2.4 trägt und nicht Korollar 4.9 selbst. Für MULTIFIT+J beziffert Abschnitt 4.2.3 das Verhältnis zum Optimum je Instanz

durch $C_{\max}^*(\Gamma, m - 1)/C_{\max}^*(\Gamma, m)$; über alle Instanzen genommen ergibt das nur den Wert 2 und damit keine mit 13/11 vergleichbare Aussage. Für das Konfigurations-LP zeigt Abschnitt 4.3.4, dass Implikation (2.3) mit $r = 2$ trägt, sobald $m \geq 2(d - 1)$ gilt - auf dem HM-Gitter ist das für 75,0 % der Instanzen der Fall. Eine Schranke gewinnt man daraus nicht, denn das Startintervall hält den Faktor 2 ohnehin ein; ob eine von d unabhängige Konstante $r < 2$ existiert, ist offen. Für das Konfigurations-LP sind die gemessenen Werte damit die einzige verfügbare Aussage über das Verhältnis auf diesen Instanzen.

Der direkte Vergleich (Tabelle 5.10, Abbildung 5.8) fällt nicht einheitlich aus. Gegenüber MULTIFIT+FFD liefert MULTIFIT+J auf 453 Instanzen den kleineren Makespan - und auf 8 den größeren. Die kleinere obere Schranke für die Binzahl ($\text{OPT} + 1$ gegenüber (2.1)) garantiert also keinen kleineren Makespan; woran das auf diesen Instanzen liegt, zeigt Abschnitt 5.2.4.

Referenz A	Kandidat B	B besser	gleich	B schlechter	Instanzen
FFD	Jansen $\text{OPT}+1$	453	619	8	1080
FFD	Jansen ε -dual	457	618	5	1080
FFD	GG-LP	446	633	1	1080
FFD	GG-IP (exakt)	461	619	0	1080
Jansen $\text{OPT}+1$	Jansen ε -dual	23	1048	9	1080
Jansen $\text{OPT}+1$	GG-LP	24	984	72	1080
Jansen $\text{OPT}+1$	GG-IP (exakt)	29	1051	0	1080
Jansen ε -dual	GG-LP	13	989	78	1080
Jansen ε -dual	GG-IP (exakt)	15	1065	0	1080
GG-LP	GG-IP (exakt)	80	1000	0	1080

Tabelle 5.10: Paarweiser Vergleich der Makespans über alle 1080 Instanzen des HM-Gitters.

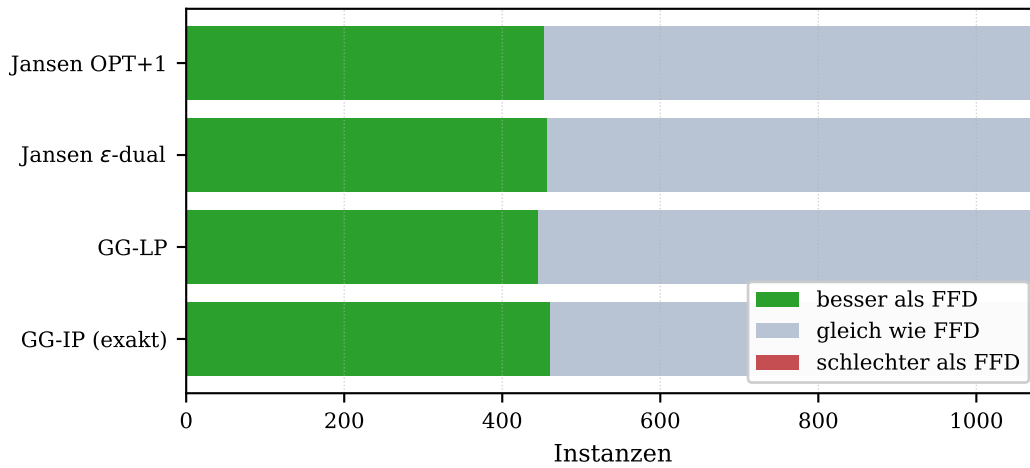


Abbildung 5.8: Jede Subroutine gegen die Grundlinie MULTIFIT+FFD.

5.2.4 Die Garantien im Test

Weil GG-IP OPT $[\Gamma, C]$ bei jeder getesteten Kapazität liefert, lassen sich die Schranken aus Kapitel 4 unmittelbar nachrechnen. Verglichen werden kann jedoch nur, wo beide Verfahren dieselbe Kapazität getestet haben; die Binärsuche läuft je Subroutine verschieden, sodass die Grundmenge kleiner ausfällt als die Spalte *Aufrufe* in Tabelle 5.11.

Für den OPT + 1-Algorithmus gilt in allen 7092 so verglichenen Aufrufen $J[\Gamma, C] - \text{OPT}[\Gamma, C] \in \{0, 1\}$; der Wert 1 tritt in 43 Fällen auf (0,6%), sonst ist die Packung optimal. Bewiesen ist diese Schranke für die publizierte Fassung; die Implementierung weicht nach Abschnitt 4.2.2 im Verteilen der Reste von ihr ab und hält sie in jedem gemessenen Aufruf ebenfalls ein.

Für das Konfigurations-LP erlaubt Lemma 4.13 den additiven Term $d - 1$, hier also bis zu 5. Gemessen wird er nicht annähernd erreicht: Über 7040 vergleichbare Aufrufe ist $\text{GG}[\Gamma, C] - \text{OPT}[\Gamma, C]$ höchstens 1 und in 95 Fällen überhaupt von null verschieden. Auch der Abstand zur LP-Schranke bleibt klein: In 98,5% der Aufrufe gilt $\text{GG}[\Gamma, C] = \lceil \text{LP}[\Gamma, C] \rceil$, der größte beobachtete Abstand ist 1.

Nicht ursächlich dafür ist die Trägerschranke $|F| \leq d$ aus Lemma 4.12: Sie wird ausgeschöpft. Die Zahl der fraktionalen Spalten erreicht im Maximum 6 und ist in 32,4% der Aufrufe gleich d , im Median beträgt sie 3. Klein bleibt der Verlust vielmehr durch die Rundung selbst. Schritt 3 aus Abschnitt 4.3.3 packt den Restbedarf nicht mit einem Bin je fraktionaler Spalte, sondern nimmt die kleinere der beiden dort beschriebenen Packungen: In 6497 Aufrufen bleibt überhaupt ein Rest; er kostet im Median 2 und höchstens 5 Bins und bleibt in 81,5% der Fälle unter $|F|$ - genau der Vorsprung des Abrundens, den Bemerkung 4.14 begründet. Die kleinere der beiden Packungen stammt dabei 4164-mal von FFD und 2333-mal aus der Musterpackung; keine der beiden Varianten ist durchweg die bessere.

Der ε -duale Algorithmus soll eine Last von höchstens $(1+\varepsilon)C$ zulassen, mit $\varepsilon = 1/(2d -$

1), hier also zwischen 0,0909 und 0,2000. Ausgeschöpft wird das nicht: Über die 5 595 Aufrufe, bei denen das Programm lösbar war und eine Packung entstand, überschreitet nur in 30 Fällen überhaupt ein Bin die Kapazität C . Die größte gemessene Überschreitung beträgt $0,0370$ und entspricht einer Last von höchstens $1,0370 \cdot C$.

Was MULTIFIT davon merkt. Für die Binärsuche zählt nicht die Packung, sondern die Entscheidung über die Kapazität. Tabelle 5.11 stellt sie den exakten Werten gegenüber; die letzte Spalte trägt die Erklärung für den Befund aus Abschnitt 5.2.3. FFD verwirft 774 Kapazitäten, für die eine Packung in m Bins existiert, der $\text{OPT} + 1$ -Algorithmus 38. Jede dieser 38 Ablehnungen tritt gemessen bei $J[\Gamma, C] = m + 1$ auf - dem Fall, der nach Abschnitt 4.2.3 zweideutig ist und deshalb konservativ als Ablehnung behandelt wird. Beide Verfahren verwerfen also zulässige Kapazitäten, nur an verschiedenen Stellen - deshalb fällt der Vergleich in Tabelle 5.10 in beide Richtungen aus. Für die Richtung, in der der $\text{OPT} + 1$ -Algorithmus verliert, lässt sich das nachprüfen: Er verwirft auf 29 Instanzen mindestens eine zulässige Kapazität, und alle 8 Instanzen, auf denen MULTIFIT mit ihm den größeren Makespan liefert, gehören dazu; 0 liegen außerhalb. Der ε -duale Algorithmus und das ganzzahlige Programm verwerfen dagegen keine einzige zulässige Kapazität (Spalte *zu Unrecht* in Tabelle 5.11: 0 und 0).

Ihre beiden Zeilen in Tabelle 5.11 stimmen vollständig überein, und das gilt nicht nur in der Summe: Auf allen 1 080 Instanzen akzeptieren die beiden Verfahren genau dieselben Kapazitäten. Dennoch liefert der ε -duale auf 15 von ihnen einen größeren Makespan (Tabelle 5.10). Die Ursache kann folglich nicht die Entscheidung über die Kapazität sein, sondern nur die zurückgegebene Packung: Nach Eigenschaft (ii) von Definition 4.6 darf ein Bin die Kapazität um den Faktor $1 + \varepsilon$ überschreiten, und auf diesen 15 Instanzen geschieht das - bis zum 1,0303-fachen der akzeptierten Kapazität. Für den Makespan zählt demnach nicht allein, welche Kapazitäten eine Subroutine akzeptiert, sondern bei einem ε -dualen Verfahren auch, wie hoch es die Bins dabei füllt.

Subroutine	Aufrufe	Entscheidung		davon	
		angen.	abgel.	zertifiziert	zu Unrecht
FFD	7 056	5 219	1 837	—	774
Jansen $\text{OPT} + 1$	7 137	5 573	1 564	—	38
Jansen ε -dual	7 146	5 595	1 551	—	0
GG-LP	7 124	5 544	1 580	1 485	95
GG-IP (exakt)	7 146	5 595	1 551	1 551	0

Tabelle 5.11: Entscheidungen der Subroutinen über alle getesteten Kapazitäten. *Zertifiziert* meint eine Ablehnung mit $\lceil \text{LP} \rceil > m$, aus der nach Abschnitt 4.3.4 $C < C_{\max}^*(\Gamma)$ folgt. *Zu Unrecht* zählt die Ablehnungen einer Kapazität, für die $\text{OPT}[\Gamma, C] \leq m$ gilt, für die es also eine gültige Packung in m Bins gibt.

Beim Konfigurations-LP zerfallen die Ablehnungen wie in Abschnitt 4.3.4 beschrieben: 1 485 sind zertifiziert, die übrigen 95 unentschieden (1,33 % aller Aufrufe). Von diesen 95 unentschiedenen Ablehnungen betreffen 95 eine tatsächlich zulässige Kapazität; eine

zertifizierte Ablehnung kann das nach Abschnitt 4.3.4 nie sein, da aus ihr $C < C_{\max}^*(\Gamma)$ folgt.

Nicht-Monotonie. Lemma 4.4 lässt zu, dass $J[\Gamma, \cdot]$ beim Übergang zu einer größeren Kapazität um eins steigt. Unter den Kapazitäten, die die Binärsuche selbst besucht, zählt die Messung 0 derartige Paare. Auf einer zusammenhängenden Kapazitätsreihe der Instanz `HM_dense_d5_n0080_m10_7` tritt der Fall dagegen auf: $J[\Gamma, 327] = 10$, aber $J[\Gamma, 328] = 11$.

5.2.5 Laufzeit

Tabelle 5.12 und Abbildung 5.9 zeigen die Laufzeit über d . FFD bleibt bei 0,6 ms im Median und ist von d unabhängig; die High-Multiplicity-Verfahren wachsen mit d , wie es die Größe der Konfigurationsmenge erwarten lässt (im Median 312 Spalten, im Maximum 1 378 772).

d	Instanzen	Median der Laufzeit [ms]				
		FFD	Jansen OPT+1	Jansen ε -dual	GG-LP	GG-IP (exakt)
3	270	0,6	43,4	10,8	5,1	8,4
4	271	0,6	91,6	20,2	9,9	15,5
5	270	0,6	250,1	56,5	26,9	38,9
6	269	0,6	673,3	141,6	70,3	111,1
alle d	1 080	0,6	125,4	25,7	12,8	19,4

Tabelle 5.12: Median der Laufzeit eines vollständigen Laufs von MULTIFIT je Subroutine, gestaffelt nach d .

Bemerkenswert ist die Reihenfolge: Das *exakte* Konfigurationsprogramm (19,4 ms) ist schneller als der OPT + 1-Algorithmus (125,4 ms). Die Ursache ist die Zahl der Aufrufe des Löser, nicht die Schwere der einzelnen Modelle. Ein einzelnes MILP des OPT + 1-Algorithmus braucht im Median 3,18 ms und damit kaum mehr als ein Konfigurationsprogramm mit 2,92 ms. Die innere Binärsuche nach m^* löst davon je Kapazität jedoch 6 Modelle im Median und bis zu 8, während das Konfigurationsprogramm mit einem auskommt. Das Maximum ist genau die Schrittzahl einer Binärsuche über $\{1, \dots, n\}$ bei $n \leq 160$, nämlich $\lfloor \log_2 160 \rfloor + 1 = 8$. Die ε -duale Variante, die diese Binärsuche durch die von außen bekannte Maschinenzahl ersetzt (Abschnitt 5.2.2), liegt mit 25,7 ms dazwischen.

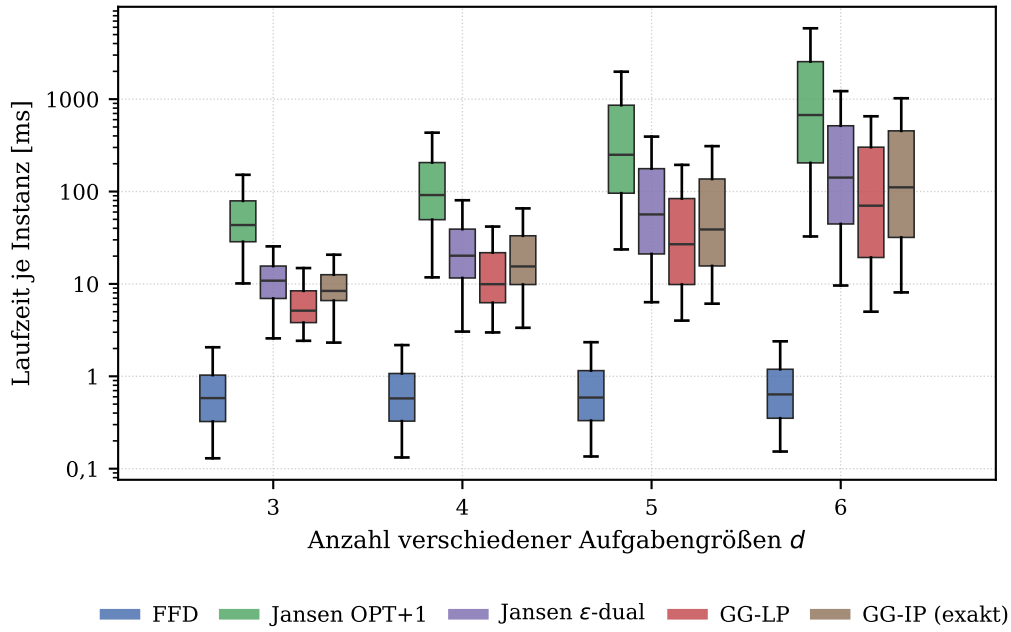


Abbildung 5.9: Laufzeitverteilung je Subroutine über d (logarithmische y -Achse, ohne Ausreißer).

5.2.6 Skalierung in der Multiplizität

Der Vorteil einer High-Multiplicity-Formulierung liegt darin, dass ihre Größe an der Anzahl d verschiedener Größen hängt und nicht an der Anzahl n der Aufgaben. Für den OPT + 1-Algorithmus ist die Zahl der großen Konfigurationen bei festem d konstant beschränkt (Abschnitt 4.2.2). Auf dem HM-Gitter mit $n \leq 160$ ist das nicht messbar; dafür ist die HM-Skalierung erzeugt worden.

Das Ergebnis trennt die Verfahren. Während n um den Faktor 1 000 wächst, steigt die Laufzeit von MULTIFIT+FFD um den Faktor 2 118,2 (Tabelle 5.13) - FFD verarbeitet jede Aufgabe einzeln und läuft in $O(n \log n)$ (Abschnitt 5.1.2). Die vier High-Multiplicity-Subroutinen wachsen nur um das 6,3- bis 9,4-fache. Bei $n = 40\,000$ ist das Konfigurations-LP mit 105,4 ms um ein Vielfaches schneller als FFD mit 979,8 ms; der Schnittpunkt der beiden Medianverläufe liegt nach log-linearer Interpolation zwischen den gemessenen Stützstellen bei etwa $n \approx 1\,400$ (Abbildung 5.10).

n	m	Median der Laufzeit [ms]				
		FFD	Jansen OPT+1	Jansen ε -dual	GG-LP	GG-IP (exakt)
40	5	0,5	105,0	22,6	12,5	18,2
400	50	5,7	416,9	55,9	22,7	37,0
4 000	500	79,9	372,7	46,6	23,9	33,8
40 000	5 000	979,8	991,1	142,2	105,4	124,3
Faktor		2 118,2	9,4	6,3	8,5	6,8

Tabelle 5.13: Median der Laufzeit über die Multiplizitätsachse auf dem Datensatz HM-Skalierung, bei festem $d \in \{3, 4\}$ und festem n/m . Die letzte Zeile gibt den Faktor zwischen der kleinsten und der größten Instanzgröße an; n selbst wächst dabei um den Faktor 1 000.

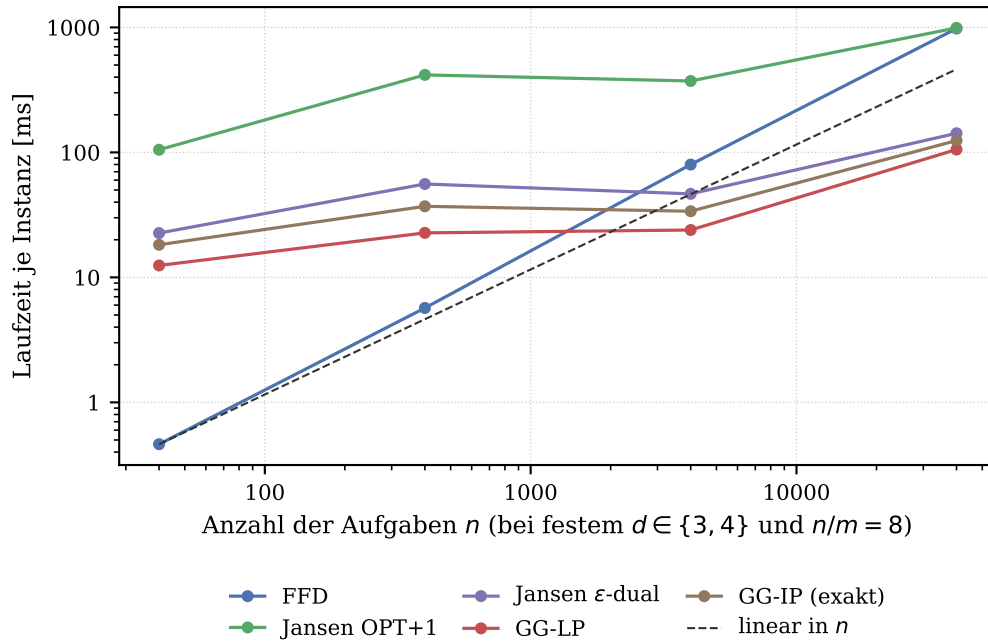


Abbildung 5.10: Laufzeit über n bei festem $d \in \{3, 4\}$ und festem n/m , doppelt logarithmisch. Die gestrichelte Linie wächst linear in n und dient als Bezug.

5.2.7 Einschränkungen dieses Vergleichs

- **Reichweite in d .** Alle Aussagen gelten für $d \leq 6$. Die Konfigurationsmenge wächst mit d ; auf den fünf Datensätzen aus Abschnitt 5.1.1 liegt d im Median zwischen 20 und 81 und damit außerhalb des Bereichs, für den Abschnitt 4.2.3 die Verfahren als sinnvoll ausweist. Gemessen wurden sie dort nicht.
- **Der additive Term.** Dass $\text{GG}[\Gamma, C] - \text{OPT}[\Gamma, C]$ hier 1 nicht überschreitet, ist eine Aussage über diese Instanzen und kein Beleg gegen die Schranke $d - 1$ aus Lemma 4.13.
- **Zwei Instanzen mit abweichendem d .** In 2 Instanzen erzeugt der Generator eine Größe mit Vielfachheit null, sodass die Instanz weniger Typen enthält, als ihr Name angibt. Die Auswertung liest d deshalb aus dem Instanzinhalt.
- **Laufzeitmessung.** Wie in Abschnitt 5.1.9: eine Messung je Instanz, Mediane über Instanzen statt über Wiederholungen. Die Zeiten gelten für Gurobi in der beschriebenen Umgebung; ein anderer Löser verschiebt sie.

6 Fazit

Zwei Fragen hat diese Arbeit verfolgt. Die erste galt der Gütegarantie von MULTIFIT und der Herleitung, welche zu ihr führt. Die zweite galt der Subroutine, also dem Bin-Packing-Verfahren, das über jede getestete Kapazität entscheidet. Beide Stränge treffen sich in Implikation (2.3): Dort wird aus einer Eigenschaft der Subroutine eine Aussage über den Makespan.

Die Performanzanalyse. Kapitel 3 führt den Nachweis $r_m \leq 5/4$ vollständig aus (Theorem 3.10). Getragen wird er von Struktureigenschaften eines minimalen Gegenbeispiels: dem Löschlemma, dem Dominanzlemma, der Mindestzahl von Aufgaben je Bin der q -Packung und den Größenschranken. Theorem 3.12 verbindet das Grenzverhältnis anschließend mit dem Verfahren und liefert $R_m(\text{MF}(k)) \leq r_m + 2^{-k}$. Die Binärsuche kostet also nur einen additiven Term, welcher sich mit jedem Schritt halbiert. In die Gegenrichtung führt Lemma 3.14 den Nachweis $r_m \geq 13/11$ für $m \geq 13$ an einer Instanz von Friesen [Fri84]. Zusammen mit der oberen Schranke von Cao [Cao95] ist das Grenzverhältnis für diese Maschinenzahlen exakt bestimmt.

Die Schranke $13/11$ selbst bleibt in dieser Arbeit unbewiesen; Abschnitt 3.9 stellt nur ihre Beweisidee dar. Caos Fallunterscheidung über Δ füllt sechzehn Seiten und stützt sich zusätzlich auf Yue [Yue90]. Zwischen der hier bewiesenen Schranke $5/4$ und dem für $m \geq 13$ exakten Wert $13/11$ bleibt somit eine Lücke; geschlossen wird sie in [Cao95] über diese Fallunterscheidung.

Der Austausch der Subroutine. Kapitel 4 prüft drei Alternativen, und die Befunde fallen verschieden aus. Für MFFD gibt Lemma 4.1 eine Klasse von Instanzen an, auf der sich die Wahl überhaupt nicht auswirkt: Ist die mittlere Maschinenlast mindestens doppelt so groß wie die größte Aufgabe, in Kurzform $\rho(\Gamma) \geq 2$, so erzeugen MFFD und FFD bei jeder getesteten Kapazität dieselbe Packung. Geprüft ist das Kriterium in einem einzigen Durchlauf der Aufgabenliste. Beim $\text{OPT} + 1$ -Algorithmus von Jansen und Solis-Oba [JS10] entsteht durch die Modifikation zu einem ε -dualen Verfahren eine eigene Garantie: Korollar 4.9 liefert den Makespan $(1 + \varepsilon)(1 + 2^{-k}) \cdot C_{\max}^*(\Gamma)$ mit $\varepsilon = 1/(2d - 1)$, wobei d die Anzahl verschiedener Aufgabengrößen bezeichnet. Neben der Schranke für MULTIFIT+FFD ist sie die einzige Makespan-Garantie dieser Arbeit, welche allein von d und k abhängt; die Schranke, die Abschnitt 4.2.3 für den $\text{OPT} + 1$ -Algorithmus angibt, hängt dagegen von der Instanz selbst ab.

Für das Konfigurations-LP von Gilmore und Gomory [GG61] fällt das Ergebnis schwächer aus. Lemma 4.13 lässt nach dem Runden $m + d - 1$ Bins zu und trägt Implikation (2.3) damit nicht für $r = 1$; erst ab $m \geq 2(d - 1)$ trägt sie mit $r = 2$, und eine von d unabhängige Konstante liefert das Lemma nicht. Was das Verfahren stattdessen mitbringt, ist

eine zertifizierte Ablehnung: Aus $\lceil \text{LP}[\Gamma, C] \rceil > m$ folgt $C < C_{\max}^*(\Gamma)$, während FFD im entscheidenden Fall $\text{FFD}[\Gamma, C] = m + 1$ nichts über $\text{OPT}[\Gamma, C]$ aussagt.

Was die Messungen zeigen. Auf den 9 092 Instanzen der fünf Benchmark-Datensätze überschreitet kein gemessenes Verhältnis 13/11; der größte Wert beträgt 1,1221. MULTIFIT+MFFD liefert dabei in 200 Fällen einen kleineren und in 180 Fällen einen größeren Makespan als MULTIFIT+FFD. Sämtliche 380 Abweichungen liegen unterhalb von $\rho = 2$, wie es Lemma 4.1 verlangt. Ein systematischer Vorteil der aufwendigeren Subroutine zeigt sich auf diesen Instanzen nicht.

Die zweite Versuchsreihe trennt Bin-Packing-Güte und Makespan noch deutlicher. Auf dem HM-Gitter erreicht MULTIFIT+FFD auf 56,3 % der Instanzen den optimalen Makespan, mit dem $\text{OPT} + 1$ -Algorithmus sind es 97,3 %. Gegenüber der Grundlinie gewinnt MULTIFIT mit diesem Algorithmus dennoch nicht durchgängig: 453 Instanzen fallen besser aus, 8 schlechter. Den Grund liefert Abschnitt 5.2.4: FFD verwirft 774 Kapazitäten, für die eine Packung in m Bins existiert, der $\text{OPT} + 1$ -Algorithmus 38 - nur an anderen Stellen. Eine im Mittel bessere Binzahl schlägt also nicht auf den Makespan durch, sondern erst die Frage, bei welchen Kapazitäten die Binzahl die Schwelle m nicht überschreitet; genau an diese Entscheidung knüpft Implikation (2.3) die Gütegarantie von MULTIFIT+FFD. Die Entscheidung allein trägt den Makespan jedoch nicht: Der ε -duale Algorithmus akzeptiert nach Abschnitt 5.2.4 auf allen 1 080 Instanzen dieselben Kapazitäten wie das exakte Konfigurationsprogramm und liefert auf 15 von ihnen dennoch einen größeren Makespan, weil er einen Bin über die akzeptierte Kapazität hinaus füllen darf.

Die Laufzeiten ordnen sich ebenfalls anders, als es die Güte nahelegt. Das exakte Konfigurationsprogramm liegt mit 19,4 ms im Median vor dem $\text{OPT} + 1$ -Algorithmus mit 125,4 ms, dessen innere Binärsuche mehrere Modelle je Kapazität löst. Wächst die Anzahl der Aufgaben um den Faktor 1 000, so steigt die Laufzeit von MULTIFIT+FFD um das 2 118,2-fache, die der High-Multiplicity-Verfahren dagegen nur um das 6,3- bis 9,4-fache.

Grenzen der Aussagen. Aus den Messungen folgt keine Schranke. Dass auf 9 092 Instanzen kein Verhältnis über 13/11 auftritt, ist für MULTIFIT+FFD mit der bewiesenen Schranke verträglich und begründet für MULTIFIT+MFFD keine. Ähnliches gilt für das Kollaps-Lemma: Es beschreibt eine Klasse, auf der die Subroutinenwahl folgenlos bleibt, und gerade der bekannte Extremfall liegt außerhalb, denn die Instanz von Friesen [Fri84] hat $\rho = 1,65$. Zur oberen Schranke trägt das Lemma somit nichts bei.

Die zweite Versuchsreihe deckt zudem nur $d \leq 6$ ab, während die fünf Benchmark-Datensätze im Median zwischen 20 und 81 verschiedene Aufgabengrößen enthalten; dort sind die High-Multiplicity-Verfahren nicht gemessen worden. Ein Teil der verwendeten Optima stammt aus Akram u. a. [Akr+25] und ist nicht nachgerechnet worden, und sämtliche Laufzeiten gelten für die beschriebene Implementierung und Umgebung.

Offene Fragen. Die naheliegendste betrifft MFFD. Auf der Instanz von Friesen [Fri84] liefert MULTIFIT+MFFD den kleineren Makespan (Abschnitt 5.1.8); über das Verhalten im schlechtesten Fall sagt das jedoch nichts, und ob MULTIFIT+MFFD eine kleinere Schranke als MULTIFIT+FFD erfüllt, bleibt offen - Definition 2.7 legt r_m allein über FFD fest, sodass die Frage für MFFD erst über eine eigene Fassung dieser Definition zu stellen wäre. Dahinter steht die allgemeinere Frage, für welche Subroutinen eine Entsprechung von (2.3) überhaupt besteht: Für FFD ist sie bewiesen, für MFFD und für das Konfigurations-LP unbekannt - beim Konfigurations-LP reicht die Rundungsschranke aus Lemma 4.13 dafür nicht aus, ein Gegenbeispiel ist damit aber nicht gegeben. Unbestimmt bleibt ferner das Grenzverhältnis für kleine Maschinenzahlen, denn Lemma 3.14 setzt $m \geq 13$ voraus; für $2 \leq m < 13$ klafft zwischen $r_m \geq 1$ aus Bemerkung 3.11 und der oberen Schranke $13/11$ eine Lücke, und nur $r_1 = 1$ ist bestimmt.

Neu entstanden ist die Frage nach der Reichweite der ε -dualen Konstruktion. Über die von Jansen und Solis-Oba [JS10] übernommene Wahl $\varepsilon = 1/(2d - 1)$ wird ihre Garantie mit wachsender Anzahl verschiedener Aufgabengrößen schärfer, die Laufzeit des Verfahrens nach Abschnitt 4.2.1 jedoch doppelt exponentiell in d . Lemma 4.8 selbst verlangt diese Kopplung nicht; sie begrenzt die Zahl der großen Konfigurationen und damit die Größe des Programms. Am schwächsten ist die Garantie demnach dort, wo sie sich berechnen lässt. Die in Abschnitt 4.3.1 beschriebene Spaltengenerierung hält das Konfigurationsmodell auch bei wachsendem d handhabbar; implementiert ist sie hier nicht, und auf das ganzzahlige Programm des ε -dualen Verfahrens ist sie nicht übertragen worden. Zu klären wäre schließlich, ob sich die zertifizierte Ablehnung des Konfigurations-LPs mit einer schnellen Subroutine verbinden lässt, welche das Suchintervall zwar nicht beweisbar eingrenzt, dafür aber deutlich schneller entscheidet (Abschnitt 5.2.5).

Literatur

- [Akr+25] Matthew Akram u. a. „Engineering Optimal Parallel Task Scheduling“. In: *2025 Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX)*. Society for Industrial und Applied Mathematics, 2025, S. 168–180. ISBN: 9781611978339. DOI: 10.1137/1.9781611978339.14.
- [Ber+22] Sebastian Berndt u. a. „Load Balancing: The Long Road from Theory to Practice“. In: *2022 Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX)*. Society for Industrial und Applied Mathematics, 2022, S. 104–116. DOI: 10.1137/1.9781611977042.9.
- [Cao95] Feng Cao. „Determining the Performance Ratio of Algorithm MULTIFIT for Scheduling“. In: *Minimax and Applications*. Hrsg. von Ding-Zhu Du und Panos M. Pardalos. Dordrecht: Kluwer Academic Publishers, 1995, S. 79–96.
- [CGJ78] E. G. Coffman Jr., M. R. Garey und D. S. Johnson. „An Application of Bin-Packing to Multiprocessor Scheduling“. In: *SIAM Journal on Computing* 7.1 (1978), S. 1–17. DOI: 10.1137/0207001.
- [Dós07] György Dósa. „The Tight Bound of First Fit Decreasing Bin-Packing Algorithm is $\text{FFD}(\text{I}) \leq 11/9 \text{OPT}(\text{I}) + 6/9$ “. In: *Combinatorics, Algorithms, Probabilistic and Experimental Methodologies. First International Symposium, ESCAPE 2007, Hangzhou, China, April 7–9, 2007*. Hrsg. von Bo Chen, Mike Paterson und Guochuan Zhang. Bd. 4614. Lecture Notes in Computer Science. Springer, 2007, S. 1–11. ISBN: 978-3-540-74449-8. DOI: 10.1007/978-3-540-74450-4_1.
- [ES06] Friedrich Eisenbrand und Gennady Shmonin. „Carathéodory bounds for integer cones“. In: *Operations Research Letters* 34.5 (2006), S. 564–568. DOI: 10.1016/j.orl.2005.09.008.
- [FL86] Donald K. Friesen und Michael A. Langston. „Evaluation of a MULTIFIT-based scheduling algorithm“. In: *Journal of Algorithms* 7.1 (1986), S. 35–59. DOI: 10.1016/0196-6774(86)90037-4.
- [FNS04] Antonio Frangioni, Emiliano Necciari und Maria Grazia Scutellà. „A Multi-Exchange Neighborhood for Minimum Makespan Parallel Machine Scheduling Problems“. In: *Journal of Combinatorial Optimization* 8.2 (2004), S. 195–220. DOI: 10.1023/B:JOCO.0000031420.05971.29.
- [Fra+94] Paulo M. França u. a. „A composite heuristic for the identical parallel machine scheduling problem with minimum makespan objective“. In: *Computers & Operations Research* 21.2 (1994), S. 205–210. DOI: 10.1016/0305-0548(94)90053-1.

- [Fri84] Donald K. Friesen. „Tighter Bounds for the Multifit Processor Scheduling Algorithm“. In: *SIAM Journal on Computing* 13.1 (1984), S. 170–181. DOI: 10.1137/0213013.
- [GG61] P. C. Gilmore und R. E. Gomory. „A Linear Programming Approach to the Cutting-Stock Problem“. In: *Operations Research* 9.6 (1961), S. 849–859. DOI: 10.1287/opre.9.6.849.
- [GG63] P. C. Gilmore und R. E. Gomory. „A Linear Programming Approach to the Cutting Stock Problem—Part II“. In: *Operations Research* 11.6 (1963), S. 863–888. DOI: 10.1287/opre.11.6.863.
- [GJ79] Michael R. Garey und David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco: W. H. Freeman, 1979. ISBN: 0716710455.
- [Gra+79] R. L. Graham u. a. „Optimization and Approximation in Deterministic Sequencing and Scheduling: A Survey“. In: *Annals of Discrete Mathematics* 5 (1979), S. 287–326. DOI: 10.1016/S0167-5060(08)70356-X.
- [Gra66] R. L. Graham. „Bounds for Certain Multiprocessing Anomalies“. In: *Bell System Technical Journal* 45.9 (1966), S. 1563–1581. DOI: 10.1002/j.1538-7305.1966.tb01709.x.
- [Gra69] R. L. Graham. „Bounds on Multiprocessing Timing Anomalies“. In: *SIAM Journal on Applied Mathematics* 17.2 (1969), S. 416–429. DOI: 10.1137/0117039.
- [HS87] Dorit S. Hochbaum und David B. Shmoys. „Using Dual Approximation Algorithms for Scheduling Problems: Theoretical and Practical Results“. In: *Journal of the ACM* 34.1 (1987), S. 144–162. DOI: 10.1145/7531.7535.
- [JG85] David S. Johnson und Michael R. Garey. „A 71/60 theorem for bin packing“. In: *Journal of Complexity* 1.1 (1985), S. 65–106. DOI: 10.1016/0885-064X(85)90022-6.
- [JS10] Klaus Jansen und Roberto Solis-Oba. *An Opt + 1 algorithm for the cutting stock problem with a constant number of object lengths*. Techn. Ber. 1007. Kiel: Institut für Informatik der Christian-Albrechts-Universität zu Kiel, 2010. URL: https://macau.uni-kiel.de/receive/macau_mods_00001831.
- [Len83] H. W. Lenstra Jr. „Integer Programming with a Fixed Number of Variables“. In: *Mathematics of Operations Research* 8.4 (1983), S. 538–548. DOI: 10.1287/moor.8.4.538.
- [MS18] Mehdi Mrad und Nizar Souayah. „An Arc-Flow Model for the Makespan Minimization Problem on Identical Parallel Machines“. In: *IEEE Access* 6 (2018), S. 5300–5307. DOI: 10.1109/ACCESS.2018.2789678.
- [Sah76] Sartaj K. Sahni. „Algorithms for Scheduling Independent Tasks“. In: *Journal of the ACM* 23.1 (1976), S. 116–127. DOI: 10.1145/321921.321934.

- [WS11] David P. Williamson und David B. Shmoys. *The Design of Approximation Algorithms*. Frei verfügbare elektronische Ausgabe: <https://www.designofapproxalgs.com/book.pdf>. Cambridge University Press, 2011. ISBN: 978-0-521-19527-0.
- [YKY88] Minyi Yue, Hans Kellerer und Z. Yu. *A simple proof of the inequality $R_M(MF(k)) \leq 1.2 + (1/2)^k$ in multiprocessor scheduling*. Techn. Ber. 124. Graz: Institut für Mathematik, Technische Universität Graz, 1988, S. 1–10.
- [Yue90] Minyi Yue. „On the exact upper bound for the multifit processor scheduling algorithm“. In: *Annals of Operations Research* 24.1 (1990), S. 233–259. DOI: 10.1007/BF02216826.
- [YZ95] Minyi Yue und Lei Zhang. „A Simple Proof of the Inequality $MFFD(L) \leq 71/60OPT(L) + 1, \forall L$, for the MFFD Bin-Packing Algorithm“. In: *Acta Mathematicae Applicatae Sinica* 11.3 (1995), S. 318–330. DOI: 10.1007/BF02011198.

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Ich versichere zudem, dass ich diese Arbeit in keinem anderen Prüfungsverfahren eingereicht habe.

Außerdem versichere ich, dass die beim Prüfungsamt eingereichte gedruckte Fassung dieser Arbeit mit der eingereichten digitalen Fassung übereinstimmt.

Bei der Anfertigung dieser Arbeit habe ich KI-gestützte Werkzeuge, insbesondere große Sprachmodelle, als Hilfsmittel eingesetzt. Sie dienten der Literatur- und Quellenrecherche, der Überprüfung von Zitaten und bibliografischen Angaben, der sprachlichen Überarbeitung einschließlich Vorschlägen zu Formulierung und Gliederung sowie dem Auffinden inhaltlicher, formaler und typografischer Fehler. Darüber hinaus unterstützten sie mich beim Schreiben von $\text{\LaTeX}$ -Code, bei der Implementierung der Benchmark-Pipelines in Python sowie bei der Erstellung der Skripte, welche die Tabellen und Abbildungen aus den Messdaten erzeugen. Die untersuchten Algorithmen habe ich selbst implementiert.

Die mathematischen Konstruktionen und Beweise sowie die Analysen der empirischen Tests sind meine eigene Leistung, ebenso die Auswahl und Anordnung der dargestellten Ergebnisse. Sämtliche Ausgaben KI-gestützter Werkzeuge habe ich geprüft, bei Bedarf überarbeitet und in eigener Verantwortung übernommen. Für den Inhalt dieser Arbeit trage ich allein die Verantwortung.

Ort, Datum

Unterschrift